



Master Thesis

Efficient Context Management in Large Language Models to Mitigate Attention Decay Using Human Memory-Inspired Prioritization

Submitted by:	Poonam Dashrath Paraskar
First examiner:	Prof. Dr. Andreas Pech
Second examiner:	Prof. Dr. Peter Nauth
Supervisor:	Mr. Indraneel Pole
Date of start:	06.06.2025
Date of submission:	05.01.2026

Statement

I confirm that I have written this thesis on my own. No other sources were used except those referenced. Content that is taken literally or analogously from published or unpublished sources is identified as such. The drawings or figures of this work have been created by me or are provided with an appropriate reference. This work has not been submitted in the same or similar form to any other examination board. AI tools such as ChatGPT were used only to understand complex concepts, and GitHub Copilot was used to assist in understanding existing code and debugging steps; the thesis content and final work are my own, and all sources are correctly cited.

05.01.2026

A handwritten signature in blue ink, appearing to read 'Dhaskar', is written over a horizontal line.

Date, Signature of the Student

Abstract

Large Language Models (LLMs) perform well on many natural language processing (NLP) tasks; however, they struggle to handle very long inputs because they tend to forget earlier information as the input size increases. This phenomenon is known as attention decay, and it can cause problems in tasks like multi-hop question answering, multi-turn conversations, and long-document summarization, leading to incorrect reasoning, missing information, and excessive attention to recent tokens(information).

This thesis examines this limitation as an opportunity to add human-like memory prioritization to LLMs. It explores ways to organize long-context inputs into more useful, easy-to-retrieve parts, drawing on ideas from episodic memory, event segmentation, and salience-based recall. The Episodic Memory LLM (EM-LLM) architecture is the main approach, segmenting input into events, saving them as small memory blocks, and finding relevant episodes using semantic similarity and surprisal. EM-LLM does not require additional training and works with various models, yet it performs well compared to other long-context methods such as InfLLM and retrieval-augmented generation (RAG).

The main technical contribution of this thesis is the Cross-Event Reasoner (CXR), which allows the model to reason directly across retrieved events. Instead of treating episodes as separate blocks, CXR lets the model focus on connections between events, combine related events into stronger join representations, and adjust their importance based on the current query before adding them back into the model. The thesis extends the original framework with CXR, builds an EM-LLM pipeline that follows this design, and tests it on standard long-context benchmarks. The results show promising ways to improve cross-event prioritization inspired by human memory, with modest but steady improvements, and also point out the remaining limits of EM-LLM.

Content

1	Introduction	8
1.1	Motivation	9
1.2	Thesis Structure and Overview	10
2	Theoretical Background	12
2.1	Large Language Models and Transformer	12
2.1.1	Self-Attention and Context Windows	13
2.1.2	Attention Decay and Context Forgetting	15
2.1.3	Long-Range Context Benchmarks	16
2.2	Human Memory and Episodic Memory	17
2.2.1	Cognitive Models of Human Memory	18
2.2.2	Serial Position Effects and Surprise	19
2.2.3	Event Segmentation in Human Cognition	20
2.3	Memory-Augmented Neural Architectures	21
2.3.1	Retrieval-Augmented Generation (RAG)	22
2.3.2	Memory Networks and Memorizing Transformers	24
2.3.3	Efficient and Long-Context Transformers	26
2.3.4	Summary and Positioning	27
2.4	EM-LLM: Human-Inspired Episodic Memory for LLMs	28
2.4.1	Surprisal-Based Event Segmentation	29
2.4.2	Graph-Theoretic Boundary Refinement	29
2.4.3	Episodic Memory Storage and Hierarchy	30
2.4.4	Retrieval and Multi-Stage Attention	30
2.4.5	Limitations of EM-LLM:	31
2.5	Cross-Event Reasoning and Event-Level Modeling	31
2.5.1	Event Representations and Event Graphs	32
2.5.2	Multi-Hop Reasoning Across Events	32
2.5.3	Design Principles for Cross-Event Reasoners	32
3	Requirement Analysis	33
3.1	General Objectives	33
3.2	Problem Context and Use Cases	34
3.2.1	Long-Document Summarization:	35
3.2.2	Multi-Turn Conversational Memory	35
3.2.3	Multi-Hop Question Answering and Narrative Understanding	35
3.3	Functional Requirements	35
3.3.1	Baseline LLM and EM-LLM Restoration	36
3.3.2	Cross-Event Reasoner Integration	37
3.3.3	Evaluation of results	37
3.4	Non-Functional Requirements	37
3.4.1	Model-Agnostic and Training-Free Design	38
3.4.2	Efficiency and Scalability (Infinite Context)	38
3.4.3	Reproducibility, GitHub Project, and Open-Source Compatibility	38
3.5	Dataset and Benchmark Requirements	38
3.5.1	Multi-Hop Reasoning Datasets (HotpotQA, 2WikiMultihopQA)	38
3.5.2	Narrative and Summarization Benchmarks (NarrativeQA, QMSum)	39
3.5.3	Long-Context Evaluation Suites (LongBench Tasks)	39
3.6	Time Frames and Milestones	39

3.7	Target State/Objectives	41
4	System Design and Implementation	42
4.1	Overall System Architecture	42
4.1.1	High-Level System Pipeline	42
4.1.2	Software Architecture and Module Overview	44
4.2	Non-Episodic Baseline Pipeline: Mistral Bench	44
4.2.1	Design Goals and Role in the Thesis	44
4.2.2	Dataset Handling and Schema Mapping (Arrow/JSON/JSONL)	45
4.2.3	Ollama-Mistral Inference and Prompt Construction	47
4.2.4	Output Formats and EM-LLM-Compatible Results	49
4.2.5	Steps to run the tool:	51
4.3	EM-LLM Baseline and Restoration	52
4.3.1	Architectural Overview of the EM-LLM Baseline	52
4.3.2	Re-Implementation of Surprisal-Based Event Segmentation	54
4.3.3	Graph-Based Memory Structuring Using Topological Metrics (Modularity, Conductance)	56
4.3.4	Contiguity Buffer and Episodic Memory Management	58
4.3.5	Interaction Between Local Context Window and Global Episodic Memory	59
4.3.6	Inference-Time Memory Retrieval and Conditioning	62
4.3.7	Practical Constraints of Single-GPU Baseline Restoration	63
4.4	Cross-Event Reasoner (CXR) Design	64
4.4.1	Event Representation Pooling	65
4.4.2	Event-to-Event Multi-Head Self-Attention	65
4.4.3	Event Fusion and Event-Interaction Graph	66
4.4.4	Query-Conditioned Scoring	67
4.4.5	Final Event Reordering and Pruning	67
4.5	Integration of CXR into EM-LLM Pipeline	68
4.5.1	Placement in the Retrieval Loop	68
4.5.2	Block Representation and Query Construction	69
4.5.3	Execution Strategy and Memory Management	70
4.5.4	Interaction with Multi-Stage Attention	71
4.6	Implementation Details	71
4.6.1	Model and Tokenizer Configuration	71
4.6.2	Data Preprocessing and Episodic Encoding	72
4.6.3	Decoding Loop and Performance Metrics	72
4.6.4	Logging, Monitoring, and Attention Visualization	73
4.6.5	Experiment Orchestration and Execution	73
4.7	Development Environment	74
4.7.1	Hardware Configuration	74
4.7.2	Software Stack and Dependencies	74
4.7.3	Codebase Organization and Version Control	75
5	Evaluation and Results	77
5.1	Experiment Setup	77
5.1.1	Evaluation Scenarios and Tasks:	77
5.1.2	Baselines: LLM Without Memory vs EM-LLM vs EM-LLM Cross Events Reasoner	77
5.1.3	Evaluation Metrics	78
5.2	Evaluation of LLM Without Episodic Memory	78
5.2.1	Multi-Hop Question Answering (HotpotQA, 2WikiMultihopQA)	79
5.2.2	Narrative and Summarization Tasks	81
5.2.3	Long-Context Benchmarks	82
5.2.4	Cross-Dataset Analysis and Key Findings	82
5.3	Evaluation of LLM With Episodic Memory (EM-LLM)	83
5.3.1	Multi-Hop Question Answering	84

5.3.2	Narrative and Summarization Tasks	85
5.3.3	Long-Context Benchmarks	86
5.3.4	Cross-Dataset Analysis	86
5.3.5	Summary and Key Findings	88
5.4	Evaluation of LLM With Episodic Memory with Cross-Event Reasoning (EM-LLM + CXR)	89
5.4.1	Multi-Hop Question Answering	90
5.4.2	Narrative and Summarization Tasks	90
5.4.3	Long-Context Benchmarks	91
5.4.4	Cross-Dataset Analysis	92
5.4.5	Summary and Key Findings	95
5.5	Comparison of Three Variations: Non-Episodic Baseline, Episodic Memory, and Episodic Memory + CXR	96
5.5.1	Performance Analysis by Task Type	96
5.5.2	Visualizing the Architecture Impact	97
5.5.3	Summary	97
6	Conclusion and Future Work	98
6.1	Summary of Contributions	98
6.2	Answers to Research Questions	98
6.3	Limitations of the Proposed Approach	99
6.4	Future Work: More General Memory-Reasoning Systems	101
7	Abbreviations	103
8	References	106

List of Figures:

Figure 2.1. Transformer Architecture	13
Figure 2.2. Scaled dot-product and multi-head self-attention.....	14
Figure 2.3. Conceptual illustration of attention decay	16
Figure 2.4. Long-context benchmark overview.....	17
Figure 2.5. The multi-store model of human memory (sensory, short-term/working, and long-term memory) ...	18
Figure 2.6. Serial position curve.....	19
Figure 2.7. An illustration of the unified theory of event representation and segmentation.	20
Figure 2.8. High-level taxonomy of memory-augmented architectures: RAG uses an external vector store; architectural memory integrates explicit memory modules; efficient transformers modify attention for more extended sequences	22
Figure 2.9. RAG Architecture	23
Figure 2.10. Block diagram for RAG	24
Figure 2.11. Schematic view of memory-augmented neural architectures such as End-to-End Memory Networks and Memorizing Transformers. A controller (e.g., a transformer) reads from and writes to an explicit memory of slots or stored activations.	25
Figure 2.12. Illustration of the Transformer-XL model with a segment length of 4	26
Figure 2.13. Comparison overview of RAG, Architectural Memory, and EM-LLM.....	27
Figure 2.14. Architecture of memory formation and retrieval in each LLM layer.	28
Figure 3.1. High-level objectives of the thesis	33
Figure 3.2. Use cases targeted by the framework	34
Figure 3.3. Restored the EM-LLM pipeline and the non-episodic baseline tool.....	36
Figure 3.4. GitHub Projects Kanban Board.....	39
Figure 3.5. Gantt chart for the thesis	40
Figure 4.1. Overall System Pipeline.....	43
Figure 4.2. High-level pipeline for non-episodic test.....	45
Figure 4.3. Data Ingestion & normalization	47
Figure 4.4. Output results and plots.....	50
Figure 4.5. The streamlit UI interface	52
Figure 4.6. Architectural Overview of the EM-LLM.....	53
Figure 4.7. Surprisal-Based Boundary Detection Pipeline	55
Figure 4.8. Graph-based refined Boundary Detection Pipeline	57
Figure 4.9. Hierarchical Episodic Memory Management and Retrieval	59
Figure 4.10. Attention Computation Using Sliding Window and Global Memory	61
Figure 4.11. CUDA Out of memory error	63
Figure 4.12. Conceptual CXR architecture.....	64
Figure 4.13. Sample Event Interaction Graph	66
Figure 4.14. Workflow diagram of Cross Events Reasoner	69
Figure 5.1. Aggregate Metrics	79
Figure 5.2. F1 Score Distribution	80
Figure 5.3. F1 score across the long-bench dataset	84
Figure 5.4. EM-LLM Performance by category	87
Figure 5.5. Performance comparison of EM-LLM and EM-LLM-CXR.....	92
Figure 5.6. Perplexity changes vs F1 score change	94
Figure 5.7. Heatmap showing performance progression across three variations.....	97

1 Introduction

Applications such as document summarization, conversational assistants, knowledge extraction systems, and multi-hop question answering are all built using large language models (LLMs), which have become a central part of modern natural language processing. They are attractive in both research and industry because they can generate coherent text and follow complex instructions. However, as model sizes grow and context windows expand, LLMs still struggle to use information that appears far earlier in long inputs reliably. Although those tokens are technically within the context window, the models often “forget” earlier parts of the input and place more emphasis on more recent tokens as the sequence length increases.

This phenomenon, often called attention decay or context forgetting, becomes clearly visible in practice. For example, when an LLM is asked to summarize a lengthy report, it usually focuses on the later sections and overlooks important definitions or assumptions introduced at the beginning. In multi-turn conversations, as new exchanges are added, earlier user instructions such as style guidelines, safety constraints, or personal preferences are gradually ignored, leading to replies that contradict or override previous agreements. Similarly, when the model answers a long-context query, relevant information that appeared many pages earlier may be effectively invisible to it. [1]

These errors arise from how transformers compute self-attention and are not just based on a few isolated examples. Attention weights are obtained by taking dot products between the query and key vectors and then applying a SoftMax function. As the sequence length increases, representations of distant tokens become less aligned, their dot products decrease, and the SoftMax further down-weights them. Consequently, the model develops a strong recency bias: later tokens in the sequence receive most of the attention, while earlier tokens contribute less. Long-context variants with larger windows can access more tokens, but they do not always use them evenly, leaving them vulnerable to this problem.[2]

Human memory works in a very different way. Instead of keeping experiences as a single, continuous stream, human memory naturally breaks them into separate episodes and stores them as episodic memories. Psychological studies show that surprising, emotionally important, or goal-related events are remembered more strongly than routine details, and that humans are especially good at recalling the beginnings and endings of episodes, a phenomenon known as the serial position effect. When reasoning, humans rarely replay every detail; they remember only a few key moments and mentally link them together to reconstruct the story or find an answer. This difference has motivated a growing body of research aimed at improving LLMs by leveraging memory systems inspired by human memory.

A more recent approach that directly mimics human episodic memory is the Episodic Memory LLM (EM-LLM). EM-LLM uses surprisal-based boundary detection to break long text streams into separate events. It then refines these boundaries using graph-theoretic metrics and stores compact representations of each event as “memory cards.” When a query comes in, EM-LLM retrieves relevant events based on temporal structure and similarity and feeds them back into the model so the LLM can reason over them. Because this architecture is training-free, model-agnostic, and can handle extremely long context, it offers a practical way to extend context without retraining the base LLM.[3]

However, EM-LLM also has several limitations. It requires substantial computation to compute surprisal and graph metrics for every token. In the retrieval step, it mainly relies on semantic similarity and temporal proximity; task-related aspects such as novelty and redundancy are not explicitly considered. A key point of this thesis is that EM-LLM treats each retrieved event as a separate block, which gives the model the right pieces of memory but does not directly help link them together. The base transformer is still affected by attention decays. It is fully responsible for connecting, combining, and reasoning over these pieces in multi-hop questions or stories where the evidence is spread across multiple events.

The gap between how humans use episodic memory and how current memory-augmented LLMs work is the primary motivation for this thesis. It examines whether human-inspired memory prioritization can not only retrieve relevant episodes but also support cross-event reasoning over them. This work intro-

duces the Cross-Event Reasoner (CXR). This lightweight module explicitly models connections between retrieved events and extends the original segmentation and retrieval pipeline built on the EM-LLM framework. To evaluate the proposed system, it systematically compares three setups on long-context benchmarks that require multi-hop reasoning, narrative understanding, and reliable use of distant information: the EM-LLM baseline, EM-LLM enhanced with CXR, and an LLM without episodic memory.

1.1 Motivation

Many applications need models that can track ongoing projects, follow instructions across multiple steps, and manage extensive document collections. Users want systems that remember what happened earlier, not ones that forget anything beyond a few thousand tokens. They expect models to keep a clear picture of the story, connect distant facts, and recall earlier limits. Until attention decay is solved, long-context LLMs act more like advanced short-term memory than reliable reasoning tools.

From a research perspective, there is a gap between how machines model memory and what we know about human memory. Cognitive science shows that people break experiences into separate events, keep only some details, and recall them in context when needed.[4]

In contrast, many current LLM methods treat memory in one of three ways:

1. A larger attention window,
2. An external retrieval system with fixed chunks, or
3. Learned architectural changes inside the transformer.

These methods help models handle more data, but they do not capture the way humans link memories through events and associations over time.

Looking closer, many long-context models that claim to use long histories rely on only a small part of that history for their predictions. Earlier tokens often have little impact on the final output, even when they remain in the context window.

This raises the question of whether current long-range models really use the extra context or work around it. By comparing a memory-augmented LLM with a standard LLM on the same datasets and controlled inputs, we can better see the effect of episodic memory.[5]

EM-LLM is a useful baseline because it matches the idea of human episodic memory and is practical to use. It breaks long sequences into clear events, stores them compactly, and retrieves them when needed. However, as shown in earlier work and this thesis, EM-LLM still struggles with tasks that require information from multiple events. The model often finds the right pieces but cannot always combine them into a single explanation. This suggests that memory-augmented models need a special reasoning step that works across events, not just better retrieval.[6]

To solve this problem, this thesis introduces the Cross-Event Reasoner (CXR). Instead of just joining the retrieved episodes and letting the base transformer handle everything, CXR summarizes events into compact vectors, uses self-attention to find relationships, merges related events, and then re-scores them based on the query. This creates a smaller, more focused, and more useful memory set. Importantly, CXR does this without retraining the main LLM, as it is a lightweight module that operates on hidden states and can be added to various models.

Besides scientific interest, there is also a practical engineering reason. Many organizations lack the resources or permission to use large, custom transformer models or to fine-tune large, proprietary models.[7] Instead, they rely on open-source models or API-based LLMs, which only allow limited changes and restricted access to internal states. These limits make a human-inspired, training-free approach especially attractive for improving long-context reasoning without retraining. The method in this thesis, cross-event reasoning, prioritization, and episodic segmentation, is designed to work with any model and fit these real-world settings.

All these factors lead to the main research questions of this thesis:

Research Question 1: When standard LLMs are applied to long-context tasks such as multi-hop question answering, document summarization, and narrative understanding, what does attention decay look like in practice?

Research Question 2: Compared to a non-episodic baseline on the same datasets, to what extent does adding a human-inspired episodic memory system like EM-LLM reduce context forgetting and improve performance?

Research Question 3: Does explicit cross-event reasoning with the proposed Cross-Event Reasoner (CXR) provide additional benefits beyond EM-LLM alone, especially for tasks where the relevant evidence is distributed across multiple events?

By running controlled experiments that compare models with and without episodic memory on the same inputs, this thesis aims to show how much of the long-context problem can be solved with human-inspired memory prioritization and how much larger changes to model design are still needed.

1.2 Thesis Structure and Overview

The thesis explores theoretical foundations, practical design, and evaluation and results. Each chapter builds on the previous chapter so that the user can follow the path from the core problem to the final findings and conclusions.

Chapter 2-Theoretical Background: This chapter provides the theoretical and empirical basis for the work. It introduces large language models and transformer architecture, focusing on self-attention, context windows, and the problem of attention decay in long sequences. It then reviews cognitive theories of human memory, in particular episodic memory, event segmentation, and serial position effects. Building on this, the chapter surveys existing methods for long-context modeling, including efficient/long-context transformers, retrieval-augmented generation, and memory-augmented architectures.

Chapter 3-Requirements Analysis: Chapter 3 formalizes the core problem of the thesis and defines the requirements for the proposed solution. It explains the overall goals of using human-inspired memory mechanisms to reduce attention degradation in LLMs. It grounds them in specific cases, such as multi-turn dialogue, long-document summarization, and multi-hop question answering. The chapter then sets out both functional and non-functional requirements, including model-agnostic design, efficiency, and reproducibility, as well as the restoration of the EM-LLM pipeline, integration of the Cross-Event Reasoner, and evaluation tooling. It also specifies datasets and benchmark requirements, describes the intended target state of the system, and outlines the main project milestones and timelines.

Chapter 4-System Design and Implementation: Chapter 4 describes the architecture and implementation of the proposed system in detail. It begins with an overview of the software architecture and the data flow from raw text input to model predictions. The chapter then explains how the original EM-LLM baseline was reconstructed and extended, including contiguity handling, graph-based boundary refinement, surprisal-based event segmentation, and episodic memory storage and retrieval. Next, it presents the design of the Cross-Event Reasoner (CXR). It shows how it incorporates query-conditioned scoring, event fusion, event representation pooling, and event-to-event self-attention into the EM-LLM retrieval loop. Finally, it covers practical implementation aspects, including the hardware and software environment, model and tokenizer settings, decoding and perplexity computation, logging, and visualization support.

Chapter 5-Evaluation and Results: Chapter 5 presents the experimental setup and results. It describes the preprocessing steps, the evaluation metrics (including F1, exact match, ROUGE, and perplexity), and the selected benchmarks for long-context reasoning. The chapter then introduces the systems being compared: a baseline LLM without episodic memory, the EM-LLM baseline, and the extended EM-LLM with CXR. After reporting quantitative results for the different tasks, it presents ablation studies that analyze the contribution of each component of CXR. In addition, the chapter includes case studies

and qualitative analyses that illustrate how cross-event reasoning and episodic memory affect the practical use of distant context.

Chapter 6-Conclusion and Future Work: Chapter 6 summarizes the main findings and contributions of the thesis. It revisits the research questions and discusses how well the proposed episodic memory pipeline and Cross-Event Reasoner reduce attention decay and improve long-context reasoning. It then outlines directions for future work, such as tighter integration with external knowledge sources, partial end-to-end learning of the memory pipeline, and extending the ideas to other modalities and application settings. The limitations of the current approach are also critically discussed.

Chapter 7-Abbreviations: This chapter lists the abbreviations used throughout the thesis to maintain consistency and readability.

Chapter 8-References: Chapter 8 contains the bibliographic references cited in the thesis, formatted according to the IEEE citation style.

2 Theoretical Background

This chapter discusses the theoretical grounds of the thesis by beginning with a summary of transformer architecture and large language models (LLMs), emphasizing context windows, self-attention, and the processes that lead to attention decay. It then discusses how this declination manifests as context-forgetting and why it is detrimental to long-range reasoning. Finally, the chapter briefly discusses the primary benchmarks for assessing long-context understanding.

These principles are essential for illustrating why additional memory mechanisms, such as EM-LLM and the proposed Cross-Event Reasoner (CXR), are helpful, and for constructing the vocabulary used later to describe and evaluate the system.

2.1 Large Language Models and Transformer

Neural networks trained on massive text corpora to predict the next token in a sequence are known as large language models (LLMs). When model size, training data, and context duration are scaled up, they achieve strong performance across numerous tasks, such as question answering, dialogue, summarization, and code generation. Transformers differ from preceding recurrent systems in two fundamental ways. Initially, rather than processing tokens one at a time, they process the whole input sequence in parallel. Then, they adopt a self-attention method that allows each token to attend to all other tokens in the sequence, rather than simply transferring information through a recurrent hidden state. This architecture leads to more stable training and makes it easier to capture long-range dependencies in sequences of modest length.[1]

The transformer layer has three significant components: multi-head self-attention, where each token computes attention scores over all other tokens via learnt projections known as queries, keys, and values; and position-wise feed-forward networks, which perform non-linear changes independently for each token embedding. Layer normalization and residual connections facilitate gradient flow over deep stacks of layers and stabilize training.

Typically, LLMs combine many of these layers with positional encodings and token embeddings. In decoder-only architectures, which are often used for generative LLMs, tokens are processed from left to right with a causal mask that prohibits the model from looking into the future.

During inference, the model is given an input context of size, for example, 4k, 32k, or 128k tokens and then generates new tokens one at a time, using internal key-value caches so it does not need to recompute attention across the entire sequence.

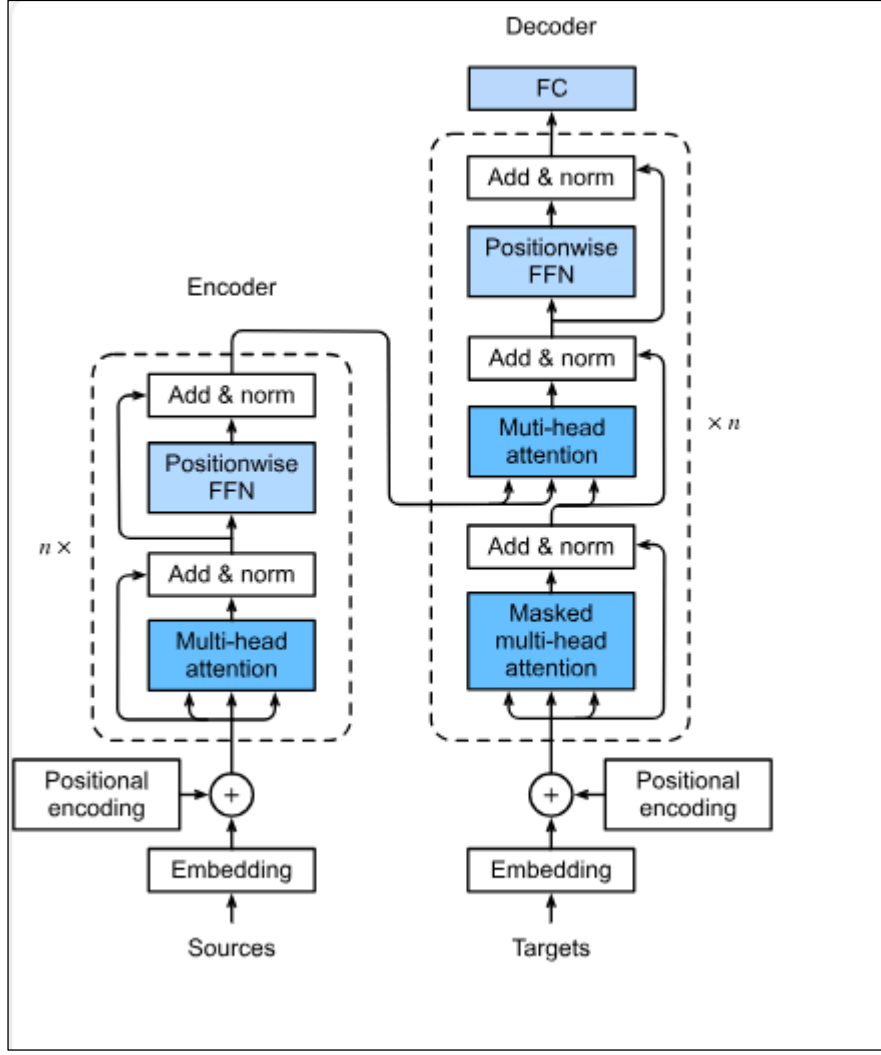


Figure 2.1. Transformer Architecture[8]

Figure 2.1 shows a high-level transformer architecture: embeddings and positional encodings feeding into stacked self-attention and feed-forward blocks. This figure helps situate where self-attention and the context window reside within the overall model.

2.1.1 Self-Attention and Context Windows

Self-attention is the primary mechanism that enables transformers to share information across different positions. For each token at position i , the model creates a query vector q_i , a key vector k_i , and a value vector v_i from the token's current hidden state using learned linear layers. The attention weight from token i to token j is calculated using the scaled dot product.

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right) [9]$$

where d_k is the dimensionality of the key vectors. The updated representation of the token i is then a weighted sum of value vectors

$$h'_i = \sum_j \alpha_{ij} v_j [9]$$

In multi-head attention, the process is repeated several times, each time using different learned projections. The resulting vectors are concatenated and fed into a feed-forward network.

Because every token in a layer attends to all other tokens in that layer, self-attention has quadratic complexity in the sequence length n for both time and memory. For short and medium sequences, this remains manageable. As n grows, however, the cost increases quickly, and in practice, models are deployed with a fixed context window that caps the number of tokens that can be processed at once.

Modern LLMs claim to handle context windows ranging from thousands to hundreds of thousands of tokens. But the formal context length is not the same as the effective context length, which is how far tokens still meaningfully affect predictions. Studies show that tokens far from the current position have tiny attention weights and gradient signals. As a result, earlier tokens have little effect on the model's output, even when they remain in context.[10]

Absolute or rotational positional embeddings are often used to represent each token's position within a sequence. These encodings help the model tell tokens apart based on their positions, but they also make the model focus more on nearby tokens. Tokens that are close together tend to pay more attention to each other than to those that are far apart, since their position encodings match better. In practice, many systems use methods such as block-wise processing or sliding-window attention to save memory, which makes it even less likely that distant tokens will interact directly.[10]

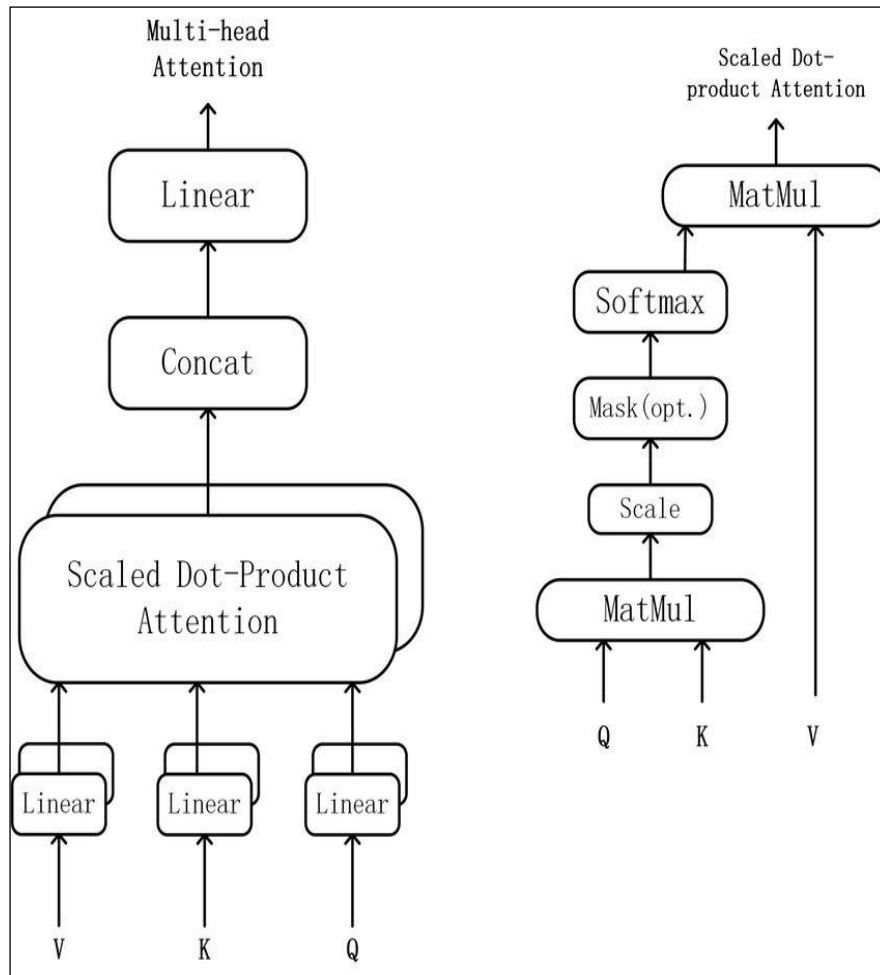


Figure 2.2. Scaled dot-product and multi-head self-attention[1]

Figure 2.2 illustrates the scaled dot-product attention mechanism. Queries (Q), keys (K), and values (V) are first projected using separate linear layers before being fed into the attention module. Inside scaled

dot-product attention, Q is multiplied by K^T , the result is scaled by $1/\sqrt{d_k}$, optionally masked (as in causal decoder-only models), and passed through a SoftMax to obtain attention weights. These weights are then multiplied by V to produce the head output.[1]

The left panel shows how multiple such heads are concatenated and fed through a final linear projection to form multi-head attention. The right panel expands the internal operations of a single attention head and highlights the role of the causal mask in restricting each query to attend only to earlier positions.

2.1.2 Attention Decay and Context Forgetting

Transformers are designed so that any token can attend to any earlier token, but they rarely use information from far back in the sequence. Attention decay means that, as sequences get longer, earlier tokens receive less attention and have less effect on predictions.[11]

There are several reasons for this behavior:

1. Early tokens experience representation drift:
When the model processes very long sequences, the hidden states of early tokens can change so much that they no longer match well with later query vectors. As a result, the dot products $q_i \cdot k_j$ for distant tokens become small, even if the content is essential.[2]
2. SoftMax amplifies minor differences:
The SoftMax function converts similarity scores into probabilities. If recent tokens have slightly higher scores than older ones, SoftMax gives most of the attention to the recent tokens and reduces the influence of distant tokens.[12]
3. Training dynamics and where loss is applied:
In many training setups, the predictive loss is calculated for later positions in the sequence. This causes the model to focus on short-range patterns near the end, which strengthens a bias toward recent tokens. Just making the context window longer at inference does not fix this learned behavior.[13]
4. Effects of positional encoding and architectural shortcuts:
Positional encoding methods and sparse attention techniques, such as sliding windows or dilated patterns, are typically designed to focus on nearby tokens. This makes it even harder for the model to use information from far away unless a retrieval or memory system brings those tokens back into focus.[14]

For users, attention decay looks like the model forgetting earlier context. Common problems include that when summarizing long documents, the model often focuses too much on the end and misses key definitions or assumptions from the start. In multi-turn conversations, instructions given at the beginning, such as “answer concisely” or “use a formal tone,” are often forgotten as the conversation progresses. In multi-hop reasoning tasks, the model may encounter all relevant facts in the context, but it struggles to connect them when they are far apart.[15]

These problems show that standard transformers act more like strong working memory systems with a limited range, rather than reliable long-term memory systems. Because this is different from how human memory works, methods like EM-LLM have been developed. These methods break experiences into episodes and store them, bringing them back when the model’s attention can no longer keep track of them.

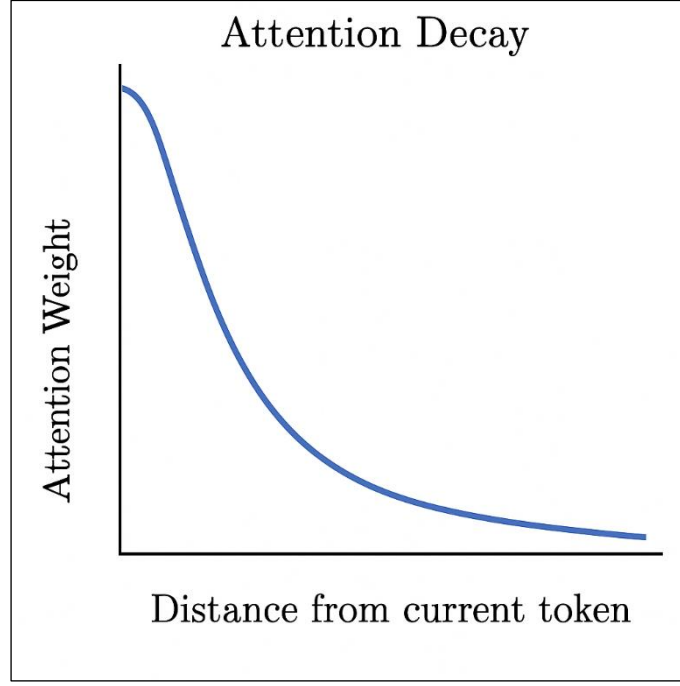


Figure 2.3. Conceptual illustration of attention decay

Figure 2.3 provides a schematic view of this behavior, showing how a token’s attention to earlier positions declines as the sequence grows longer.

2.1.3 Long-Range Context Benchmarks

Researchers have developed metrics that specifically assess how effectively LLMs handle complex situations, thereby going beyond anecdotal evidence. These benchmarks differ in task type, length, and domain, but they all aim to determine whether models can exploit information that appears distant from the query rather than simply adding more tokens to the input.

One of the most well-known long-context benchmarks is LongBench. With an average context length of several thousand words, it is a bilingual, multi-task suite that comprises summarisation, few-shot learning, code completion, single-document and multi-document question answering, and synthetic reasoning tasks. Evaluations reveal that as input length increases, even powerful commercial models experience performance declines, particularly on tasks that require reasoning across widely separated passages of text.

With average context durations reaching 100,000 tokens, ∞ -Bench and InfiniteBench aim to push into even more severe circumstances. It blends realistic and synthetic activities, and its design requires models to retain and integrate information throughout lengthy text segments rather than only retrieving a few sentences. Current long-context models continue to struggle with genuinely "super-long" contexts, according to InfiniteBench results.[16]

Long dialogues, code repositories, and structured data with contexts ranging from 8k to millions of tokens are all covered by more recent suites such as LongBench v2 and LoongBench, along with associated long-context evaluations. These benchmarks show attention decay in a controlled manner: performance deteriorates when pertinent evidence is relocated farther into the context, or when the context length increases while task difficulty remains constant.[17]

These benchmarks have three functions in this thesis. On long-range assignments, they highlight the shortcomings of a simple transformer-based LLM. It offers a fair comparison of the extended EM-LLM+CXR system, an EM-LLM configuration, and a baseline LLM under the same circumstances.

To determine whether episodic memory and cross-event reasoning generalise across domains, it provides a variety of tasks, such as multi-hop question-answering and narrative comprehension.

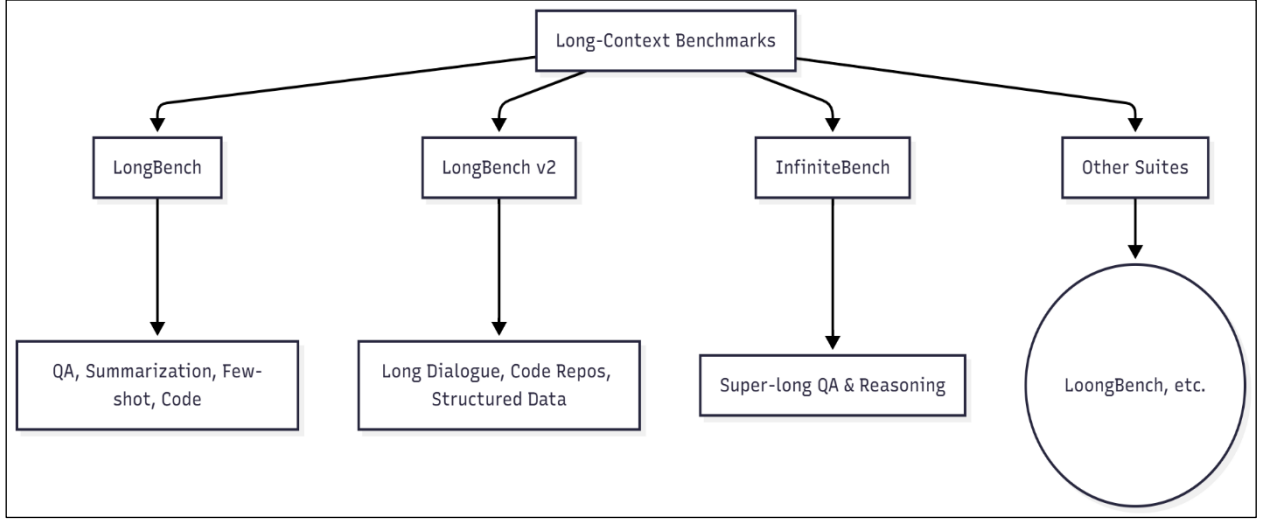


Figure 2.4. Long-context benchmark overview

2.2 Human Memory and Episodic Memory

Transformers and large language models read text as a long string of tokens, spreading their attention across the whole sequence. Human memory is different. People organize and condense information, using special memory systems to find what they need instead of remembering every detail in order. Learning about these systems shows why humans can reason over long periods, whereas transformer-based models often struggle to do so.[1]

Cognitive psychology distinguishes between working memory and long-term memory. Working memory helps with tasks like remembering a phone number or following short instructions by holding a small amount of information for a few seconds. Long-term memory, which includes episodic memory like personal experiences and semantic memory like facts and general knowledge, stores information much longer, sometimes for minutes, days, or even years.[18][19]

Episodic memory is significant for experiences like meeting a friend at a café, going to a lecture, or reading an article in the morning, which are stored as events. Instead of keeping raw sensory recordings, people save these events as organized, condensed memories that retain key details such as who was there, where it happened, and why it mattered. When reasoning later, people do not recall every detail. Instead, they remember a few relevant examples and connect them in their minds.[20]

Transformer-based LLMs work very differently; they have a large but fragile working memory, limited to the current context window. Once tokens leave that window, they cannot be retrieved. Even within the window, attention fades, so earlier tokens have little effect on new predictions. In contrast, humans can remember past events, especially if those events were important, emotional, or surprising.[21]

This thesis focuses on three main aspects of human episodic memory: First, we organize our experiences into separate events. Instead of remembering everything as one long stream, we break it into meaningful units called events. Then, we often forget some details of an event. People usually remember the beginnings, endings, and surprising parts more easily, often because of where they appear or how noticeable they are. Third, memories are connected by associations. Remembering one event usually brings up related ones, especially if they share people, situations, or topics. These links help us think more deeply.[22][23] These concepts are used in the EM-LLM framework and the Cross-Event Reasoner (CXR). CXR shows how recalled events are connected to the imitation of human memory. EM-LLM

uses graph-based methods and surprisal signals to split text into events and save them as separate memory blocks.

2.2.1 Cognitive Models of Human Memory

Traditional cognitive theories view memory as a system composed of several interconnected parts. A common way to describe these parts is: Sensory memory stores raw information for a few milliseconds to a few seconds. Working memory, also called short-term memory, lets us use a small amount of information while we solve problems or talk with others. Long-term memory stores information for much longer periods.[24] Within long-term memory, semantic and episodic memory operate independently. Episodic memory stores detailed experiences, such as "I visited Paris last summer and saw the Eiffel Tower at night." Semantic memory stores general facts, such as "Paris is the capital of France."

Episodic memories are usually organized by who was there, what happened, where it took place, and when. These memories help us feel like we are reliving the event and include details about time and place. However, not all moments are stored in the same way: What we notice and remember depends on where we focus our attention. Information that seems useful for future decisions is more likely to stay in our memory. We remember surprising or emotional events more easily because strong feelings and surprises help memories last.[20]

Episodic memories depend on context and connections; not just how recent events are. For example, if someone asks, "Why did I choose that route yesterday?" the brain connects memories with similar places, people, or goals to answer. In contrast, transformers do not deliberately create chains of related events. They focus on individual tokens, and their attention fades as the distance between tokens increases.

This thesis clearly shows that humans do not process experiences as a continuous stream of tokens. Instead, we break them into episodes, store them in long-term memory, and recall them based on what matters, not just the order. The EM-LLM architecture is built on this idea by treating sections of text as whole events rather than single tokens and storing them as memory blocks rather than keeping all tokens active at once. Retrieving events based on how similar, important, or connected they are, not just how recent they are. The model builds on this by adding a reasoning layer over events, which reflects how humans compare, combine, and re-rank memories when answering complex questions.[25]

Humans usually remember the first and last items in a list better than those in the middle. When recall accuracy is plotted by list position, it forms a U-shaped curve: high at the beginning (primacy), low in the middle, and high again at the end (recency).[26]

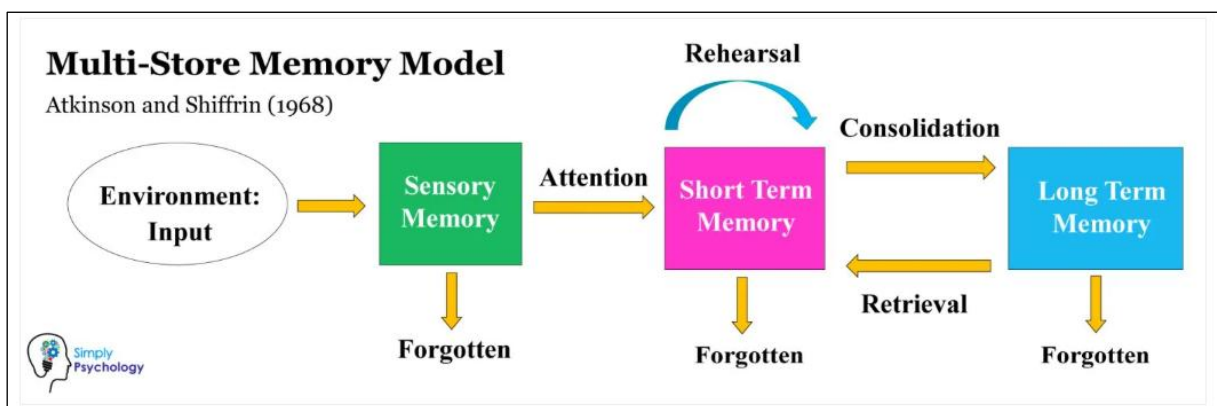


Figure 2.5. The multi-store model of human memory (sensory, short-term/working, and long-term memory)[27]

Figure 2.5 shows the multi-store memory model, which describes memory as a flow through three leading stores. Information first enters sensory memory from the environment, and most of it is quickly forgotten unless attention moves it into short-term memory. In short-term memory, rehearsal helps keep information active and can consolidate it into long-term memory. Long-term memory stores information

more permanently, and relevant items can later be retrieved from long-term memory into short-term memory. In contrast, unused information in any store may eventually be forgotten.

2.2.2 Serial Position Effects and Surprise

The serial position effect has two main components: the primacy effect, which is linked to deeper encoding, because early items are rehearsed more often and are more likely to be stored in long-term memory. The recency effect is usually explained by working memory, since the most recent items are still active and easy to recall without needing long-term storage.

In this thesis, the serial position effect is significant because it shows that human memory does not treat all parts of a sequence equally. Humans naturally focus on the beginning and end, while the middle is often less well remembered. This differs from models that assign equal weight to every token within a context window. It suggests that focusing on certain parts of input is a valuable feature of memory, not a flaw in the connected component. We are much more likely to remember experiences that run counter to our expectations. Greater memory capacity is another essential factor. We tend to remember things that go against our expectations. When something unexpected happens, it leaves a stronger memory trace and gets more attention. These are the moments we recall later, like an unusual comment in a meeting, a sudden twist in a story, or an odd result in an experiment: "before X happened" and "after X happened." EM-LLM has a similar concept: a signal for possible event boundaries is token-level surprisal, the extent to which a token is unexpected given the model's current expectations. Surprisal spikes indicate potential boundaries between occurrences.[3], [20]

To put it another way, EM-LLM converts two human memory intuitions into a tangible algorithmic form; EM-LLM turns two ideas from human memory into a real algorithm, significant consideration. The segmentation and storage of the input should be influenced by sudden changes or unexpected tokens (i.e., high surprisal). Like primacy and recency, the beginning and end of an episode are likely to be essential and deserve extra attention, and the architecture of the episodic memory pipeline. These ideas are present later in the thesis, both in how the Cross-Event Reasoner handles event boundaries and in the design of the episodic memory pipeline. The algorithm picks segments that are more likely to matter for long-term reasoning, instead of treating every token the same. This is much like how human memory focuses on beginnings, endings, and surprises.

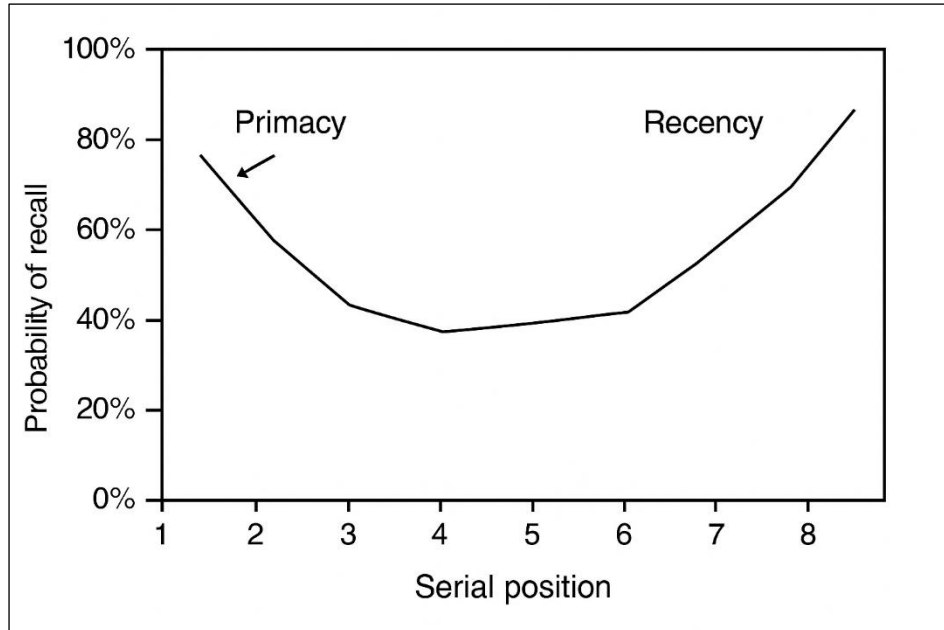


Figure 2.6. Serial position curve.

Figure 2.6. shows a Simple graph with serial position on the x-axis and recall probability on the y-axis, showing the U-shape (high at start and end). Items at the beginning of the list (primacy) and at the end (recency) show higher recall than items in the middle. Adapted from Practical Psychology and standard serial position effect demonstrations.[28]

2.2.3 Event Segmentation in Human Cognition

According to event segmentation theory, people naturally divide continuous experience into distinct events with distinct internal structures and smooth transitions between them. For example, going into a room, saying hello, and striking up a conversation are considered one event, whereas driving home and leaving the place are considered the start of another. Changes in location, people, objectives, or activities frequently align with these boundaries.[29]

Event borders are times of enhanced processing, according to behavioral and neuroscientific research. Prediction errors, changes in goals, or changes in sensory information cause our internal picture of "what is happening now" to be updated. These updates mark the conclusion of one episode and the beginning of the next. The following episodes eventually develop into the basic building blocks of episodic memory.[30]

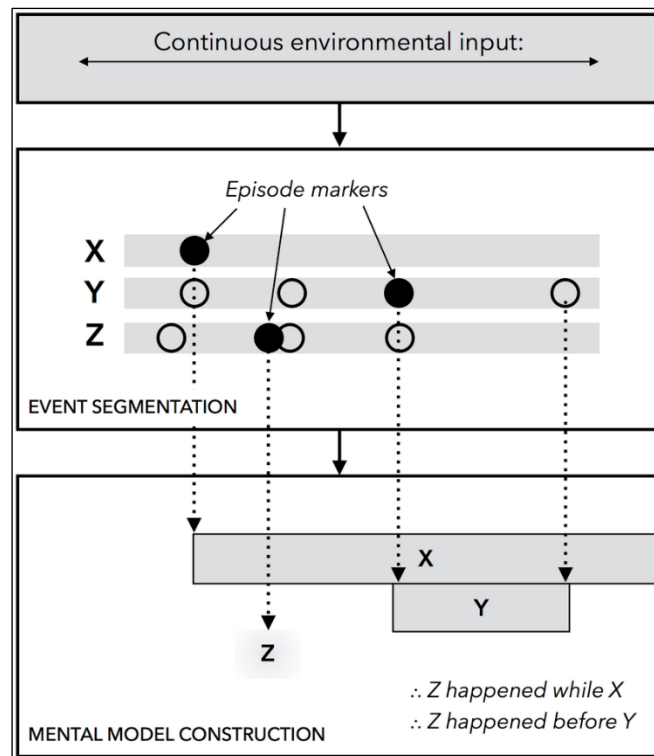


Figure 2.7. An illustration of the unified theory of event representation and segmentation.[31]

Figure 2.7. illustrates the unified theory of event representation and segmentation. Changes in continuous environmental input across a limited set of perceptual stimuli, denoted by X, Y, and Z in the diagram, are identified by the system's event segmentation component, which runs online and concurrently with other cognitive processes. A new episodic marker is created at the beginning of each stimulus, denoted by a black circle. A new episodic marker is also produced when a stimulus is neutralized. The markers are used to build a discrete-event model when the system is asked for information about temporal relationships. After that, the system scans the model to reach a conclusion.

This procedure and the EM-LLM architecture are directly compared in the requirement analysis for this thesis. Instead of using individual words, humans divide experience into episodic experiences, such as scenes, conversations, and acts, which serve as memory units. Long after the initial sensory stream has

passed, they can recollect these occurrences because they are stored in long-term memory. They recover events during reasoning, not just based on recentness, but also on context, similarity, and relevance.[3]

EM-LLM offers a computer equivalent to this technique:

1. Event segmentation based on surprise: High-surprisal tokens, which capture significant shifts, topic changes, or unexpected information, are considered provisional boundaries.
2. Graph-theoretic improvement: EM-LLM creates activation adjacency graphs over token representations because raw boundaries can be noisy. It then uses graph metrics such as conductance and modularity to identify coherent clusters that better represent semantically unified events.
3. The storage of episodes: Every event is stored as a small memory block that includes key/value tensors, pooling representations, and metadata such as length and boundaries. Together, these provide the model's global episodic memory, distinct from the local context window. This pipeline is like how humans precisely continue experience into structured episodes; the model saves event-level summaries that can be reactivated later, rather than keeping every token in active memory.[3]

By including a feature crucial to human cognition, explicit interaction between events, the Cross-Event Reasoner expands EM-LLM. "The café reminds you of a meeting which reminds you of a decision" is an example of an associative chain that occurs when people recollect an end.

2.3 Memory-Augmented Neural Architectures

Standard transformer-based LLMs struggle with long inputs, which has led to much research on memory-augmented architecture. Instead of relying solely on a fixed context window, these models provide mechanisms for storing and retrieving information beyond what self-attention can retain. The main goal is to reduce attention loss and help models reuse important information over longer texts.[32]

In general, memory-augmented models add a non-parametric component to the main architecture, working alongside its parametric weights. Parametric memory is built into the model's parameters and can only change during training. Non-parametric memory, on the other hand, stores explicit representations of past data, like documents, activations, or events, and can be updated while the model is running. There are three main types of approaches for handling long-context reasoning:

Retrieval-augmented generation (RAG), which pulls in relevant text from an external corpus and incorporates it into the prompt[33]. Memory networks and memorising transformers, which integrate differentiable memory modules directly into the architecture[34]. Efficient and long-context transformers, which adapt the attention mechanism or positional encoding to accommodate more extended sequences.[35]

This section examines three types of models, explains their strengths and weaknesses, and prepares to introduce EM-LLM and the Cross-Event Reasoner.

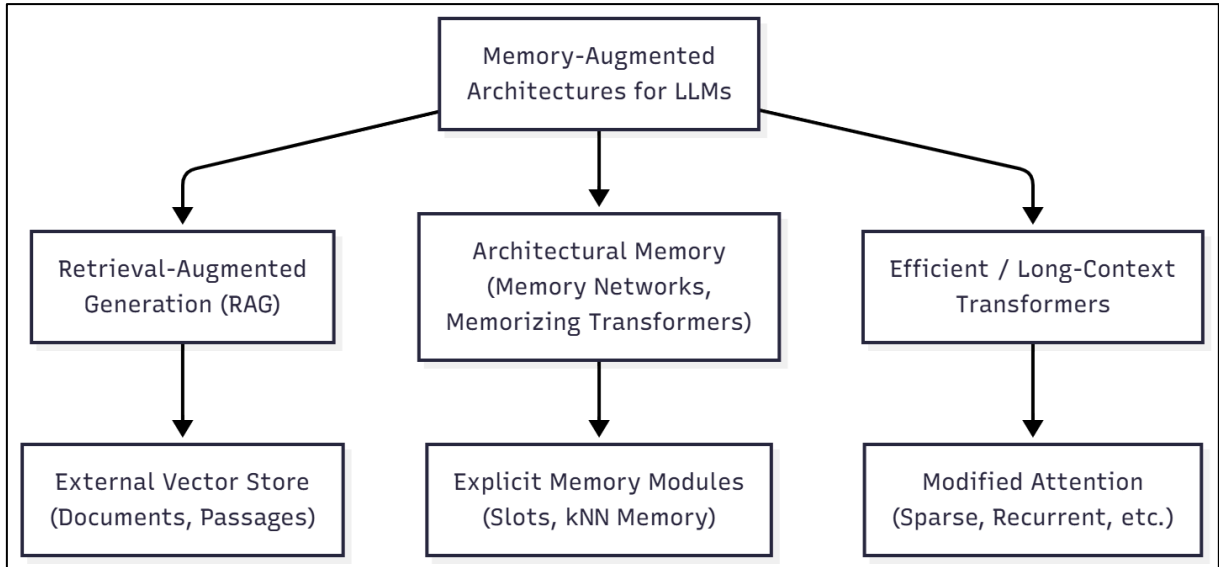


Figure 2.8. High-level taxonomy of memory-augmented architectures: RAG uses an external vector store; architectural memory integrates explicit memory modules; efficient transformers modify attention for more extended sequences

Figure 2.8. shows how the three families relate: RAG, memory networks, and efficient transformers. A general overview of memory-boosted designs for large language models is shown in the figure. There are three main types: retrieval-augmented generation, which lets models look up valid documents from an outside database; architectural memory models, which add clear memory features like slot-based or nearest neighbor modules into the system; and efficient or long-context transformers, which change how the model pays attention so it can handle much longer pieces of text.

2.3.1 Retrieval-Augmented Generation (RAG)

Retrieval-augmented generation (RAG) is often used to help large language models (LLMs) leverage additional information, especially when substantial knowledge is required. Instead of storing everything in its own memory or in a single chunk of text, RAG uses a separate memory. This is usually a special list of numbers made from documents or passages, which lets the model find valid text when needed.[32]

A typical RAG system has two main parts:

1. **Retriever:** Given a question (for example, from a user), a neural retriever turns the question into a set of numbers and looks for the closest matches in a list of text pieces. This list is usually compiled from sources such as Wikipedia, special collections of documents, or records of earlier conversations.
2. **Generator:** The top-matching passages are combined with the question and sent to a language model, which generates the final answer. Some versions let the model see different passages at each step, while others use the same set of passages throughout.[33]

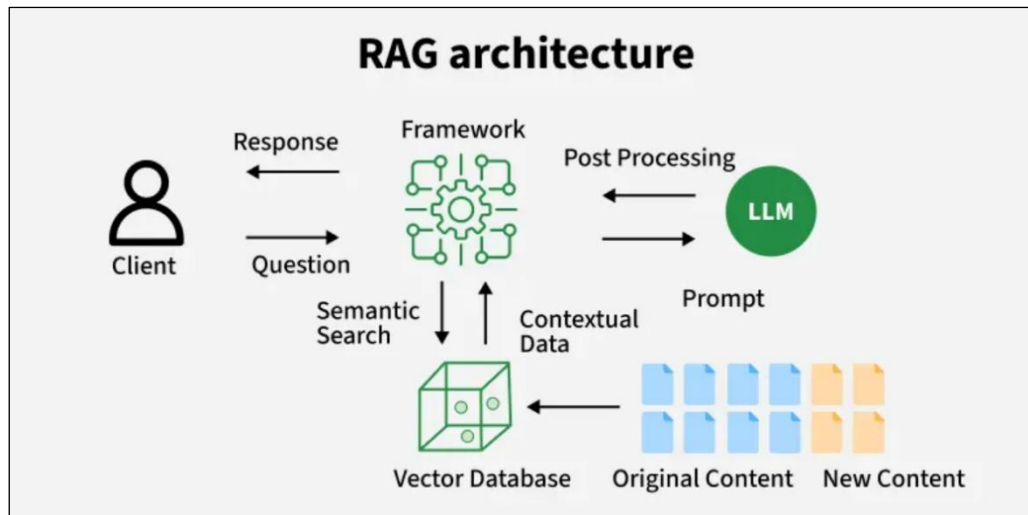


Figure 2.9. RAG Architecture[36]

Retrieval-Augmented Generation (RAG) improves large language models by integrating them with external knowledge sources. This lets them use current or specialized information and helps prevent hallucinations. RAG has two main parts: a retriever, often using Dense Passage Retrieval (DPR), and a generator based on transformer models like BART or GPT. The retriever converts the user's question and a set of documents into dense vectors, identifies the passages most like the question, and sends them to the generator. Since these parts operate independently, the system can leverage both the model's built-in knowledge and information from a vector database.[36]

The generator combines retrieved passages using either the Fusion-in-Decoder (FiD) or the Fusion-in-Encoder (FiE) approach. FiD keeps retrieval and generation separate, merging the information during decoding. This usually gives better answers but needs more computing power. FiE combines the passages earlier in the encoder, which is faster but may be less flexible. The process starts by embedding the query, searching the vector database, finding relevant passages, and then creating a final answer. The model's response mixes what it already knows with the new information it finds.[36]

Retrieval-augmented generation brings several advantages:

It gives the model access to knowledge that lies beyond its training set, since the external index can be updated independently. It helps the model use more information by showing only the most essential parts of long documents or big collections of text. It works with many different language models, so most existing models can be used as generators without changing their architecture.

From the perspective of this thesis, however, several limitations matter:

1. **Chunk-based segmentation:** Documents are usually split into pieces of a fixed size (for example, 512 to 1024 words), without regard for where stories or ideas naturally break. Important information can be spread across several pieces, so the model ends up reading a mix of texts rather than a smooth story.
2. **Limited structural reasoning:** The retriever assesses how similar each piece of text is to the question. It does not directly consider connections between pieces, such as the order of events, cause-and-effect, or links that involve several steps. Tasks that require combining different pieces of information still rely solely on the generator's detailed attention.
3. **Sensitivity to retrieval errors:** If retrieval does not find the crucial passages or only finds somewhat related ones, the generator may make up answers that sound right but are wrong. This problem gets worse in tasks with lots of information, where the details needed are hard to find.

Although recent work has suggested improvements such as rewriting questions, reordering results, changing chunk sizes, and finding information across several steps, the basic setup remains the same: the language model receives a set of found text pieces, joined together, as its input. For long stories or

tasks that require several steps of thought, this leaves a critical gap that EM-LLM and CXR aim to address: the need to show and think about events rather than just separate pieces of text.

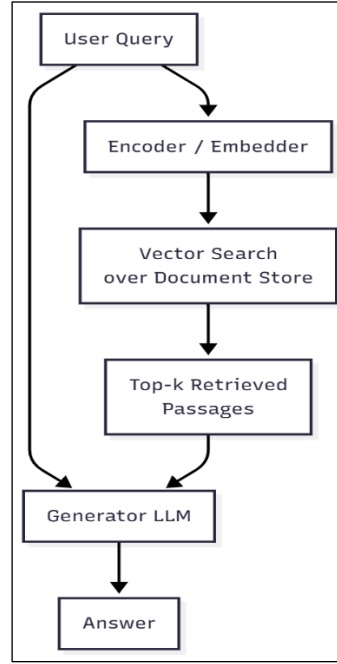


Figure 2.10. Block diagram for RAG

Figure 2.10. shows the main steps in a Retrieval-Augmented Generation (RAG) system. It explains how a user's question is turned into an answer by using both retrieval and generation.

The process starts when the user submits a question. This question is sent to both the encoder and Generator LLM simultaneously. The encoder converts the question into a set of numbers, which is used to search a database of documents that have been prepared for search. The system finds the most relevant parts and sends them, along with the original question, to Generator LLM. The generator combines this information with what it already knows to produce a final answer that draws on both its own knowledge and the latest external information. The system picks the most relevant passages that match the user's question. These passages, along with the original question, are sent to Generator LLM. The generator then combines this new information with what it already knows to create a final answer that is accurate and up to date.[32]

In this thesis, RAG is seen as something that works alongside other methods. While RAG finds valid documents or passages from extensive collections, EM-LLM builds an internal memory over a single stream (e.g., a lengthy document or conversation), and CXR reasons about these internal events. The mixed-method approach that combines external retrieval with internal memory is promising, but it is not the primary focus of this work.

2.3.2 Memory Networks and Memorizing Transformers

A second group of methods adds clear memory parts directly into the neural network. End-to-End Memory Networks, in which input facts are stored in a memory table and accessed stepwise via attention. At each step, the model focuses on different memory slots, reads a mix of them, and updates its state, allowing it to reason through the stored facts in several steps.[34]

This area is closely related to Neural Turing Machines and the Differentiable Neural Computer, which connect a controller network with an external memory that can be read from and written to during learning. Even though they work differently, these designs share the main idea of clearly separating thinking from storing information.

Subsequent models integrate memory more tightly into transformer-style architectures: Some add permanent memory slots to transformer layers, updating them over time to retain important information for longer. Others maintain external memory banks of hidden states and regularly update them with important information, allowing the model to look back beyond its current input.

Memorizing Transformers extends this approach by attaching a k-nearest-neighbour (kNN) memory of hidden representations. During training or inference, selected activations are written to an extensive index. Later, when processing new inputs, the model retrieves similar past representations and attends to them. This provides a form of long-term, non-parametric memory that allows information to be added without modifying model weights.[37]

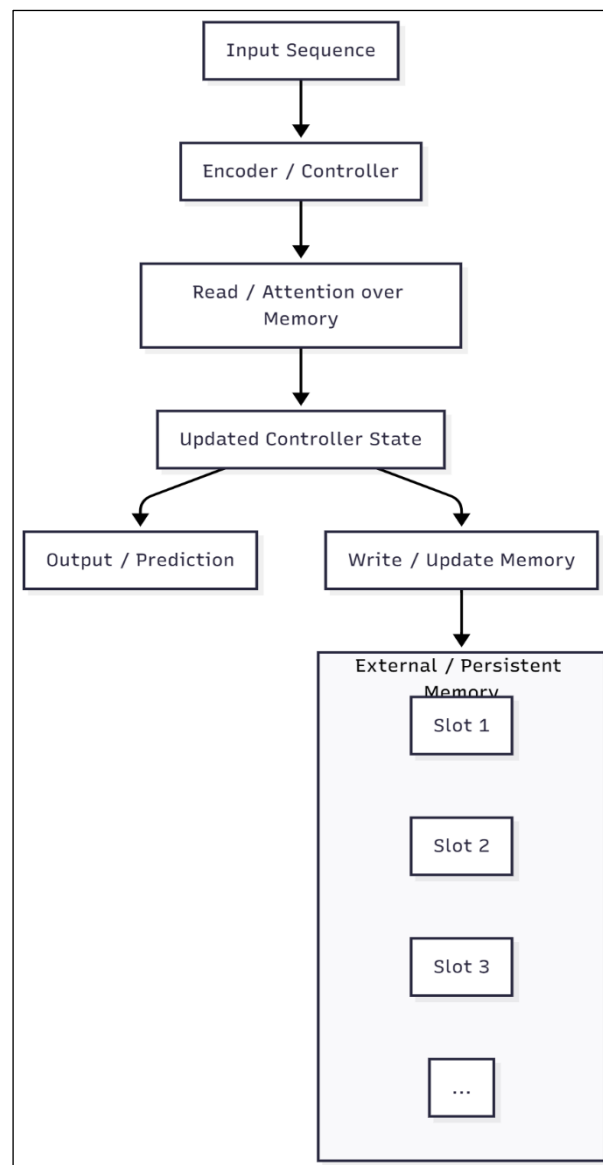


Figure 2.71. Schematic view of memory-augmented neural architectures such as End-to-End Memory Networks and Memorizing Transformers. A controller (e.g., a transformer) reads from and writes to an explicit memory of slots or stored activations.

Figure 2.11 shows the main workflow of a memory-augmented neural architecture, where the model interacts with an explicit external memory while processing an input sequence. The input is first handled by an encoder or controller, which generates a representation used to read from or attend to the memory. Using the retrieved information, the controller updates its internal state, which then leads to two parallel operations: producing an output or prediction and writing updated information back to the external memory. The memory is organized into individual slots that persist across steps, enabling the model to store, retrieve, and refine information over time. This read-update cycle supports long-term reasoning, retrieval, and contextual coherence beyond what a standard transformer or recurrent model can maintain using only its internal hidden states.

These architectures illustrate that explicit memory can improve performance on tasks involving long-term dependencies and multi-step reasoning. Enable models to acquire new information during inference by writing to memory instead of retraining.

It also has a few limitations, such as often requiring specialized training to learn to read and write well and to keep learning steadily. The memory is primarily unorganized, mainly consisting of raw hidden states or slots, with no clear steps for representing events, episodes, or stories.

These systems are complex to build and require significant computer power and hardware. For this thesis, these models show the value of explicit memory, but they do not target the human-inspired, event-based memory organization central to EM-LLM. They are also less suitable when the goal is to build a training-free wrapper around existing LLMs, a key design constraint for EM-LLM and CXR.

2.3.3 Efficient and Long-Context Transformers

Another critical area of research focuses on making transformers faster and able to handle longer sequences by changing how they attend to words, how they keep track of position, or both. Standard transformer self-attention requires much more computing power as the sequence length increases, which becomes too much for sequences longer than a few thousand words. Efficient transformer designs aim to reduce this cost while delivering better performance.[37]

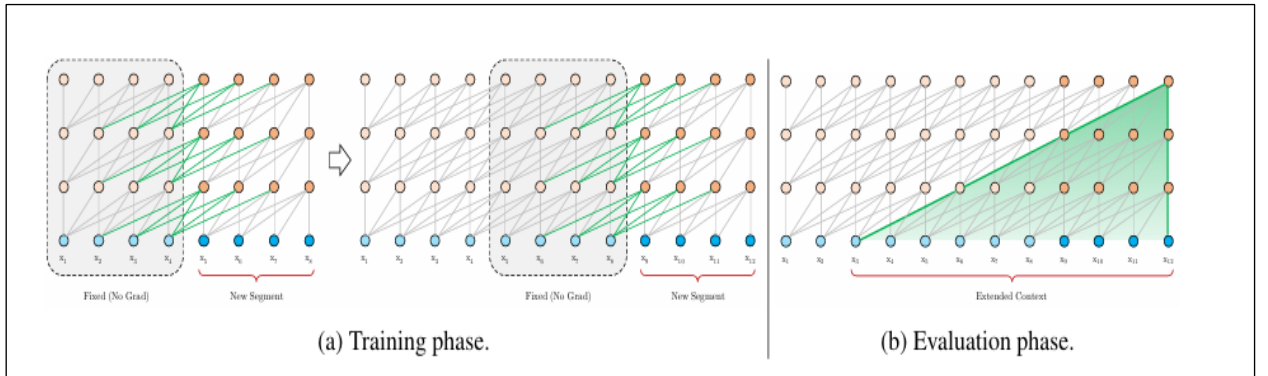


Figure 2.82. Illustration of the Transformer-XL model with a segment length of 4

The mechanism is shown in Figure 2.12. rather than processing each segment in isolation, Transformer-XL reuses the hidden states from the preceding segment as cached memory when encoding the current segment [38]. At evaluation time, these cached states are concatenated with the current segment's representations and serve as keys and values in self-attention, allowing each token to attend not only to others within its own segment but also to tokens from earlier segments. This segment-level recurrence extends the effective context length without recomputing attention over the entire history. It reduces context fragmentation caused by naively splitting long sequences into separate chunks.

The Long-Range (LRA) benchmark tests several efficient transformer models, including Reformer, Linformer, Performer, Sparse Transformer, and Longformer, on tasks with sequences up to 16,000 tokens.

These models use techniques such as sparse attention, which limits attention to local windows or a chosen subset of positions, thereby compressing memory or splitting data into smaller parts.[15]

New large language models with long context can handle much longer text, even tens or hundreds of thousands of words at once. They do this by using faster attention methods, along with tricks such as rope scaling, sliding windows, and paged attention. Because of this, these models can read and process entire books or codebases at once.

However, there are still some limitations, including making the context window longer, which does not solve the problem of attention decay. Studies show that even in long-context models, the model mostly pays attention to recent tokens, and tokens farther away have little effect on predictions. The input is still processed as a straight sequence of tokens. The model lacks a built-in mechanism for representing events, episodes, or higher-level structures, so it must learn these patterns on its own.

Many efficient transformer types need to be trained from scratch or require extensive additional training. This is not practical for large models that cannot be changed or for situations where there are not enough resources for training. These transformers primarily serve as tools for handling long texts and working alongside memory-boosting methods. In this thesis, these models show how changing the design can help with longer context, but they still do not replace the need for a clear, event-based memory that works more like human memory.

2.3.4 Summary and Positioning

In retrieval-augmented generation (RAG), memory networks, and efficient transformers, models obtain additional information via external retrieval, memory modules, or larger context windows. Still, this information is usually handled as unstructured tokens or chunks. As a result, the base transformer is mainly responsible for organizing, interpreting, and integrating these elements.

EM-LLM and the Cross-Event Reasoner use a different approach. These models treat events as primary units, drawing inspiration from human episodic memory. They clearly separate local context, which acts like working memory, from global episodic memory, which serves as long-term storage. These systems serve as training-free wrappers for existing models, making them usable even when fine-tuning is not possible.

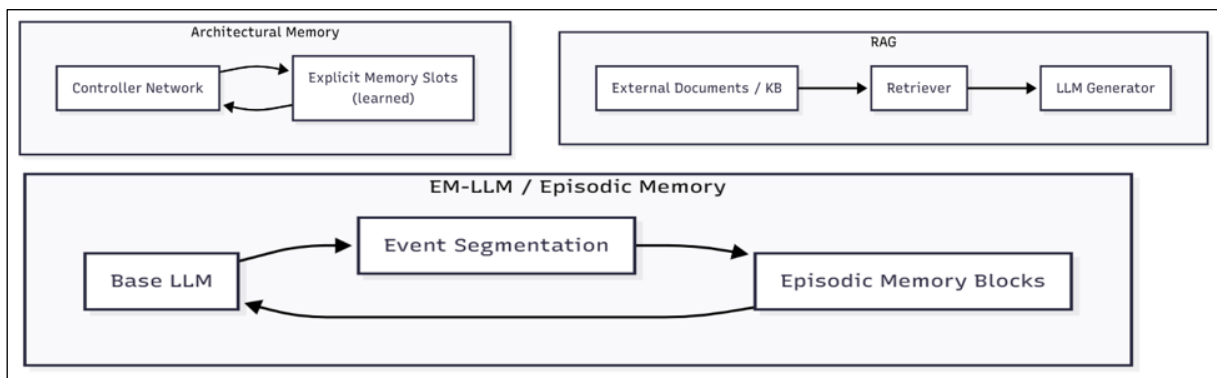


Figure 2.93. Comparison overview of RAG, Architectural Memory, and EM-LLM

Figure 2.13 shows a conceptual comparison of RAG, architectural memory models, and EM-LLM. RAG retrieves external documents to augment the generator, architectural memory networks maintain learned memory slots that the controller reads from and writes to during processing, and EM-LLM segments long input streams into events that are stored in episodic memory for later retrieval.

EM-LLM and CXR bring together several research ideas. They use non-parametric memory from RAG and memory networks, address scalability issues in efficient transformers, and add an event-based structure grounded in cognitive theories of human memory. The following sections explain how EM-LLM uses this episodic memory pipeline and how CXR builds on it with cross-event reasoning.

2.4 EM-LLM: Human-Inspired Episodic Memory for LLMs

Standard LLMs struggle to manage vast amounts of information, whereas the human brain can organise and retain experiences throughout a lifetime. In this work, the authors introduce EM-LLM, a design inspired by key principles of human event memory and experience understanding, implemented without any additional training. This approach enables models to handle effectively unlimited input lengths while remaining computationally efficient. EM-LLM groups token sequences into coherent events by detecting surprising changes and refining boundaries through graph-based methods, all performed online as data arrives. When information is needed, the system retrieves events through a two-stage process that first identifies semantically similar events and then incorporates those that occurred nearby in time, supporting efficient, human-like recall of relevant material.[3]

Experiments on the LongBench and ∞ -Bench benchmarks show that EM-LLM consistently outperforms the leading retrieval method, InfLLM, across multiple base LLMs. EM-LLM also surpasses RAG on many tasks while using comparable computational resources. It even outperforms full-context models on most evaluations and can successfully retrieve information across 10 million tokens, a scale that full-context approaches cannot reach.

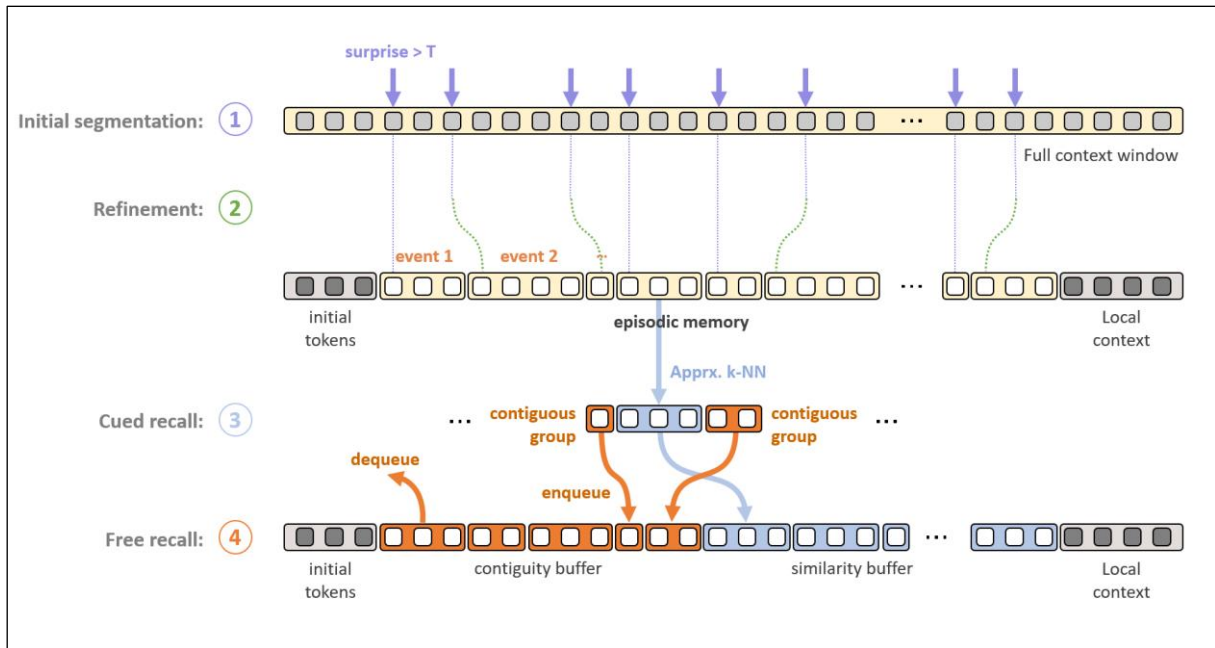


Figure 2.104. Architecture of memory formation and retrieval in each LLM layer.[3]

The overall EM-LLM architecture is shown in Figure 2.14. A long token stream is first processed by a base LLM, whose surprisal and hidden states are used to segment the input into events. These candidate event boundaries are refined using a graph-based step to ensure that segments align with semantically coherent regions. Each resulting event is then stored as an episodic memory block containing pooled representations and key-value tensors outside the standard context window. At inference time, a retrieval module ranks these events by similarity and importance, selects a small subset, and injects their key-value tensors back into the model's attention, allowing the LLM to condition on information far beyond its native context length.[25]

EM-LLM helps language models get closer to human-like memory, but it still has some significant problems. The main problem is that the model struggles to link information across different events. It gets each event as a separate piece and must connect them on its own. When important details are spread out, this often does not work well. For example, one event may introduce an entity and its attributes. A later event may describe an action involving that entity. A third event may present an outcome that only makes sense when connected back to the earlier details.

Humans can easily connect to these events and build stories that explain what happened and why. EM-LLM can remember the events, but it cannot put them together or show why they matter as a group. Because of this, the model may see each event in isolation and miss the bigger picture, especially when it cannot attend to everything at once.

2.4.1 Surprisal-Based Event Segmentation

EM-LLM splits text based on the idea that people remember life as separate episodes rather than a continuous stream. To mimic this, EM-LLM uses surprisal as a real-time signal to find where one event ends and another starts.

$$s_t = -\log p(x_t | x_{<t}) [3]$$

In this equation, x_t stands for the current word or symbol, and $p(x_t | x_{<t})$ is the model's predicted probability. Low surprisal means the token is expected, so the narrative continues smoothly. High surprisal means the word or symbol is unexpected. This could signal a topic change, a new character, a shift in the story, or a change in style. In practice, EM-LLM calculates surprisal at one or more points as it moves through the text. To avoid random spikes, surprisal can be averaged or made less sensitive, and the cut-off points can be adjusted for each dataset. When surprisal exceeds a set threshold or rises rapidly, EM-LLM flags a possible event boundary.[3]

There are a few practical points to consider:

- Granularity: If boundaries are too frequent, events become too short and lose coherence; if they are too sparse, events become long and behave like ordinary chunks. Segmentation parameters, therefore, determine the effective “episode length” and can be adapted to different domains (e.g., dialogue vs. narrative).
- Model dependence: Since the main LLM calculates surprisal, splitting depends on what the model finds surprising. Boundaries appear where the model is surprised, which is where extra memory is most useful.
- Online processing: Surprisal can be calculated step by step as the model reads the text. This lets EM-LLM process streaming text without loading the whole document at once.

However, surprisal alone does not guarantee clear semantic boundaries. Minor errors, style changes, or rare words can cause minor spikes that do not show fundamental changes in events. This is why EM-LLM uses a second segmentation stage called graph-theoretic refinement.

2.4.2 Graph-Theoretic Boundary Refinement

The original EM-LLM design includes a graph-based refinement module that aligns surprisal-based boundaries with the semantic structure in hidden states. In the public version, this module was partly disabled. This thesis restores and efficiently integrates it. The method creates an activation adjacent graph using token-level hidden states.[20]

1. Select one or more transformer layers that capture higher-level semantics, usually the top layers.
2. For each token, extract its hidden vector and measure its similarity to nearby tokens, either within a sliding window or using k-nearest neighbors in the embedding space.
3. Build a graph where each node is a token, and edges connect highly similar tokens.

EM-LLM then applies metrics like modularity and conductance to this graph. Modularity measures how well the graph can be split into tightly connected groups. Clusters with high modularity often match semantically coherent segments. The conductance measure quantifies the quality of a cut between two

node sets; low-conductance cuts indicate firm boundaries with relatively few edges crossing between them. The method then refines surprisal-based boundaries by aligning them with meaningful cuts in the graph. If an event boundary falls within a high-modularity cluster, it can be moved to the nearest low-conductance cut. This helps avoid splitting into different coherent events. New boundaries can be added where the graph clearly separates two communities, even if surprisal does not show a substantial spike.[20] [3]

This refinement solves two main problems: it captures gradual topic shifts that might not cause firm surprisal peaks but do show up as changes in the representation space. Then it reduces boundary artefacts caused by noise or stylistic quirks, thereby improving event stability and interpretability.

Building complete graphs for very long sequences is computationally intensive. In practice, EM-LLM systems use approximate graphs, such as local windows or kNN graphs, and focus their analysis on areas near event boundaries. This thesis uses methods that balance segmentation quality and computational efficiency.

2.4.3 Episodic Memory Storage and Hierarchy

After identifying events, EM-LLM saves them in a special episodic memory. The main goal is to keep short-term and long-term information separate. A local context window, such as the most recent few thousand tokens, is handled with standard transformer attention. Earlier content is compressed into event-level memory blocks and removed from the active context.

Each memory block usually contains the following:

- Key and value tensors for attention linked to the event’s tokens.
- Token or span embeddings that help with similarity scoring and pooling.
- A pooled event embedding that sums up the whole event as one vector.
- An importance score, like `repr_score`, shows how often the event has been retrieved and used.
- Metadata includes things like token indices, event length, and where it is stored (GPU, CPU, or disk).

This setup creates a layered memory system:

1. At the base level, there are raw tokens in the current local window.
2. Above that are event representations of earlier parts from the same document or conversation.
3. There could also be higher-level summaries made by merging events, but this feature is not fully developed in the original EM-LLM.

This structure is like how human memory works. Recent details remain active and detailed, while older information is grouped into episodes and brought back only when needed. In practice, EM-LLM manages its memory pool using several policies:

- Offloading means moving rarely used events to the CPU or disk.
- Garbage collection involves removing ancient events that are never retrieved.
- Priority updates raise an event’s `repr_score` when it is retrieved and slowly lower it otherwise. This helps the system find episodes that are still useful.

With these mechanisms, EM-LLM can handle much longer input streams than a fixed context window and keep access to important past events.

2.4.4 Retrieval and Multi-Stage Attention

When the model needs to answer a question or keep writing, it cannot reload every saved event. Instead, EM-LLM selects and ranks events, then sends only a small subset to the transformer. Event ranking usually relies on three main signals:

- Usage-based importance (`repr_score`): Events that were helpful in previous retrieval steps get a higher score.

- Semantic similarity: A query from the current state is compared to stored events using simple math methods to see how similar they are.
- Contiguity: Events that are close in time to high-scoring ones are also included. This helps keep local continuity and avoids retrieving isolated fragments.

These signals are adjusted and combined, often using simple arithmetic, to produce a final ranking. The top events and their nearby neighbors are then picked and added back into the model's context.

EM-LLM uses a multi-stage attention process:

1. The transformer first attends the current local tokens as it usually does.
2. The retrieved events provide additional information to the attention process, allowing the model to use details from earlier.
3. The outputs from local and episodic attention are combined using special connections and layers to produce the final prediction state information from much further back without reprocessing the entire history.

However, in the original EM-LLM, events are still treated as separate units. Retrieval primarily operates at the event level, and there is no special way to reason about relationships between events before they enter the transformer.

2.4.5 Limitations of EM-LLM:

EM-LLM helps language models get closer to human-like memory, but it still has some significant problems. The main problem is that the model struggles to link information across different events. It gets each event as a separate piece and must connect them on its own. When important details are spread out, this often does not work well.

For instance:

- One event may introduce an entity and its attributes.
- A later event may describe an action involving that entity.
- A third event may present an outcome that only makes sense when connected back to the earlier details.

The human brain can easily connect these events and build stories that explain what happened and why. EM-LLM can remember the events, but it cannot put them together or show why they matter as a group. Because of this, the model may see each event in isolation and miss the bigger picture, especially when it cannot attend to everything at once.

2.5 Cross-Event Reasoning and Event-Level Modeling

As explained in Section 2.4, EM-LLM improves on standard LLMs by dividing long token streams into episodic events, storing them as compressed memory blocks, and retrieving them using similarity and contiguity signals. However, in the original EM-LLM framework, these retrieved events are mainly treated as separate pieces of evidence. They are chosen and placed back into the context window, but their interactions are not directly modeled.

Research in narrative understanding, knowledge graphs, and multi-hop question answering shows that reasoning often depends on relationships between events rather than isolated facts. For example, narrative event-chain models show stories as sequences of linked events connected by a shared protagonist and 'before/after' time relations.[1] The Cross-Event Reasoner (CXR) in this thesis builds on these ideas. It adds an event-level reasoning layer to EM-LLM's episodic memory. Instead of just retrieving a list of relevant events, CXR represents events as nodes in a graph, models their relationships with learned or heuristic edges, and runs a simple reasoning step over this graph before sending the final event set to the base LLM.

2.5.1 Event Representations and Event Graphs

Many cognitive and computational models treat events as organized units rather than just pieces of text. Events include people, actions, and timing. For example, narrative event-chain models represent each event as a pair that connects an action to a main character, such as (pleaded, subj) or (convicted, obj), and learn which events often occur together in a row. Similarly, script learning and event schema research produce graphs in which each point represents a typical situation, such as “enter restaurant,” “order food,” or “pay bill,” and the lines show typical time or causal connections.

For EM-LLM and CXR, we use a more straightforward but related approach. Here, an event is a text segment made by EM-LLM’s segmentation process, along with a vector embedding that captures its main meaning. EM-LLM forms event-level representations by pooling token activations into episodic memory blocks. CXR then uses these embeddings as nodes in an event graph, where the edges can show:

Temporal relations, such as how close or far apart events are in the original document, are determined by topical or semantic similarity based on event embeddings. Entity overlaps, for example, occur when events share names or refer to the same thing. This approach is like earlier methods that model narratives, knowledge bases, or scenes as graphs and use their structures to support reasoning.

2.5.2 Multi-Hop Reasoning Across Events

To answer questions on multi-hop QA benchmarks like HotpotQA or newer long-context datasets, models need to collect evidence across several sentences or chapters. Traditional systems often create graph structures that link phrases using shared items or hyperlinks. These systems learn to follow a path in the graph that connects the question to the answer. In knowledge-graph Question Answers, this approach turns into a direct search over entities and their relationships. CXR uses a similar approach but works at the event level.

Rather than sending every candidate event that EM-LLM finds for a query to the LLM, CXR carries out a simple multi-hop reasoning process that:

1. Spreads relevance across the event graph. Events directly linked to the query are essential, but nearby events that help complete the reasoning chain also matter, especially those that add connecting details or intermediate entities.
2. Rather than treating events as separate pieces of evidence, CXR focuses on chains of events that together support a theory. Events that are distant or only loosely related are less likely to contribute to a clear explanation.

In simple terms, this acts like a quick message-passing or attention step across the event graph. Event embeddings share information and update their scores based on the query and their neighbors. This process produces a reweighted set of events that reflects both local similarity and their part in possible multi-hop reasoning paths.

2.5.3 Design Principles for Cross-Event Reasoners

Since EM-LLM is already a complex framework, CXR is based on a few practical principles. CXR works at the event level rather than the token level. Using event representations instead of raw tokens simplifies computation. This approach aligns with EM-LLM’s memory structure and supports cognitive theories that treat events as the primary units of episodic memory.

CXR is made to work with existing episodic memory pipelines. After EM-LLM retrieves events, CXR takes these events and their details, refines them using graphs, and produces a filtered, re-ranked list. This method is like recent plug-in episodic memory modules that add features to LLMs without retraining the main model.

CXR uses a simple, question-based approach to decision-making. Unlike large networks that need training, CXR works well with pre-trained models. Later chapters will describe versions in which events can

pay attention to each other, share information at random, and use basic scoring methods. The main idea is that the question highlights the most essential event chains, as people recall and connect experiences.

In summary, it is a cross-event reasoning layer inspired by narrative event chains and multi-hop question answering, but made for the episodic-memory setup of EM-LLM. The following chapters will discuss architecture, algorithms, and ablation studies, comparing CXR to EM-LLM without cross-event reasoning and to models without episodic memory.

3 Requirement Analysis

This chapter describes the goals, requirements, and design constraints for the system developed in this thesis. The system aims to efficiently manage context in LLMs to address attention decay, drawing on ideas inspired by human memory. It also defines non-functional requirements like scalability, modularity, and reproducibility, and turns the concepts from Chapter 2 into specific implementation steps. These include breaking long inputs into meaningful events, storing them in episodic memory, and retrieving relevant context when needed. The chapter also covers practical challenges, such as working with the EM-LLM research codebase, using a single-GPU lab setup, and maintaining a training-free wrapper compatible with current LLMs. Finally, it summarizes the datasets, benchmarks, and project-management tools that shape the system's design, implementation, and evaluation in the following chapters.

3.1 General Objectives

The initial research question for this project was, "How can concepts from human memory be used to mitigate attention decay in large language models?" As a result, the first stage was mostly experimental. Long-context transformers, retrieval-augmented generation, memory networks, and cognitive models of episodic memory were among the many publications that were studied. The objective was to find methods that prioritize and organize data in ways like human memory systems, rather than simply expanding the context window.

Recurrent or sparse-attention transformers, external memory modules, and retrieval-based techniques are among the potential approaches that appeared during this analysis. However, the majority of these either handled memory as an unstructured bundle of passages or required a significant retraining of the base model. EM-LLM, a training-free wrapper around existing LLMs that directly incorporates human memory-inspired ideas (episodic events, surprisal-based boundaries, contiguity, and relevance), was selected among the examined approaches as a particularly relevant and effective solution. As a result, EM-LLM emerged as the thesis's primary reference system and starting point.

After choosing EM-LLM, the project objectives were divided into three main areas.

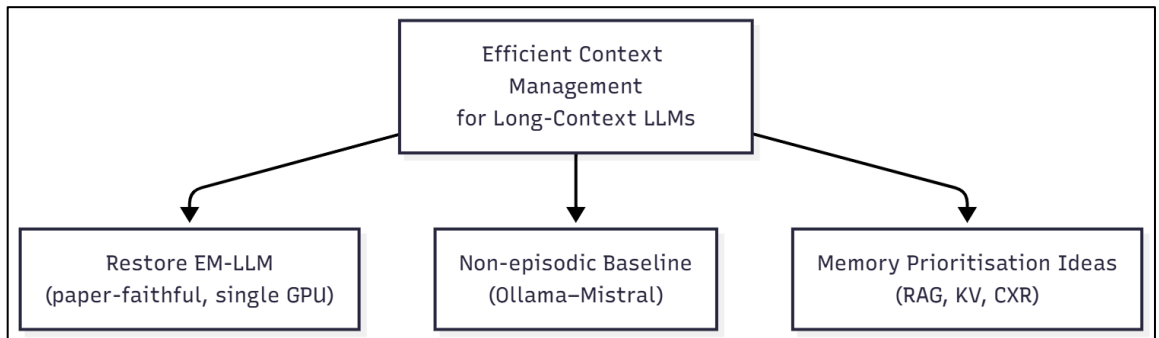


Figure 3.1. High-level objectives of the thesis

Figure 3.1 provides a concise overview of these three objectives and their relationship to the topic's overall thesis.

1. Using the same LLM to create a strong non-episodic baseline. The thesis also aims to create an independent evaluation method by using Ollama to run the same model (Mistral) on the chosen datasets, but without episodic memory or perplexity-based segmentation. This approach helps show the effect of episodic memory and prioritization. As a baseline, this Mistral pipeline shows how attention decay can limit performance when handling longer inputs in the usual way.
2. Explore ways to prioritize information in EM-LLM, inspired by how human memory works: The last goal is to find simple ways to better manage context after setting up a reliable EM-LLM and a non-episodic starting point. Several ideas are being tested now, either as designs or early versions. These include adding a step to pick which episodic events to show the LLM, combining extra information retrieval with EM-LLM, and creating easy methods to highlight important information across the model's layers.

A Gantt-style timeline and a GitHub Projects board are used to keep all code, configuration files, and planning documents for these three parts up to date in a separate GitHub repository. These resources formed the practical basis for the specifications and limitations discussed later in this chapter.

3.2 Problem Context and Use Cases

The chapter number explained that attention decay causes LLMs to struggle with long inputs. They may forget earlier content, have trouble with multi-step reasoning, and create summaries that focus too much on the end of documents. To clarify these issues, this section presents three examples of the proposed framework: summarizing long documents, remembering conversations across several turns, and answering questions or understanding stories that require information from different parts.

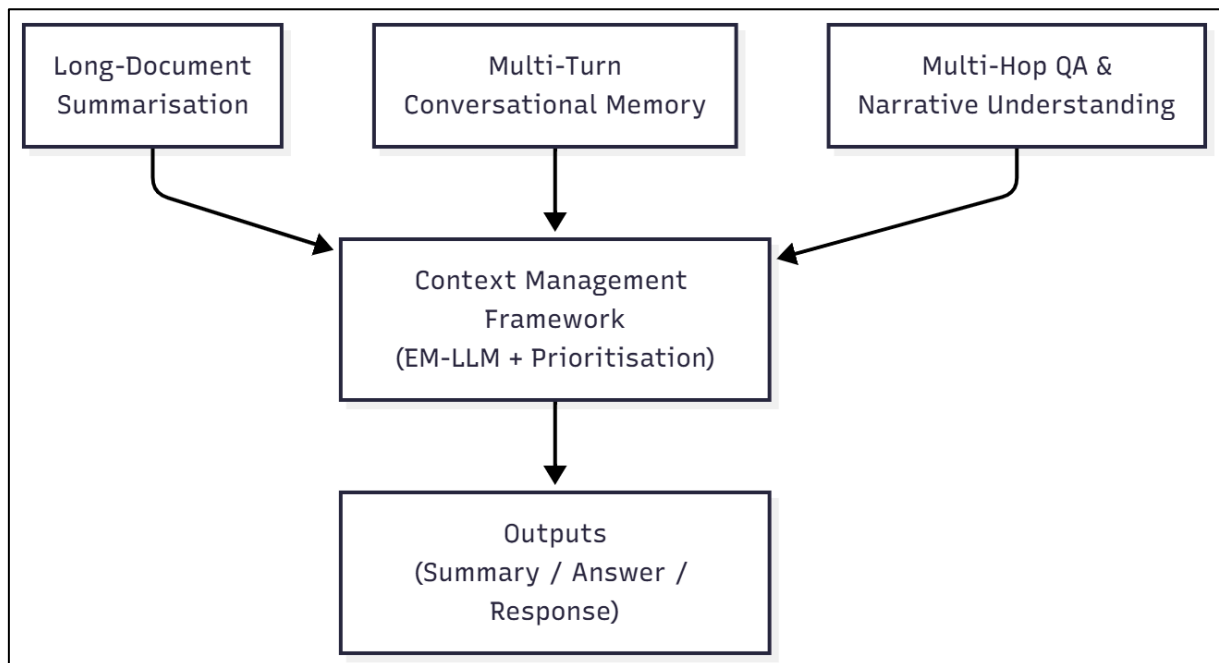


Figure 3.2. Use cases targeted by the framework

Figure 3.2 illustrates how these three scenarios fit into the proposed framework. Use cases targeted by the framework. Long-document summarization, multi-turn conversational memory, and multi-hop QA

or narrative understanding all feed long inputs into the same context-management layer. This layer segments text into events, stores them in episodic memory, and prioritizes relevant events before handing them to the underlying LLM to produce summaries, answers, or responses.

3.2.1 Long-Document Summarization:

In long-document summarization, the model is given an entire scientific paper, report, or story and must generate a summary. The conclusion and recent sections are overemphasized in the resulting summary because standard transformer models tend to focus primarily on the concluding segments of the input, effectively removing the start and early parts from the model's practical context. In this scenario, the structure should be as follows: these steps divide the text into episodic events, usually corresponding to scenes, paragraphs, or sections. Keep these events in an episodic memory that includes not only the latest few thousand tokens but the entire document. Prioritize globally significant events, problem statements, essential findings, and significant conclusions during summarization, even if these occur at the start of the text.

3.2.2 Multi-Turn Conversational Memory

Users often set general guidelines or preferences at the start of multi-turn conversations, like "always answer in JSON" or "use short bullet points." Standard LLMs may forget these instructions as the conversation goes on, and even the initial system message or early turns can lose context. In a multi-turn discussion, the framework should treat the conversation history as a series of interaction events, including phases such as problem definition, planning, and debugging. Key directives, choices, and limitations should be saved in episodic memory as high-priority events. Make it so that the retrieval process pulls not just the most recent turns, but also relevant earlier events when a new user message comes in.

3.2.3 Multi-Hop Question Answering and Narrative Understanding

Multi-hop question answering involves collecting evidence from different parts of one or more documents. Benchmarks like HotpotQA and 2WikiMultihopQA ensure that no single passage contains all the necessary information. Models must connect facts from other places to find the answer. Datasets like NarrativeQA go even further by asking questions about entire books or movie scripts. In these situations, EM-LLM helps by finding supporting events that are far apart. But the original version treats each event separately. As a result, the model might identify both parts of a reasoning chain but fail to link them clearly.[39]

To solve this problem, the framework should respond to these problems by finding multiple events that together answer them, even if they are spread out in the input. Add a method, even a simple one, to re-score or group related events. This helps the LLM see them as a connected reasoning chain. Include evaluation and logging tools that show these cross-event effects. For example, please keep track of which events were found and how often they were used during decoding.

3.3 Functional Requirements

Functional requirements describe how the system works. For this thesis, the scope is grouped into three categories:

1. Restoration of the baseline LLM and EM-LLM pipeline,
2. Integration of a lightweight event-level prioritization component, and
3. Evaluation of results.

To make the difference between the two main evaluation pipelines explicit, Figure 3.3 contrasts the Ollama-based baseline with the EM-LLM-based system.

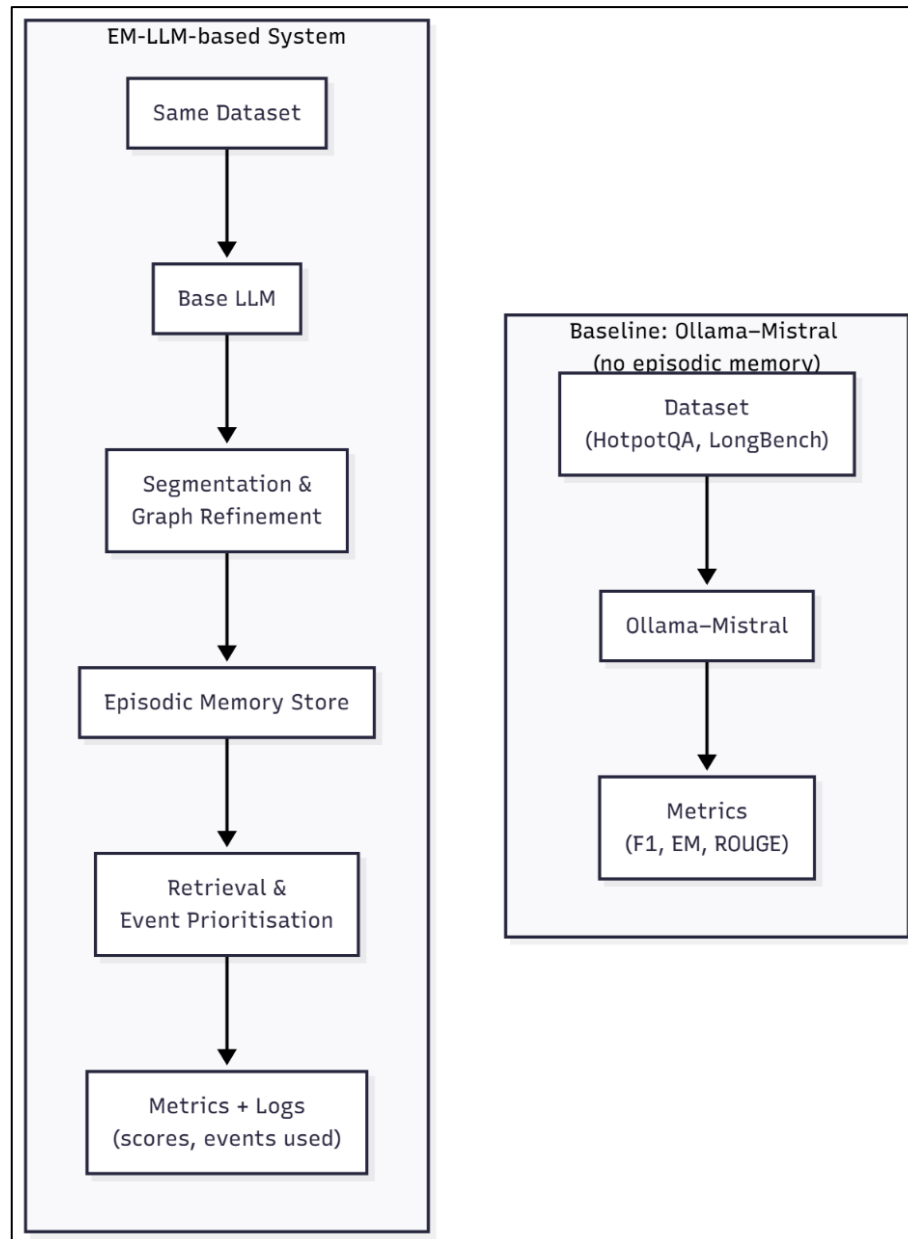


Figure 3.311. Restored the EM-LLM pipeline and the non-episodic baseline tool

Figure 3.3 compares the two evaluation pipelines used in this thesis. The baseline pipeline (right) runs the same underlying Mistral model via Ollama on the datasets without any episodic memory. The EM-LLM pipeline (left) passes the input through surprisal-based segmentation and graph refinement, stores events in an episodic memory, applies retrieval and event-level prioritisation, and then feeds the selected events back to the LLM. Both pipelines use the same datasets and metrics, allowing the impact of episodic memory and prioritization to be directly compared.

3.3.1 Baseline LLM and EM-LLM Restoration

1. **Baseline LLM inference (Ollama Mistral):**
Run Mistral-7B with Ollama on chosen LongBench or HotpotQA-style question sets, making sure not to use episodic memory. Keep the tokenization, answer generation settings, and evaluation methods the same. This will make it easier to compare results with EM-LLM later.

2. EM-LLM segmentation and episodic memory:
Reintroduce event splitting based on surprise, following the method described in the EM-LLM paper. Enable the feature that uses connection graphs to improve event boundaries and adjust how events are stored in memory. Each event should be saved as a block with key-value data, combined summaries, an importance score, and additional details such as length and boundaries.
3. Retrieval and multi-stage attention:
Set up the EM-LLM (S+M+C) system to retrieve information. It should use scores based on usage frequency, question-key similarity, and a buffer to include nearby events. Feed the retrieved events back into the model using EM-LLM's multi-stage attention, combining a close focus on current information with a broader view of memory blocks.
4. Robust execution on the lab machine:
Verify that the restored EM-LLM works well on the lab computer with a single GPU. If memory is low, shift some tasks to the CPU or disk as needed.

3.3.2 Cross-Event Reasoner Integration

This thesis explores how to prioritize context, inspired by how human memory works. EM-LLM is a valid starting point, but it typically retrieves events one at a time. Instead of building a large new model, the goal is to add a simple event-level prioritization step that can be tested with available resources.

Based on this, the system should do the following:

1. Make summaries of each event:
For each event found, please write a summary that captures its main idea, for example, by combining the key parts.
2. Identify connections between events:
For a small group of events, determine how similar each pair is, or use a simple method to find which events are most closely linked.
3. Update event scores based on the current question:
Combine the original retrieval score from EM-LLM with structure-based signals and a query embedding from the current LLM state to make a new priority score for each event. If needed, merge closely linked events into one summary to avoid repeating similar information.
4. Integrate the new step without disrupting the system:
Add this step between finding events and the subsequent processing stage. This keeps the main LLM settings unchanged and allows testing with or without the new part.

3.3.3 Evaluation of results

To determine whether the framework reduces attention decay, use appropriate evaluation and analysis tools.

1. Per-dataset evaluation scripts:
For each dataset chosen (like HotpotQA, 2WikiMultihopQA, NarrativeQA, QMSum, and relevant LongBench tasks), use scripts that test all model versions on the same data. Then, we evaluated the results using standard metrics such as F1, exact match, and ROUGE.
2. Context and memory logging:
For each example, I kept track of which events were split, which ones were saved in memory, which were retrieved at each step, and how their scores changed over time.

3.4 Non-Functional Requirements

Non-functional requirements describe how the system should behave in areas such as architecture, performance, and the ease with which results can be reproduced.

3.4.1 Model-Agnostic and Training-Free Design

Training large models is not possible with the limited resources available. The current framework should work with any decoder-only transformer that produces outputs, has hidden layers, and stores key-value information. All context-management features in EM-LLM and its add-ons work without needing extra training. The main LLM settings remain the same, and only memory structures and simple scoring methods are updated during runtime. We can set model options like model path, maximum context length, and offload settings using setup files and command-line options.

3.4.2 Efficiency and Scalability (Infinite Context)

EM-LLM aims to provide “infinite context” by using episodic memory and offloading. However, GPU memory and Input/Output bandwidth still limit its performance.

To address this, the framework must:

Reduce GPU usage by moving unused events to the computer’s central processor or storage, and load only the minimum memory needed for each task. Allow long inputs to be split into smaller sections, so even documents that are much longer than the usual limit can be processed step by step. While using episodic memory and focusing on specific data may slow things down compared to a basic LLM, everything should still run on a single-GPU lab computer.

3.4.3 Reproducibility, GitHub Project, and Open-Source Compatibility

The system should be easy to reproduce and simple to build on:

The GitHub repository includes environment files, such as requirements.txt, which list all packages and their versions, along with configuration files for different experiments. A GitHub Projects board managed tasks like setup, literature review, EM-LLM restoration, baseline tool development, evaluation, and writing. It also helped keep the work on track with the milestones in section 3.6. All experiments in the thesis are supported by corresponding configuration files and scripts, allowing results to be rerun. The framework works with standard open-source tools, especially Hugging Face Transformers and Datasets, so models and benchmarks are easy to reuse or extend.

3.5 Dataset and Benchmark Requirements

To test context management, it should be evaluated on datasets that require understanding information spread across long sections and summarizing it. The main types to consider are multi-hop reasoning datasets, narrative/summarization benchmarks, and dedicated long-context suites.

3.5.1 Multi-Hop Reasoning Datasets (HotpotQA, 2WikiMultihopQA)

- HotpotQA: It is a question-answering task that uses Wikipedia, where each question requires information from multiple documents. It provides supporting facts at the sentence level and uses F1 and exact-match scores.
- 2WikiMultihopQA: It is similar, but it asks for answers by linking information across several Wikipedia pages. The questions are more complex and need reasoning across multiple steps.
- Requirements: The system should combine both datasets into a single format and generate long texts that include all necessary paragraphs. It should support exact match and F1 scoring, which work with standard evaluation scripts. The system should record which events match the correct supporting facts so it can be checked for how well it remembers and selects essential information.

3.5.2 Narrative and Summarization Benchmarks (NarrativeQA, QMSum)

- NarrativeQA: It asks questions about whole books and movie scripts, so models need to understand the entire story.
- QMSum: It gives summaries based on specific questions from long meetings with many speakers, where the transcripts are often much longer than usual.
- Requirements: The system should handle long stories and meeting transcripts by breaking the text into sections and remembering earlier parts to cover the entire text. It should use summarisation metrics like ROUGE for QMSum and the same metrics as earlier work for NarrativeQA. There should be tools to check how well the system tracks people and events across chapters or meeting sections, as mentioned earlier in Chapter 2 about the risk of losing focus over time.

3.5.3 Long-Context Evaluation Suites (LongBench Tasks)

LongBench and similar datasets offer tests made for models that handle long pieces of text.

- Requirements: Choose LongBench tasks that can run on the available computers and match the examples in Section 3.2. Use or adjust LongBench’s evaluation scripts to make sure results can be compared to those for EM-LLM and other long-context baselines. The system should run Mistral, EM-LLM, and EM-LLM with prioritisation, all with similar text length limits, to see how memory and event selection affect results.

3.6 Time Frames and Milestones

Planning has started with setting up the environment and restoring the codebase. To manage this process and track our progress, we used two primary methods: we tracked our tasks on a GitHub Projects board, using columns such as “To do”, “In progress”, “Done”, and “Reviewed”. We have also used a Gantt-style timeline, either in a spreadsheet or a diagram, to schedule our main milestones from June to December 2025. Figure 3.4 shows snapshots from the GitHub project board, and Figure 3.5 summarises the planned milestones in Gantt form.

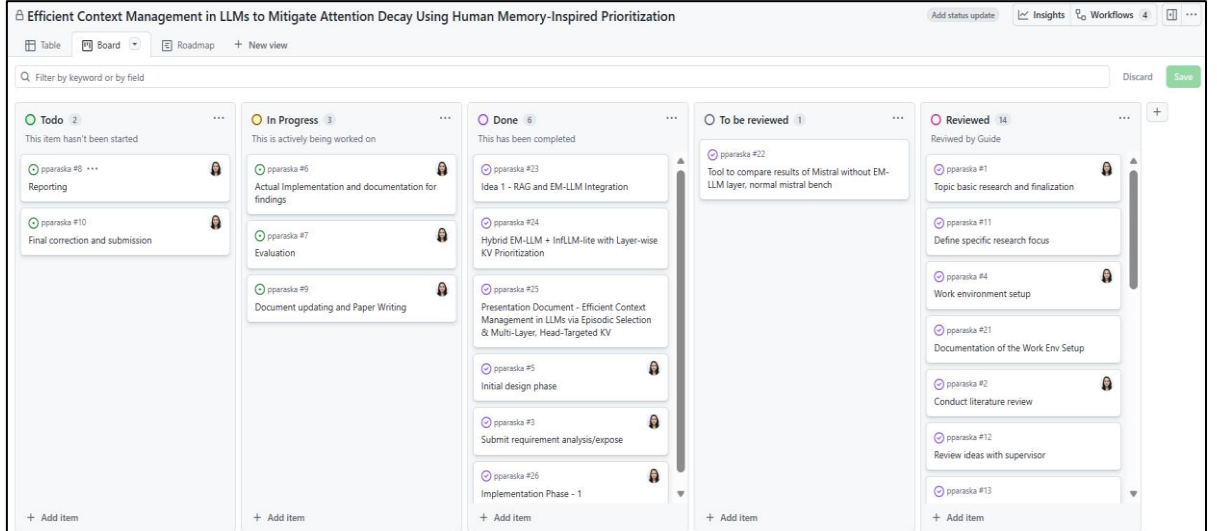


Figure 3.4. GitHub Projects Kanban Board

Figure 3.4 shows the GitHub Projects Kanban board used for this thesis. Tasks move from “Todo” through “In Progress” and “To be reviewed” to “Done” and “Reviewed”. The board captures environment setup, literature review, EM-LLM restoration, baseline tool development, evaluation, and reporting, mapping directly to the phases described in this section.

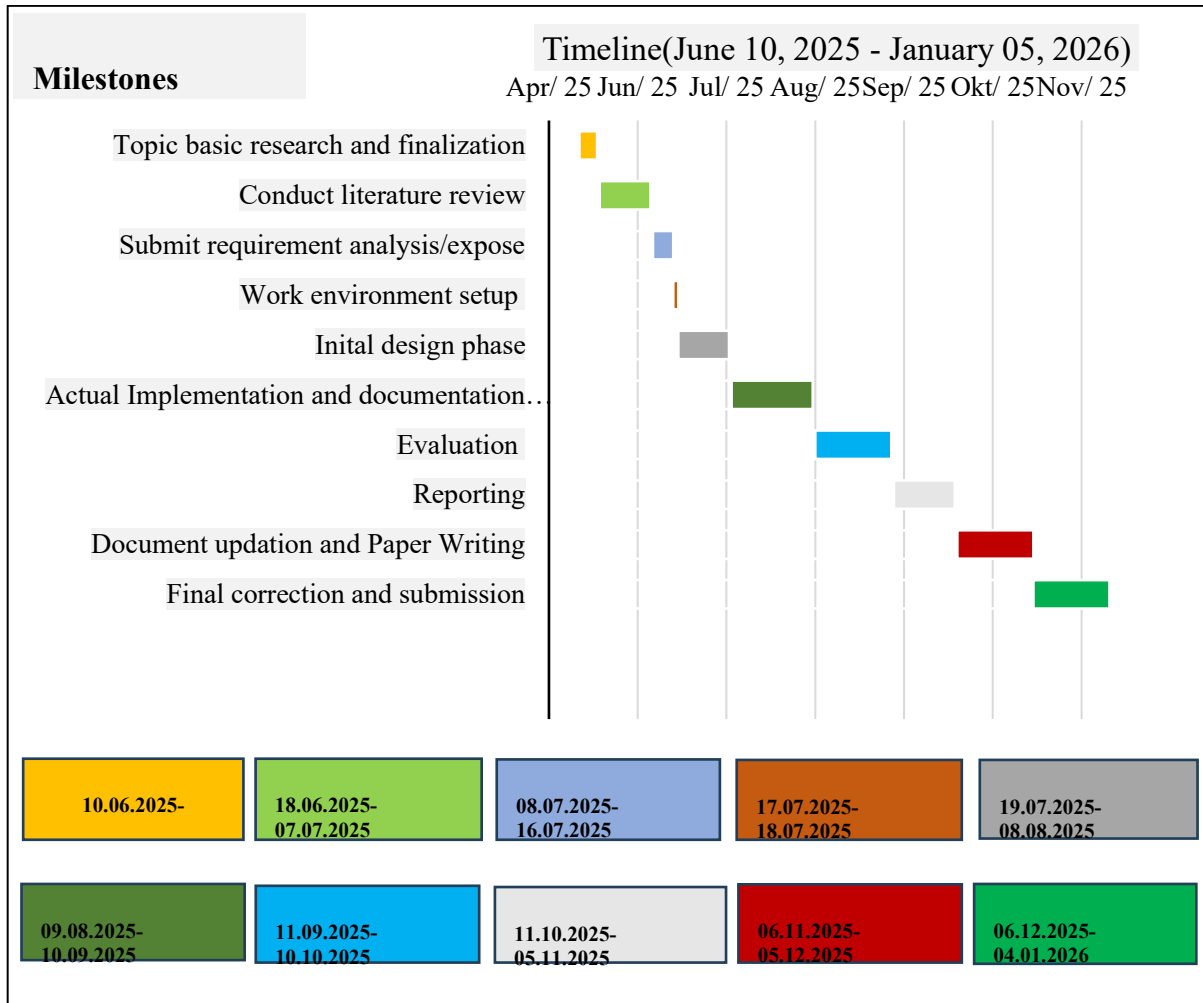


Figure 3.512. Gantt chart for the thesis

Figure 3.5 shows a Gantt chart for the thesis (June-January 2026). Each colored bar represents major milestones like topic selection and basic research, literature review, requirement analysis, work-environment setup, initial design, implementation and documentation of findings, evaluation, reporting, document updating and paper writing, and final correction and submission.

Using these tools, we divided the project into the following phases:

1. Environment set up and familiarization: We set up the lab computer with a single GPU, installed the required software, and resolved early issues with Hugging Face and Ollama on Windows. Forked the EM-LLM code, ran basic tests to verify it worked, and identified missing components, such as graph improvement, a buffer for organization, and tools for checking results.
2. Restoring EM-LLM: Re-enabled and debugged surprisal-based segmentation, graph-theoretic boundary refinement, and episodic retrieval, making sure the implementation matched the version described in the EM-LLM paper. Verified the small part of LongBench to see whether our results and patterns matched those reported on paper, using the computers we had available.
3. Baseline tool for non-episodic Mistral: Developed a separate evaluation tool to run Ollama-Mistral on LongBench/HotpotQA without episodic memory, using the same prompts and metrics as EM-LLM and comparing the results to show how performance drops due to attention decay when no memory mechanism is used.
4. Exploration of prioritization ideas: Designed and partially prototyped the RAG+EM-LLM and InfLLM-lite-style KV prioritization ideas, focusing on data flow, and we built an initial version of the Cross-Event Reasoner to sort events by importance, at least for small-scale tests.

5. Evaluation and reporting: Ran final experiments on the selected datasets for baseline Mistral, restored EM-LLM, and EM-LLM with event-level prioritization, collecting metrics and qualitative examples.

3.7 Target State/Objectives

Considering the previously outlined objectives and constraints, the target state of the project is summarized as follows:

- A stable, paper-faithful EM-LLM implementation operating on a single-GPU laboratory machine. The restored system implements the primary components of EM-LLM, including surprisal-based segmentation, graph-theoretic boundary refinement, episodic storage, and (S+M+C) retrieval. It also reproduces the qualitative behavior and approximate metric ranges reported in the original work.
- A clear baseline comparison is established using the same LLM without episodic memory. The Ollama-Mistral evaluation tool demonstrates the performance of the same underlying model when used conventionally, without episodic memory or perplexity-driven segmentation. This approach quantifies the impact of attention decay on long-context tasks.
- A prototype of human memory-inspired prioritization is developed. At least one event-level prioritisation mechanism, specifically the initial Cross-Event Reasoner, is integrated into the EM-LLM pipeline and evaluated at prototype scale. This integration demonstrates how simple, training-free re-ranking can influence the LLM's event selection. Additional concepts, such as RAG+EM-LLM and InfLLM-lite KV prioritization, are documented in sufficient detail to guide future implementation.
- A reproducible GitHub repository and associated planning artefacts are provided. The final GitHub repository contains the environment setup, EM-LLM changes, the baseline Mistral tool, dataset loaders, evaluation scripts, and the project board and timeline. This setup lets others rerun key experiments or build on the framework without having to recreate it.

Reaching this goal makes the thesis both an engineering and research achievement. It turns a complex research model into a tool that runs on simple computers, clearly compares memory-based and standard behavior within the same LLM, and begins using ideas from human memory to help the model stay focused on long texts.

4 System Design and Implementation

This chapter explains the design and implementation of the system developed in this thesis for efficient context management in LLMs. Based on the requirements defined in Chapter 3, the system brings together the scope of work:

1. A non-episodic baseline pipeline, implemented as an open-source Mistral Bench tool that runs the same underlying Mistral model via Ollama on LongBench datasets.
2. A paper-faithful restoration of EM-LLM on a single-GPU lab machine.
3. An initial memory-prioritization layer built on top of EM-LLM in the form of a lightweight Cross-Event Reasoner (CXR).

4.1 Overall System Architecture

The system consists of two tools, or pipelines, that work together and use the same type of base LLM. The first pipeline is a baseline that runs Mistral-7B Instruct through Ollama. It does not use episodic memory and works directly with LongBench-style inputs. The second pipeline, called EM-LLM, uses a Hugging Face version of the same model. It adds a step to split data into episodes, improves boundaries with graphs, stores and retrieves past information, and can be extended by CXR if needed. Both pipelines use the same evaluation layer, which calculates scores for each task (e.g., F1, exact match) and generates logs and charts.

4.1.1 High-Level System Pipeline

The system compares a standard inference pipeline with a pipeline that uses episodic memory. To keep results consistent and reproducible, all benchmark datasets undergo a normalization step that converts different input formats into a single unified format. In this manner, every part of the system uses the same type of input, irrespective of the original format.

Once the data is normalized, the system takes two paths. The first is a baseline pipeline based on Mistral Bench, where each example is handled separately with a standard prompt-based method. This baseline does not keep information from earlier inputs, showing how a typical long-context model works. The results from this path are used as a reference to compare with memory-augmented methods.

The second path uses the new EM-LLM pipeline, which adds context management to the baseline. It splits inputs into segments based on how surprising or informative the inputs are, then uses a graph-based method to make the segments more coherent. These segments are stored in episodic memory, so the system can pull up relevant past information when needed. Before generating an answer, an optional Cross Event Reasoner (CXR) step can re-rank the retrieved events to highlight the most relevant details for the current input.

Predictions and logs are generated by both inference paths and forwarded to a common evaluation stage. In addition to producing summary tables and charts for analysis, this step computes standard metrics including Exact Match, F1, and ROUGE. Any performance discrepancies are due to episodic memory and context prioritization rather than to the processing or assessment of the data, since all approaches are based on identical datasets, evaluation procedures, and metrics.

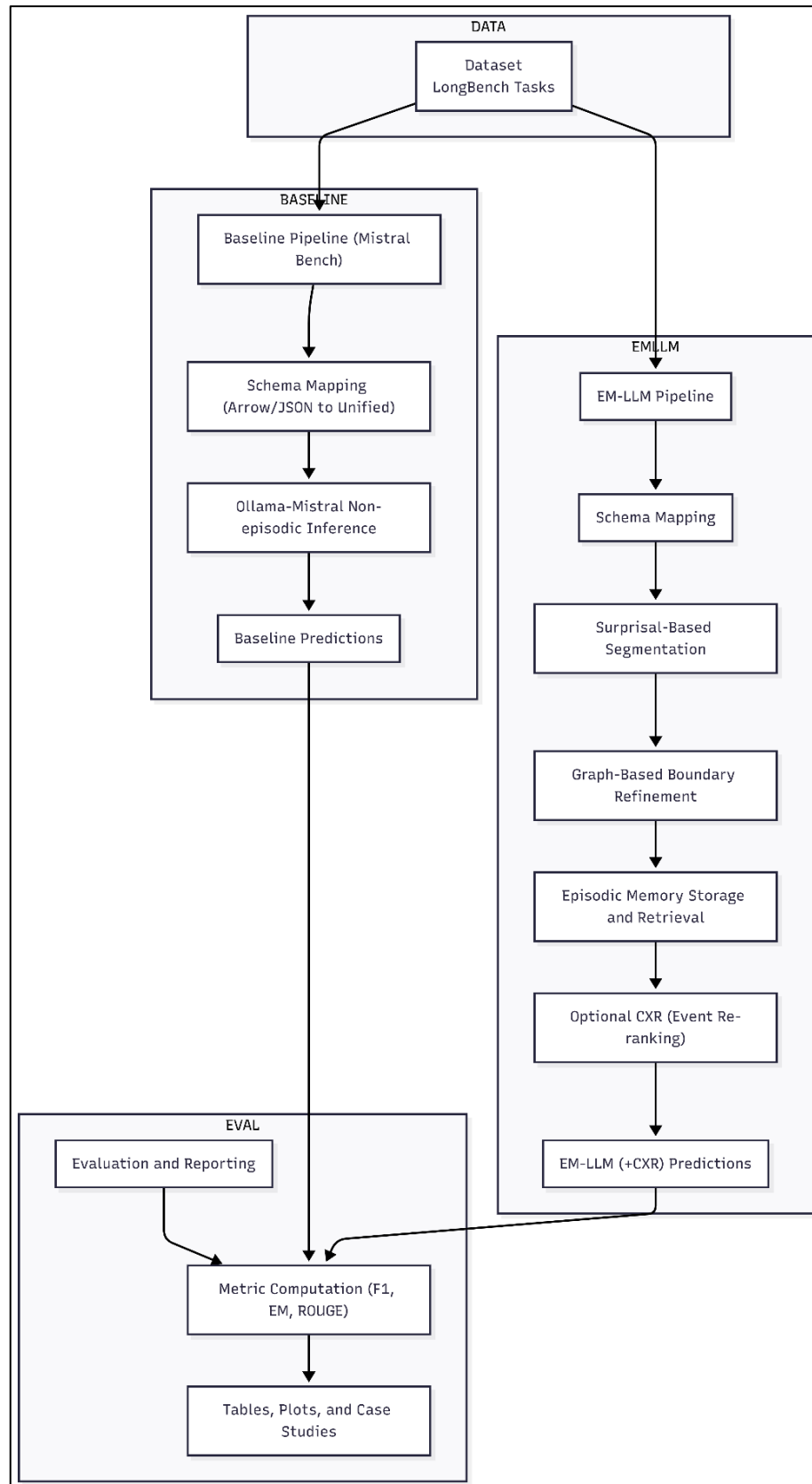


Figure 4.1. Overall System Pipeline

Figure 4.1 shows the high-level system pipeline. Normalized datasets feed into two inference routes:

- (i) Mistral Bench (non-episodic baseline), and

- (ii) EM-LLM with optional CXR. Both routes write predictions and logs to disk, from which evaluation scripts compute metrics and produce tables and plots.

4.1.2 Software Architecture and Module Overview

Internally, the system is implemented as a Python package with separate modules:

- Data: Dataset loaders and schema converters
- LLM-backend: Wrappers for Hugging Face models (EM-LLM pipeline) and for the Ollama HTTP API (Mistral Bench)
- EM_LLM-core: Restored EM-LLM components (segmentation, graph metrics, episodic memory, retrieval)
- CXR: Cross-Event Reasoner functions (event pooling, similarity computation, re-ranking)
- Decoding: Greedy search and perplexity computation
- Evaluation: Metric computation, result aggregation, plotting
- CLI: Command-line interfaces (EM_LLM-run, mistral-bench, etc.)

The system interacts with two types of the same model:

- For EM-LLM, the base model is loaded through Hugging Face Transformers using a checkpoint such as `mistralai/Mistral-7B-Instruct`.
- For Mistral Bench, the model is accessed through Ollama, which exposes Mistral as a local HTTP API (`/api/generate`) and handles model loading.

In both cases, the system requires tokenization and de-tokenization compatible with the model’s vocabulary. Access to token-level probabilities (or at least logits) for EM-LLM to compute surprisal and perplexity. (Ollama’s API does not currently support log-probabilities, so Mistral Bench sets perplexity fields to null while keeping file formats compatible.) Efficient management of the key/value cache during autoregressive decoding, particularly when re-using episodic memory blocks across steps. The following sections describe how these requirements shape the open-source Mistral bench, EM-LLM baseline, and the CXR extension.

4.2 Non-Episodic Baseline Pipeline: Mistral Bench

Mistral Bench is a tool for testing stateless LLM inference on standard QA and summarization datasets. It supports EM-LLM output format, making it easy to compare results with memory-augmented systems. We can use datasets in Hugging Face Arrow or JSON/JSONL formats; the tool converts them to a standard format before sending fixed queries to a Mistral model running on Ollama. The model stays separate from any memory system. Each example is processed in its own context window, so the results show only the model and prompt, without influence from episodic retrieval or changes in event order.

Mistral Bench is designed to be easy to use for experiments. For each run, it saves predictions, timing data, and standard QA metrics (such as Exact Match and F1) for each item. The tool creates per-item JSON/JSONL files and summary files in `res.json` or CSV format, which work with EM-LLM comparison tools. This makes Mistral Bench a reliable reference for your thesis. Any performance differences between Mistral Bench and memory-based pipelines are due to the memory mechanisms, not the evaluation process. Saving results for each item also helps with error analysis and the creation of visualizations.

4.2.1 Design Goals and Role in the Thesis

Mistral Bench is a reproducible, non-episodic reference pipeline that runs stateless LLM inference on standard datasets and produces outputs compatible with the EM-LLM evaluation framework. The main design goals are portability via an HTTP-based Ollama adapter; compatibility with EM-LLM output formats; simplicity by running one example at a time with fixed context windows; and transparency by

recording details and timing for each item. In this thesis, the baseline sets the minimum performance level. The differences between Mistral Bench and episodic systems like EM-LLM and CXR highlight the effects of memory and prioritisation, while limiting confusion caused by changes in preprocessing or scoring.

Figure 4.2 below shows the main steps of the pipeline. It outlines how datasets are converted and pre-processed, then sent to the evaluator, which uses the Ollama-hosted Mistral model. The outputs and plots are then used in the comparison tools for the thesis tables and figures.

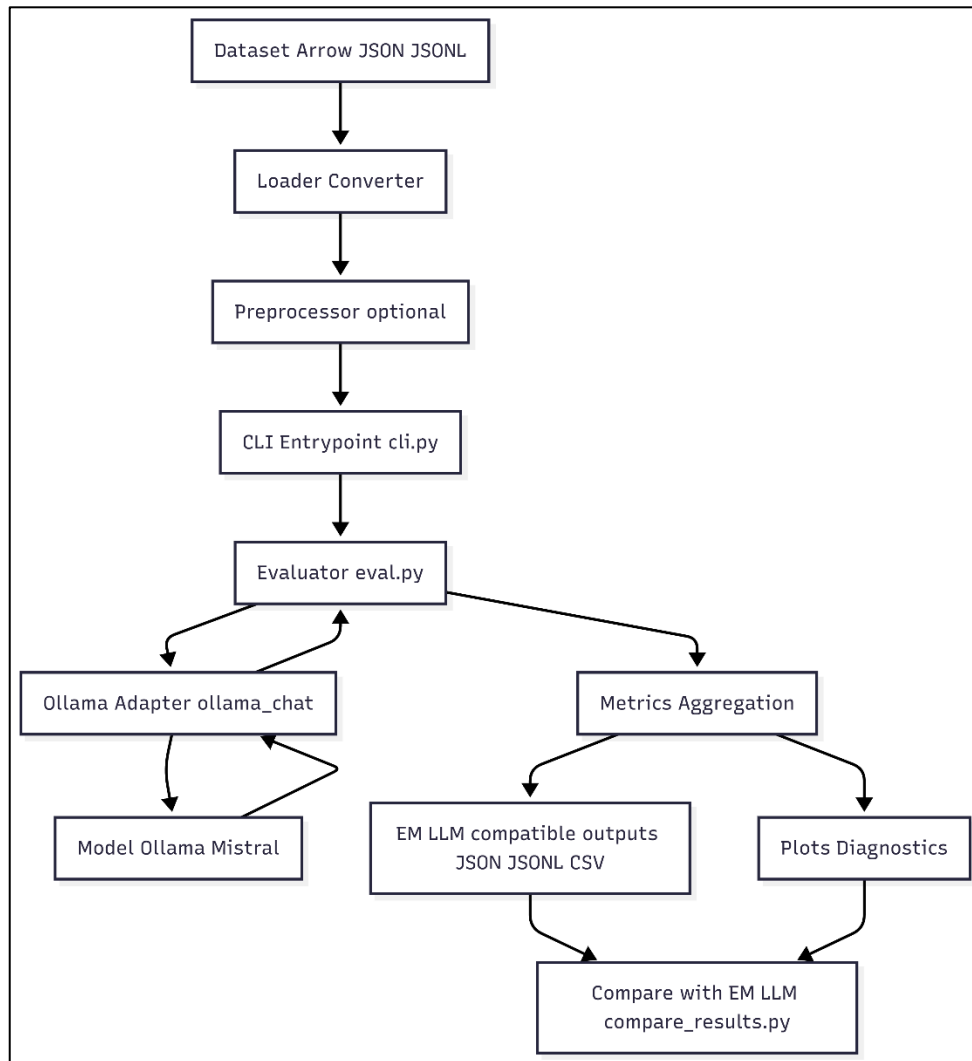


Figure 4.2. High-level pipeline for non-episodic test

Figure 4.2. shows that all flow inputs are standardized, checked using an HTTP adapter with the Mistral model, and that the process produces both individual results and overall metrics used in the comparison stage.

4.2.2 Dataset Handling and Schema Mapping (Arrow/JSON/JSONL)

This subsection documents how raw dataset files, available in several standard formats, are translated into the single, stable input representation used by the rest of the pipeline. The goal is to guarantee that the same textual inputs reach the Mistral baseline and the episodic systems, so measured differences reflect model/architecture changes rather than input mismatches.

The flow of the pipeline is as follows:

1. Entry point and format detection:
 - The CLI (`cli.py`) passes the `--data` path to the evaluator (`evaluate()` in `eval.py`). The evaluator calls `load_any(data_path)`.
 - `load_any()` implements detection: if `data_path` is a directory or ends with `.arrow` it dispatches to `load_longbench_arrow()`; otherwise, it calls `load_json_or_jsonl()`.
2. Arrow loader behavior:
 - Uses Hugging Face datasets API (`Dataset.from_file`) to read them. `arrow` file. The code expects records to contain fields such as `input`, `context` (a long text), `answers` (list), and `_id`. For each row, the loader:
 - Reads `input` - as an internal question.
 - Reads `context` - it splits on double newlines (`\n\n`) to produce paragraph-level items; this produces the internal contexts list.
 - Reads `-id` or `id` - reads internal id and if no id is present, the loader generates `q{index}`.
 - Reads `answers` (string or list) - creates a list and stores it as `answers`.
3. JSON / JSONL loader:

When parsing the file as newline-delimited JSON (JSONL), it reads lines and parses each line as JSON (memory-friendly). If that fails, it parses the file as a JSON array. The same normalization rules as above are applied.
4. Conversion utilities and dataset cleanup:

`Arrow_to_json.py` converts Arrow datasets to JSON or JSONL for inspection or manual filtering. Use it when you need to examine rows or create a filtered subset. `json_to_arrow.py` shows how filtered JSON can be saved back to HF Arrow using `datasets.Dataset.from_list(...).save_to_disk(...)`. This is useful for persisting with clean subsets across repeated experiments. For dataset-specific normalization (field names, nested structures), we include example scripts (e.g., `filter_json_initial_check_hotpot.py`) that adapt a dataset to the canonical schema.
5. Reason for the context to split into blank lines:

Many datasets store multi-passage contexts concatenated with blank lines separating passages. Splitting on `\n\n` yields clean passage candidates for the evaluator to select the first `k_ctx` passages. This deterministic heuristic avoids token-level segmentation differences across systems.
6. Robustness and edge cases:

Missing answers or contexts are handled defensively: strings are coerced to single-element lists, empty contexts are not fatal (the evaluator will pass empty context lists), and missing packages (`datasets` / `pyarrow`) will raise a clear error instructing the user to install them.

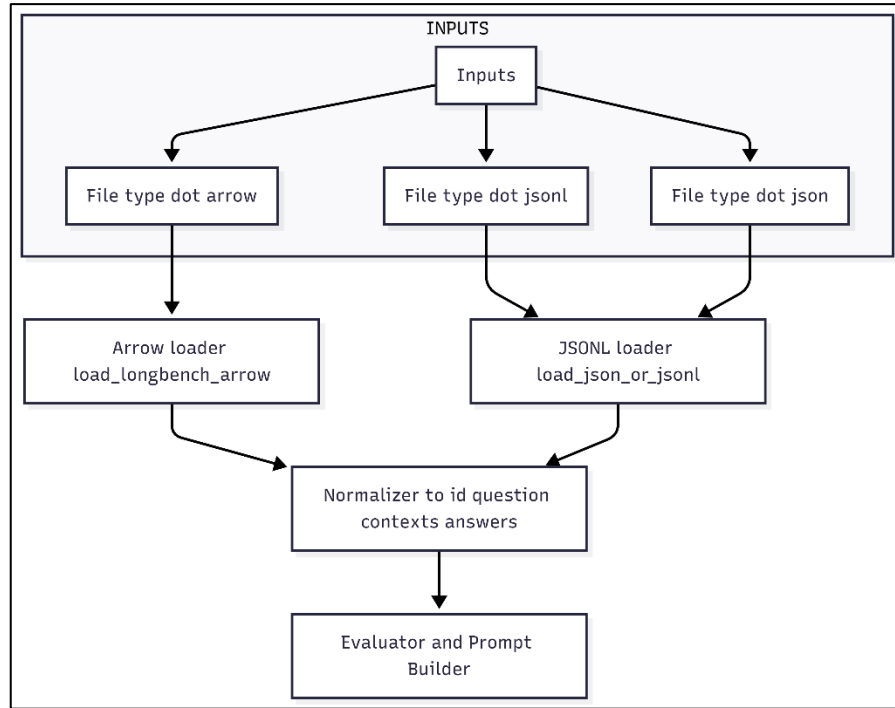


Figure 4.3. Data Ingestion & normalization

Figure 4.3. shows file-level entry points converging to a single normalized representation. This canonical shape keeps prompt construction and metric computation identical across datasets.

4.2.3 Ollama-Mistral Inference and Prompt Construction

This subsection describes how the pipeline converts a normalized input record (question + selected context passages) into a concrete HTTP request to the Mistral model (via Ollama) and how that request is constructed to produce short, comparable answers suitable for automatic scoring. In the thesis context, this stage is critical as the exact prompt template, the selection of context blocks (k_{ctx}), and the generation settings determine the behavior of the non-episodic baseline. They therefore must be identical across baseline and episodic comparisons.

Prompt design and generation policy:

- Each prompt consists of two parts: a short system instruction that constrains the model to use only the provided context and to answer concisely, and a user message that places the selected context passages, followed by the question. The system instruction used throughout the experiments is: "You answer multi-hop questions using only the provided context. Cite minimal spans if needed. Answer with a single short sentence." The user message is formatted as: "Context:" followed by the first k context passages separated by blank lines, then a blank line and "Question: <question>" followed by "Answer:".
- For example, if $k = 3$, the prompt body appears as:

Context:

 - <context passage 1>
 - <context passage 2>
 - <context passage 3>
 - Question: <the question>
- Answer: The benchmark uses deterministic generation settings where possible: temperature is set to 0.0, and a maximum token cap is applied. This reduces sampling variance and makes Exact Match (EM) and F1 comparisons more stable.

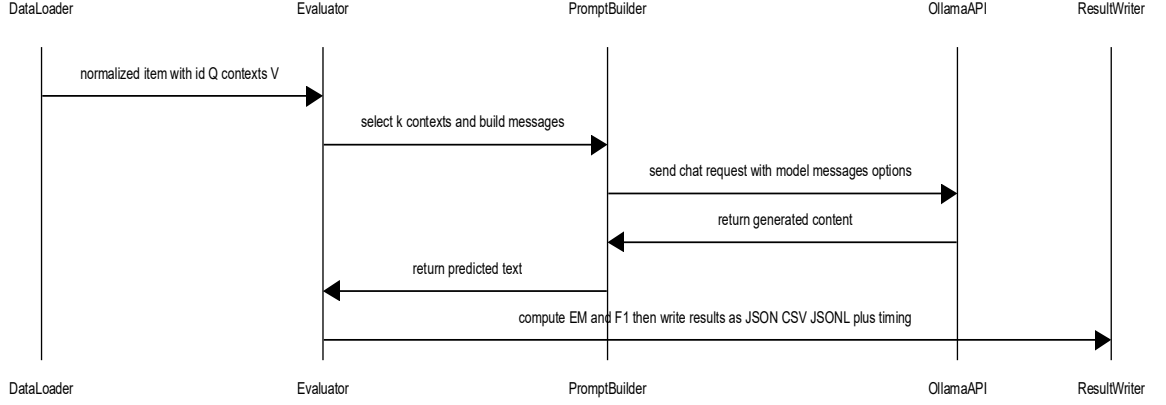
Per-item runtime flow:

1. The loader provides a normalized record containing the item identifier, the question, the list of context passages, and the list of reference answers.
2. The evaluator selects the first k context passages (the local context window) and creates the system and user messages described above.
3. The evaluator sends a single synchronous POST request to the model-backed HTTP endpoint with the chat payload and generation options. The call waits for the model response and records wall-clock generation time.
4. The evaluator extracts the first non-empty line of the model's returned text as the predicted answer.
5. The evaluator appends a per-item record that includes the id, question, contexts used, prediction, generation time, and placeholder fields for token-level diagnostics that are unavailable from this backend. After processing the batch, the evaluator computes per-item EM and F1 scores using the standard normalization across the project and writes per-item JSON/JSONL, a CSV table, and an aggregated summary JSON.

Reproducibility and practical notes:

- The prompt template, the value of k, the maximum token cap, the model tag, and the base URL of the model server are all recorded for each run. These values must be included with any published experimental artifacts to enable exact reproduction.
- The approach uses short, deterministic answers to make automatic scoring meaningful and to reduce variance between runs. If log-probability or perplexities are required, use a backend that exposes logits; the current HTTP adapter does not expose token logprobs so perplexity fields remain empty in the outputs.
- Choosing the number of context passages(k) trades off between giving the model more information and keeping the input small and comparable to a constrained non-episodic setting. For experiments reported in this thesis, k is kept consistent across baseline and episodic runs unless a specific study of context window size is performed.

In summary, the Ollama-Mistral inference stage constructs a short, constrained chat prompt from the first k context passages, issues a single synchronous HTTP call per example to the model server, takes the first returned line as the answer, records timing, and then computes EM and F1, these artifacts are saved in standardized formats so the baseline can be directly compared with episodic and memory-prioritised systems.



The above sequence diagram shows the synchronous per-example call sequence used in the Mistral Bench: the evaluator builds a constrained prompt from the first `k_ctx` context passages, calls the Ollama adapter, extracts the prediction, and records it with timing and metrics.

4.2.4 Output Formats and EM-LLM-Compatible Results

This section describes the exact artifacts produced by the Mistral Bench for each dataset, explains why the outputs are structured, and shows how those files are used to compare the non-episodic baseline with episodic systems. The goal is to create outputs that are immediately usable for tables, figures, and per-item qualitative analysis in the thesis.

Primary outputs:

- For each dataset processed by the evaluator, the following files are created below:
 - `<out_dir>/results/<dataset_name>/` - Dataset or folder name
 - `"<dataset_name>_predictions.json"` - A pretty-printed JSON array containing the full per-example objects. Each object includes `id`, `question`, `answers`, the context used (first `k_ctx`), the predicted value (`pred`, model output), `generation_time`, and placeholder fields (`total_ppl`, `block_sizes`, etc.) that remain null when the backend does not provide them.
 - `"<dataset_name>_result.json"` - Top-level EM-LLM style summary mapping the dataset name to aggregated metrics and meta fields. The `score` field contains the mean F1 scaled to a percentage, and other fields, such as `ppl_mean`, are null when unavailable.
 - `"<dataset_name>.jsonl"` - EM-LLM-compatible per-item JSON Lines: one compact object per line, with keys like `id`, `pred`, `answers`, and `generation_time`. This format is convenient for streaming and comparison tools.
 - `"<dataset_name>.json"` - The same per-item objects as a JSON array.
 - `"<dataset_name>_results.csv"` - CSV table with columns `id`, `question`, `answers`, `pred`, `EM`, `F1`, `generation_time`.
- Diagnostic plots are saved under `<out_dir>/plots/<dataset_name>/` and include:
 - `<dataset_name>_agg_metrics_bar.png` - bar chart of mean EM and F1.
 - `<dataset_name>_f1_hist.png` - histogram of per-item F1 values.

Example file content:

- One line in `<dataset_name>.jsonl` (EM-LLM compatible per-item line):

```
{
  "id": "q123",
```

```

    "pred": "Paris",
    "answers": ["Paris"],
    "length": 123,
    "token_length": null,
    "chunk_ppl": null,
    "total_ppl": null,
    "block_sizes": null,
    "mean_block_size": null,
    "generation_time": 0.345
  }

```

- A row in `<dataset_name> results.csv`:

id	question	answers	pred	EM	F1	generation_time
q123	"What is the capital of France?"	"['Paris']"	"Paris"	1	1	0.345

- The top-level `"<dataset_name>_result.json"`:

```

{
  "hotpotqa": {
    "score": 12.34,
    "len_predictions": 100,
    "ppl_mean": null,
    "ppl_std": null,
    "block_sizes_mean": null,
    "block_sizes_std": null
  }
}

```

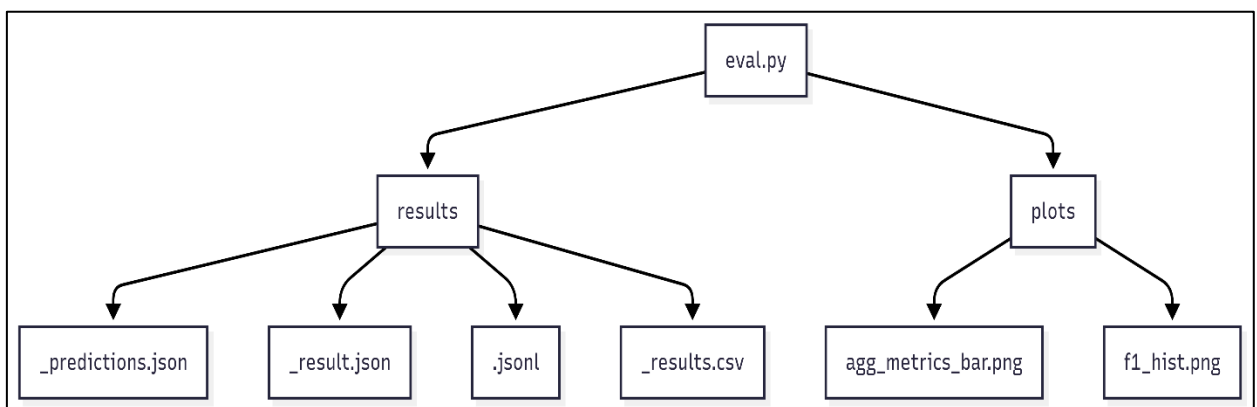


Figure 4.4. Output results and plots

Figure 4.4 shows the output artifacts produced by the evaluator and aggregate statistics, plots, and intervals used in the thesis analysis.

4.2.5 Steps to run the tool:

This section describes the steps required to set up and execute the Mistral Bench tool for running the Non-Episodic Baseline Pipeline.

Step 1: Create and Activate a Virtual Environment. A Python virtual environment is created to isolate dependencies. The environment is then activated. On Windows systems, execution policy restrictions may need to be temporarily bypassed.

```
# Create virtual environment
python -m venv .venv

# Activate virtual environment (with execution policy bypass for the session)
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force; .\venv\Scripts\Activate.ps1

# Alternative activation (if execution policy allows)
.\venv\Scripts\Activate.ps1
```

Step 2: Install Required Dependencies

1. Once the virtual environment is active, all required dependencies are installed using the provided requirements file, followed by installing the project in editable mode.

```
pip install -r requirements.txt
pip install -e .
```

Step 3: Install and Configure OLLAMA

2. The Mistral Bench relies on OLLAMA to run the Mistral language model locally. OLLAMA must be installed and run before executing the benchmark.
 - OLLAMA can be downloaded from the official website.
 - After installation, the installation can be verified using the following command:

```
ollama --version
```

If OLLAMA is not available in the system PATH, it can be accessed using its full installation path:

```
& "$env:LOCALAPPDATA\Programs\Ollama\ollama.exe" --version
```

The Mistral model is then pulled and deployed locally:

```
ollama pull mistral
```

Alternatively, using the full path:

```
& "$env:LOCALAPPDATA\Programs\Ollama\ollama.exe" pull mistral
```

Step 4: Run the Benchmark on an Arrow Dataset: After successful setup, the benchmark can be executed on a dataset stored in Arrow format. The following example demonstrates running the benchmark on the HotpotQA dataset.

```
mistral-bench -data data\hotpotqa -out runs\mistral_longbench -n 0 -k_ctx 3 -
num_predict 128
```

This command specifies the input dataset, the output directory, the number of samples, the context size, and the maximum number of tokens generated. In addition to the command-line interface, the tool also provides an optional web-based user interface implemented using Streamlit. The UI can be launched as follows:

```
streamlit run app\streamlit_app.py
```

This interface allows users to configure parameters and run experiments interactively through a browser.

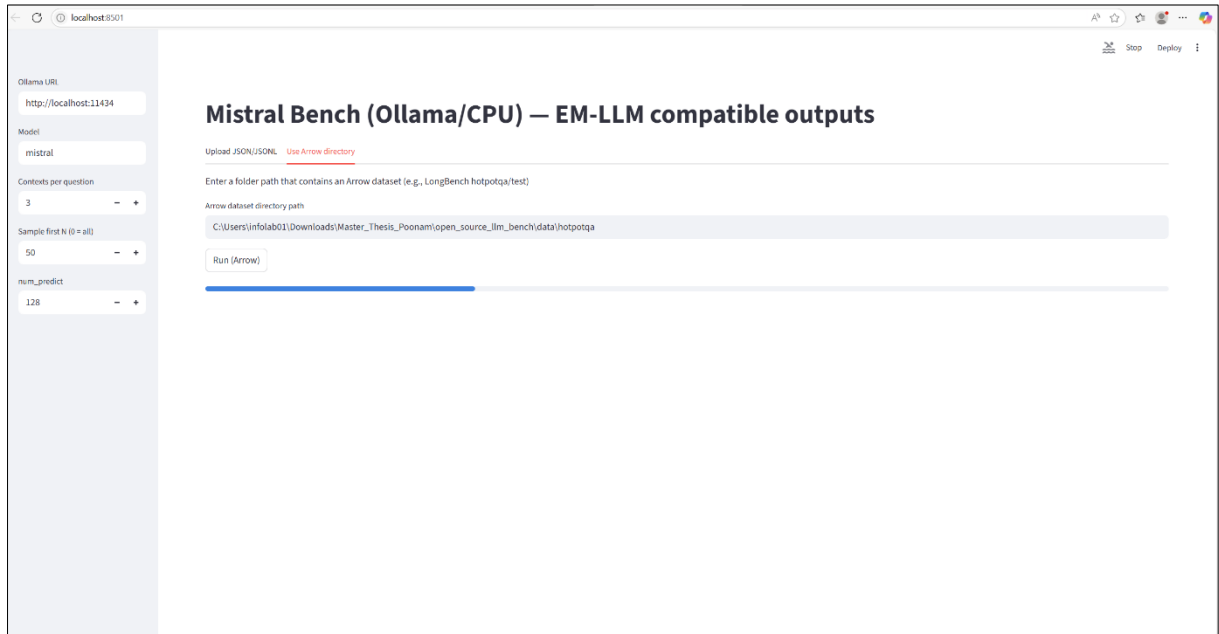


Figure 4.5. The streamlit UI interface

4.3 EM-LLM Baseline and Restoration

EM-LLM is a tool that equips transformer-based language models with a form of human-like episodic memory. It segments the input into events using a combination of Bayesian surprise and graph-theoretic boundary refinement, stores those events as memory blocks, and retrieves them via a two-stage process that combines similarity and temporal contiguity.[3]

The public codebase, however, is challenging to run on a standard single-GPU lab machine. Several components were incomplete, and environment settings (CUDA version, Linux-only scripts, specific filesystem paths) caused repeated failures. Restoring EM-LLM to a paper-faithful state, therefore, became a significant engineering task.

4.3.1 Architectural Overview of the EM-LLM Baseline

The episodic memory extension improves pre-trained language models by adding a separate memory system that mimics how people remember events, without retraining the model. Instead of making the model process more data at once, this approach breaks the input into clear, meaningful events, compresses them, and brings them back only when needed. This lets the model handle very long pieces of text without going over hardware limits.

The system uses a dual attention mechanism across two types of memory. A local sliding window that spans several thousand words retains all details from recent input, enabling the model to understand nearby context and keep the conversation clear. At the same time, a global episodic memory stores only clear sections of text that stand out as separate events, rather than every word. This design is based on

research showing that people remember experiences as individual events, especially when the situation or story changes.

Figure 4.6. explains the high-level Architectural Overview of the EM-LLM. Event segmentation combines how surprising each word is with how well its parts stick together. First, it checks which words are unexpected to find possible breaks between events. Then it uses graph analysis methods to fine-tune these breaks by measuring token similarity. This helps remove mistakes and create event boundaries that align with how people naturally perceive changes between events. Then the system stores the final events as memory blocks, and it preserves the original key-value data. Then, a set of essential tokens is selected by attention-based scoring. Memory is organized in layers: recent events are kept in GPU memory, and older ones are moved to CPU memory or disk. This setup lets the system process documents with millions of tokens using just one GPU.

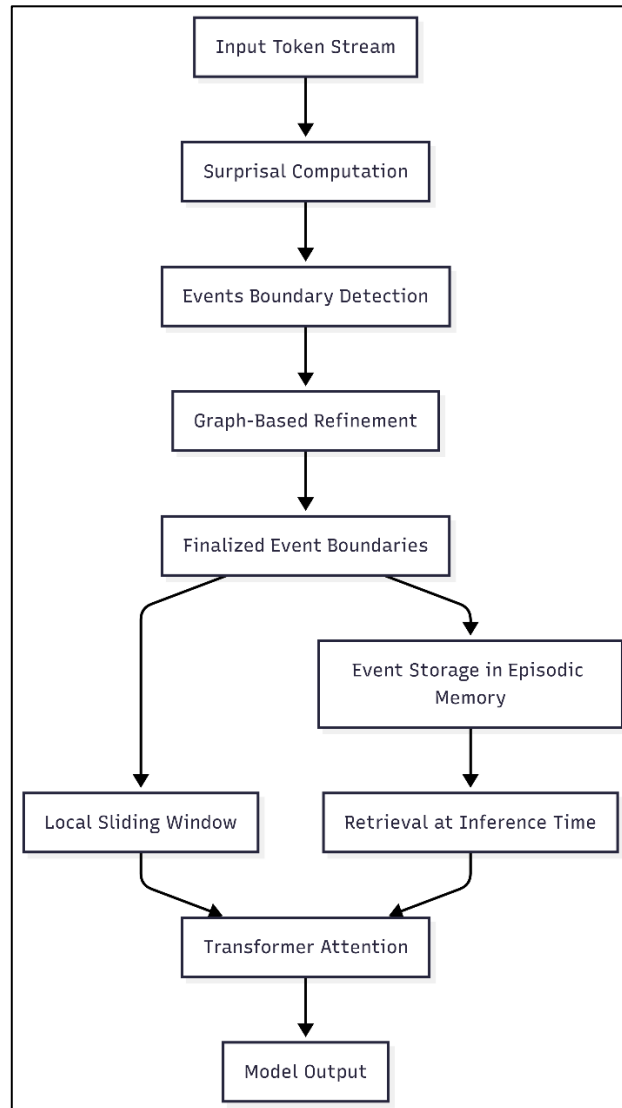


Figure 4.6. Architectural Overview of the EM-LLM

While making predictions, retrieval happens in two steps. First, the system finds the most similar events by comparing the query to the important tokens. Then it adds the events that occurred right before or after these, keeping the story connected, as humans do when they remember things that happened close together in time.

To combine the information, the model joins the key-value states from the current window with those from the retrieved events, creating a mixed context for standard transformer attention. This add-on can be used with any autoregressive language model without changing its design, allowing the model to handle longer texts than it was initially trained on while retaining what it has learned. The restoration ensured that all parts matched the original research, making it ready for use on devices with limited resources.

4.3.2 Re-Implementation of Surprisal-Based Event Segmentation

Event segmentation uses surprisal, a measure of how unexpected a token is based on the tokens that came before it. Surprisal shows that sudden changes in how predictable a word is, which appear as quick jumps in uncertainty about what word comes next, often happen when the topic or meaning of the text changes.

For token x_t at position t , surprisal is defined as:

$$\text{surprisal}(x_t) = -\log p(x_t | x_{<t}) [3]$$

where, $p(x_t | x_{<t})$ denotes the conditional probability from the language model's SoftMax-normalized logits.

Higher surprisal means the model has a difficult time predicting the next word, which often happens when the topic changes, a story starts, or the type of information changes. Peaks that exceed set limits are used to automatically identify possible breaks between clear sections, without requiring someone to mark them by hand.[40]

The restoration setup used surprisal calculation to run in real time and fit on a single GPU. First, the model runs in teacher-forcing mode, where it is given the correct word at each step and the chance it assigns to that word is saved. These values are maintained in a moving buffer that stores surprisal scores for a recent group of words, typically a few hundred at a time. In this group, the system looks for high points where surprisal is higher than the words right before and after, and higher than a changing limit based on recent data:

$$\theta(t) = \mu(t) + \gamma \cdot \sigma(t)$$

where, $\mu(t)$ and $\sigma(t)$ Represent the mean and standard deviation of windowed surprisal values, and γ is a tunable scaling factor controlling detection sensitivity. Values of $\gamma = 1.0$ Mark boundaries at peaks exceeding one standard deviation above local means; higher values produce fewer, more pronounced boundaries. This adaptive thresholding ensures robustness across documents with varying linguistic characteristics, automatically adjusting to baseline unpredictability inherent in each text.

To handle sequences that are too large for the GPU, the data is split into smaller parts and processed one after another, ensuring cross-chunk connections remain intact.

- Chunk-level processing: Each part has a set number of tokens, usually matching the main model's size or a portion of it, to use memory well and work efficiently.
- Cross-chunk continuity: The surprisal buffer moves from one part to the next, ensuring essential points near the edges are not missed.
- Boundary candidate accumulation: Peaks found in each part are added to a main list, which is then improved further.

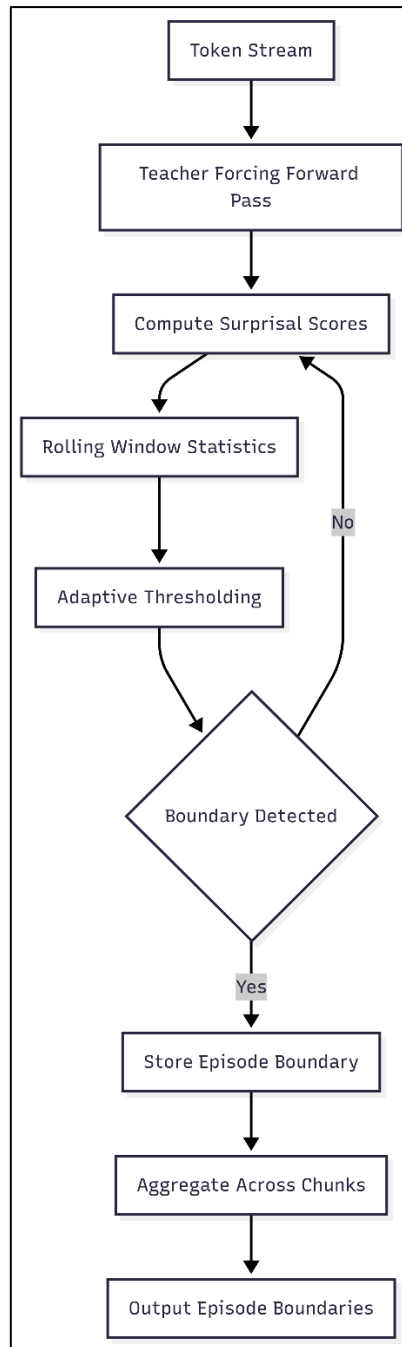


Figure 4.7. *Surprisal-Based Boundary Detection Pipeline*

Figure 4.7 shows how the chunked architecture handles documents of any size by sending the text through the model in smaller, manageable chunks, while ongoing surprisal buffers help maintain consistent event detection over time. Event segmentation based on surprisal uses only the main model's probability outputs and does not require external supervision or special rules for specific topics. The restoration ensured that all settings, buffer sizes, threshold adjustments, and chunking methods matched the original research, preserving the intended event-splitting method. The possible boundaries found at this point are then sent to a graph-based step, which assesses how well each segment fits together and adjusts the boundaries to better align with meaningful events. Together, detection based on information and validation based on graphs yield strong segmentations that break long sequences into episodes suitable for selective retention and retrieval.

4.3.3 Graph-Based Memory Structuring Using Topological Metrics (Modularity, Conductance)

Surprisal-based segmentation provides initial episodic boundaries, but they are often noisy and may not capture meaningful transitions. The surprisal signal responds to unexpected events, such as lexical surprises, syntactic ambiguities, or brief shifts in discourse, rather than to actual episodic boundaries. To improve this, the baseline uses graph-based refinement. It evaluates and adjusts candidates using topological metrics derived from the model's internal representations. This process builds similarity graphs over token representations and uses community-detection methods to find cuts that keep segments internally cohesive and separate them where there are real content breaks. This way, the final boundaries mark areas of strong semantic unity divided by actual discontinuities in content.

The similarity graph is constructed from hidden-state representations extracted during the forward pass. For tokens spanning windows around candidate boundaries, final-layer hidden states compute pairwise cosine similarities, yielding a symmetric adjacency matrix. A where entry A_{ij} quantifies representational similarity between positions i and j . Formally, for hidden-state vectors h_i and h_j : [41]

$$A_{ij} = \frac{h_i \cdot h_j}{\|h_i\| \|h_j\|} [41]$$

This encodes the intuition that tokens within coherent segments exhibit higher mutual similarity than tokens spanning boundaries between distinct topics. The graph undergoes community detection analysis, wherein potential boundary positions are evaluated by how well they partition the graph into internally cohesive communities.

Modularity, from network science, measures partition quality by the concentration of edges within versus between communities. For partitioning the graph into communities C_1 and C_2 at boundary b :

$$Q = \frac{1}{4m} \sum_{c \in \{C_1, C_2\}} \sum_{ij \in c} (A_{ij} - \frac{k_i k_j}{2m}) [42]$$

where $m = \frac{1}{2} \sum_{ij} A_{ij}$ is the total edge weight, and $k_i = \sum_j A_{ij}$ is node i degree. The term $\frac{k_i k_j}{2m}$ represents the expected similarity under a null model with random edge placement that preserves degrees. High modularity indicates observed within-community connections significantly exceed random expectations, suggesting successful separation of cohesive but distinct segments. Refinement evaluates candidate positions within local search windows, selecting maximum-modularity positions as refined boundaries.

Conductance provides an alternative perspective by directly quantifying segment separation. For community S and complement $\bar{S}$:

$$\phi(S) = \frac{\text{cut}(S, \bar{S})}{\min(\text{vol}(S), \text{vol}(\bar{S}))} [43]$$

where $\text{cut}(S, \bar{S}) = \sum_{i \in S, j \in \bar{S}} A_{ij}$ measures boundary-crossing similarity, and $\text{vol}(S) = \sum_{i, j \in S} A_{ij}$ measures internal volume. Low conductance indicates few connections relative to internal density, signifying clean separation. Refinement minimizes conductance, selecting positions with the lowest values to identify edges that most effectively isolate adjacent segments.

The refinement process employs adaptive search, balancing precision with efficiency. For each candidate boundary, local search windows are defined with bounds proportional to the preceding event size, extending symmetrically around the original positions. Within windows:

- Submatrix extraction: Relevant adjacency matrix portions are extracted, reducing metric evaluation cost.
- Candidate evaluation: Selected metrics (modularity or conductance) compute for all positions satisfying minimum event size constraints, producing score profiles.
- Optimal selection: Positions achieving optimal scores (maximum for modularity, minimum for conductance) replace original candidates.
- Constraint enforcement: Minimum event size thresholds prevent excessively short segments violating design assumptions.

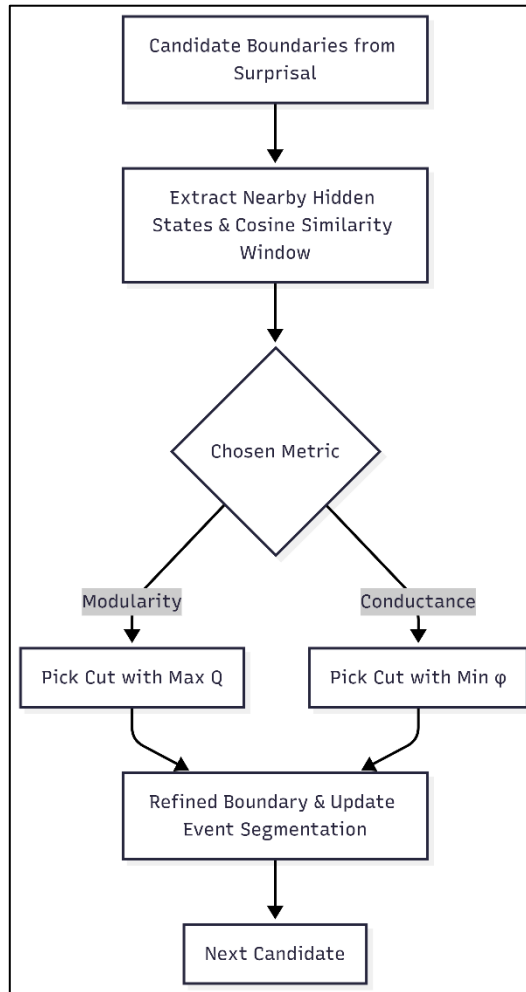


Figure 4.8. Graph-based refined Boundary Detection Pipeline

Figure 4.8. shows graph-based refinement, which addresses the problem of relying solely on surprisal by basing decisions on data patterns. Modularity makes sure that segments are more connected inside than you would expect by chance, while conductance makes sure that segments are clearly separated with little overlap at their edges. The review confirmed that both measures were used precisely as described and that the flexible search method balanced improving small parts with the overall quality of the segments. Combining surprise with how groups form in graphs creates event boundaries that align well with how people naturally perceive episode changes, as found in the original research with human-marked boundaries. This two-step process, first finding surprising points and then refining the structure, turns a stream of tokens into organized memories that can be stored and found later.

4.3.4 Contiguity Buffer and Episodic Memory Management

The episodic memory system is built to preserve the natural order and story-like flow of human memory, rather than just saving and retrieving individual events based on what they mean. Research shows that when people remember something, they often also remember other things at the same time. This is called temporal contiguity. The system copies this by keeping events that happened just before and after the one being remembered in a special memory area. By making these similar events easy to find when a memory is recalled, the system keeps the story clear and builds links between different events. To find the most similar events, the special memory area needs enough space, often saving 20 to 40 percent of nearby events.

The memory system stores information in layers, allowing it to handle sequences of any length, even with a single GPU. Each event is saved as a memory block containing the original data and smaller versions generated from selected parts using specialized scoring methods. These blocks are split into three groups: active blocks stay in GPU memory for quick access, warm blocks move to CPU memory and can be brought back fast, and cold blocks are stored on disk, which takes longer to reach but lets the system handle much longer sequences. The system decides where to place each block based on when it was last used and manages space in advance. When there are too many active blocks, the least-recently used blocks are moved to CPU memory to make space for new events. If the system is almost complete, more blocks are transferred to disk and saved as small files in neat folders for quick searching and loading. The special memory area is used when getting memories by changing which events are shown to the model during predictions. After finding the most similar events, the system checks their original order.

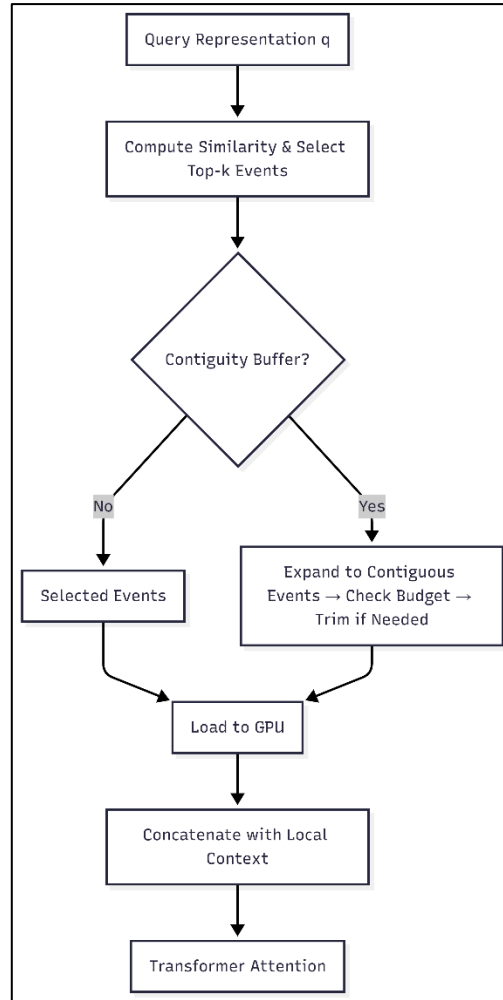


Figure 4.9. Hierarchical Episodic Memory Management and Retrieval

Figure 4.9 shows how the system manages and quickly finds event blocks. It first checks if a block is already in GPU memory. If it is not, it loads it from the CPU or, if needed, from disk, and moves it up through the different levels. A special cache keeps only the most recently used blocks on the GPU, so the system does not move the same data more than once.

When a search finds the top-k most relevant events, the system can also include nearby events to keep the story connected. If this larger group is too big, the system removes the farthest events until it fits the allowed size. The final group of events is then combined with the local context and sent to the Transformer. By using similarity-based searches, keeping related events close in time, and storing data across layers, the system enables fast, well-organized memory access even with limited GPU space.

4.3.5 Interaction Between Local Context Window and Global Episodic Memory

The episodic memory system uses two parts that work together: a local context window that provides detailed access to recent words, and a global memory that retrieves important information from earlier events. These parts operate in parallel while the model is making predictions, with the attention system attending to both sets of data, ensuring results include both recent and essential past information. Their combination is managed by a step-by-step attention process that lets information flow smoothly while preserving the unique timing and meaning of each memory type.

The local context window keeps a moving list of the most recent words, usually set to match or slightly go beyond the model's usual limit. It works like a line: as new words are created, the oldest ones are removed from the window and placed in a global buffer, where they wait to be grouped into events. This

window catches short-term links like references, grammar agreement, and immediate clues in the text that are needed for smooth writing. It provides a clear view of the current language by maintaining the exact word order rather than using shorter versions, which is important for tasks that require exact word or grammar details. It is also where surprise values and scores are computed, which are later used when moving words from local to global memory. The data for the local window is kept in full detail in fast memory during predictions, so checking recent context is very quick.

Global episodic memory, on the other hand, serves as a store that only retains and retrieves important information. Instead of removing every word from the local window, it keeps only words that are part of events deemed important to the current question, based on how similar they are and how close in time they occurred. Events are shown with their original details, along with shorter summary versions generated from keywords, making it easy to compare them without having to look at every word from the past. Each time the model needs more context than the local window gives, the global memory system looks for matches: it compares the current question, usually by averaging the new word details, to the summaries of all saved events, ranks them by how similar they are, and loads the best matches into fast memory if they are not already there. This process varies by question, so different parts of a document may bring up different sets of past events depending on what is being discussed and how closely the topics match.

Bringing together local and global data for attention uses a step-by-step process that keeps their distinct roles clear while allowing them to work together. Each time the model runs, attention first looks at the local context window, using a standard way to compare the current question to local data and make a partial result based on recent context. Then, the system adds the global memory data it found, joining it with the local data in order they appear. Global data looks like extra input words to the model, but their positions are set to match where they were in the original document, not just where they are in the combined list. This ensures the model's sense of word order remains correct, even when it recalls information from much earlier in the text.

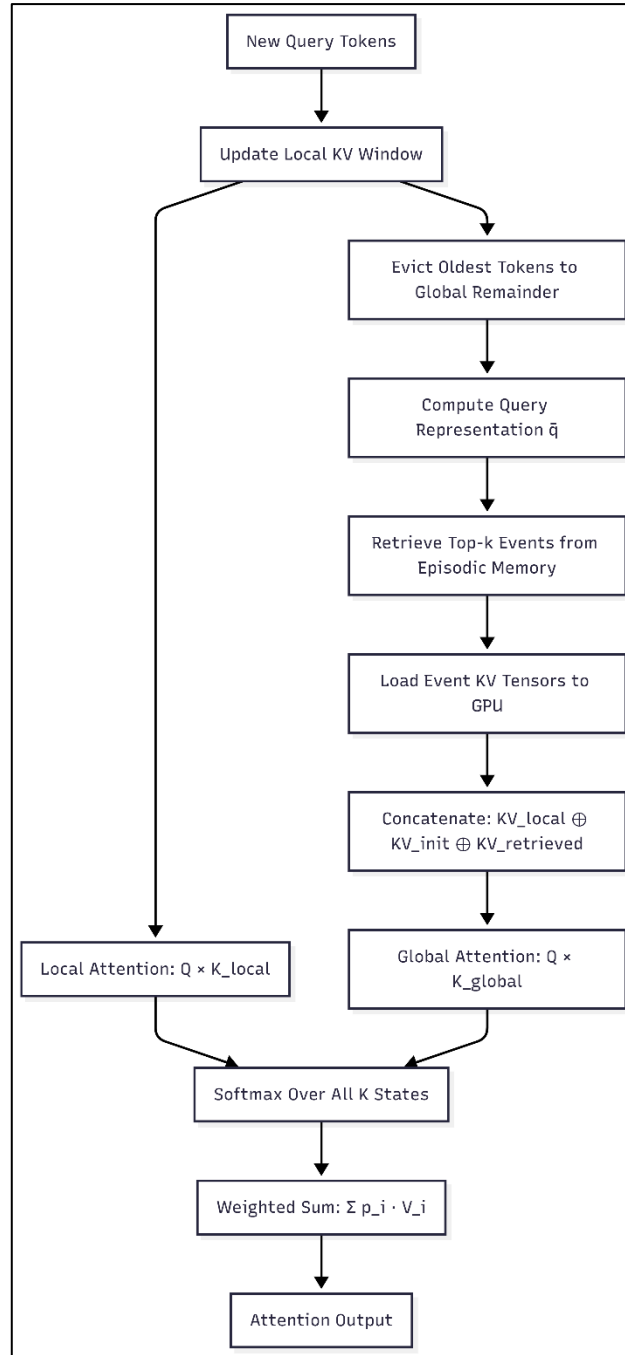


Figure.4.10. Attention Computation Using Sliding Window and Global Memory

Figure 4.10. explains that attention in the system operates over a combination of local, prefix, and retrieved memory states. The local window uses a simple rule, so each token considers only earlier tokens in the nearby buffer. This keeps the order of information and avoids extra work on far-away tokens. Retrieved events, on the other hand, act as background knowledge that can be used from any position, not just the current stream.

The model also keeps a buffer with the first tokens of the document. Even though these tokens are usually not very important for meaning, they help keep attention steady in very long sequences. Studies show that keeping these early tokens stops the attention system from breaking down, especially when dealing with very long sequences. That is why the prefix part is always included in the main set and comes before any retrieved events.

When the system calculates attention, it combines and balances the scores from the local, prefix, and episodic parts. This allows the model to focus on nearby context, steady starting points, and chosen long-range information. By keeping local and global attention rules separate, the design stays both accurate and flexible.

Altogether, this three-part memory setup local recency, important events, and prefix stability lets the model handle much more information than usual. It helps the model make sense of large documents while remaining efficient and maintaining steady attention, which is needed for tasks that require long-range understanding.

4.3.6 Inference-Time Memory Retrieval and Conditioning

Inference-time retrieval helps a language model leverage important past information without having to read entire documents each time. By selecting only the most useful earlier events, the system allows a standard Transformer to leverage selected long-term memory. Because retrieval must work with limited memory and speed, it needs to quickly find similarities, move data easily between different storage, and add the retrieved content into the attention process.

Retrieval begins when the model processes a new group of tokens. After generating the query, key, and value versions, the system averages the query vectors from the current group to produce a summary query vector. This vector captures the model's current context and serves as the retrieval signal. The system compares this signal to smaller versions of all stored events and ranks them by how similar they are to it. Each event's smaller vector is derived from the key token representations chosen during the first step. The top events are picked based on the memory limit, so the total number of retrieved tokens stays within set limits.

Once events are chosen, the system makes sure their key-value data is on the GPU before attention runs. It does this in steps: events already on the GPU are used right away, those on the CPU are copied to the GPU, and those on disk are first loaded to the CPU, then moved to the GPU. To manage GPU space, the system removes the items used least recently and keeps recent ones, expecting they will be needed again soon. This step-by-step method lets the model handle extensive documents while keeping GPU memory use reasonable.

Using retrieved events in the model means the system must handle position information carefully. Transformers use position codes to indicate order and distance, so retrieved tokens need to retain their original positions rather than being assigned new ones based on where they are added. The system uses rotary position codes with precise offset handling, so each event's keys and values show their real positions in the original document. This helps the model keep its understanding of grammar and meaning over long distances, treating retrieved events as if they were still part of the original input. The system runs queries and stores events for each attention head separately. Because different heads look at other language patterns, scoring for each head can make retrieval more accurate by finding events important to specific parts of the model. These head-level similarities are then combined into one relevance score for each event. While this method requires more computing power, it allows for more precise retrieval for tasks that require detailed analysis of language differences.

After retrieval, events are loaded, and their positions are set correctly. They are combined with the local context window and given to the Transformer as a longer input. The model then runs its usual attention process, where learned attention weights balance local context and retrieved content. This smooth combination is key to the system's design: it works with any pre-trained Transformer without changing its structure or retraining. In practice, retrieval speed stays reasonable even for documents with hundreds of thousands of tokens, cache hit rates remain high, and model performance is better than with shorter fixed windows. By using similarity-based retrieval, layered memory management, and correct positions, the system can handle unlimited contexts while keeping clear long-range reasoning.

4.3.7 Practical Constraints of Single-GPU Baseline Restoration

Restoring the episodic memory system on a single-GPU computer posed real-world challenges that affected its design and use. Unlike the original research setup, which likely used several computers with ample memory, the single-GPU setup required careful management of memory use, data transfer, and system-specific limits. Because of this, the restoration became mostly a technical task focused on keeping the original design working with fewer resources, demonstrating how memory-augmented models can operate in typical academic settings.

```
self.apply_rotary_pos_emb(q, q.size(seq_dim), k.size(seq_dim), self.cos_cached.to(q.device), self.sin_cached.to(q.device)),
File "C:\Users\infolab01\Downloads\Master_Thesis_Poonam\EM_LLM_Implementation\EM-LLM-model\em_llm\attention\rope.py", line 48, in apply_rotary_pos_emb
    return ((x.float() * cos) + (self.rotate_half(x).float() * sin)).to(dtype)
torch.cuda.out_of_memory: CUDA out of memory. Tried to allocate 20.00 MiB. GPU 0 has a total capacity of 11.99 GiB of which 0 bytes is free. Of the
allocated memory 18.08 GiB is allocated by PyTorch, and 321.17 MiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try
setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation. See documentation for Memory Management
(https://pytorch.org/docs/stable/notes/cuda.html#environment-variables)
Directory /c/Users/infolab01/Downloads/Master_Thesis_Poonam/EM_LLM_Implementation/EM-LLM-model/benchmark/results/mistral/long-bench/offload_data does not
exist.
C:\Users\infolab01\Downloads\Master_Thesis_Poonam\EM_LLM_Implementation\EM-LLM-model\.venv\lib\site-packages\fuzzywuzzy\ fuzz.py:11: UserWarning: Using
slow pure-python SequenceMatcher. Install python-Levenshtein to remove this warning
warnings.warn('Using slow pure-python SequenceMatcher. Install python-Levenshtein to remove this warning')
Evaluation on: ['log', 'result.json']
```

Figure 4.11. CUDA Out of memory error

GPU memory capacity was the most severe limitation. Modern large-scale Transformers already occupy several gigabytes of VRAM for model weights alone, leaving limited space for key-value caches, activations, and episodic memory blocks. Early experiments using default configurations frequently triggered out-of-memory failures, as illustrated by the representative CUDA error shown in Figure 4.11. System profiling indicated that cache sizes, batch dimensions, and block allocation policies required careful tuning. The final configuration adopted a three-level memory hierarchy: active blocks remain on the GPU, recently accessed blocks are moved to host memory, and older blocks are serialized to disk. This design enabled access to an effectively unbounded history while keeping VRAM usage within the 16-24 GB range typical of consumer GPUs. The primary drawback was increased latency when loading disk-resident blocks, making the cache's behavior critical to maintaining acceptable throughput.

The layered memory system also caused slowdowns when moving data. Moving blocks from the computer's main memory to the GPU over the PCIe connection takes a few milliseconds, and this adds up if many blocks are loaded at once. To make this faster, the restoration used transfers that occurred at the same time as other work, so memory could be moved while the model was running. Using a special kind of memory in the computer also made transfers faster. Even with these improvements, there were still limits, as shown by slowdowns when too many blocks were loaded at once. The final setup limited memory retrieval to about 2,000 tokens per run, balancing the amount of history that could be used with the speed of data transfer.

Differences between computer systems caused more problems. The original version expected a Linux system with certain file behaviors, but the restoration was done on Windows, which handles file paths, shortcuts, and memory files differently. These differences caused errors, including incorrect file paths and failures during disk saves. The restoration fixed these problems by ensuring file paths were consistent, creating folders directly, and avoiding shortcuts. Hugging Face model caching, which uses shortcuts to save space, also did not work well with Windows permissions, so saving complete copies of models fixed these errors but used more disk space.

Software version requirements added more limits. The system needed Python 3.10.11 to work with specific versions of the Transformer libraries, CUDA, and other needed software. Even small version changes caused errors or hard-to-find problems, so the setup had to use carefully controlled environments with exact software versions. Logging in to protected model repositories also required special setup, since mistakes in environment settings caused confusing errors that appeared to be model-loading problems.

Despite these challenges, the restoration demonstrated that episodic memory systems can operate effectively on a single-GPU computer. The final setup handled documents with tens of thousands of tokens, maintained reasonable processing times, and retained the main ideas of the original research. Even though it was slower than setups with more GPUs, the speed was still good enough for research. Overall, the experiment shows how important it is to carefully design memory systems, move data efficiently, handle diverse computer systems effectively, and control the software environment when using advanced memory models outside powerful computing setups.

4.4 Cross-Event Reasoner (CXR) Design

EM-LLM is effective at recalling relevant events, but it primarily treats them as independent; each event is scored and treated on its own, with little focus on how they relate. Inspired by how people remember experiences as stories and connected moments, this thesis looks at a Cross-Event Reasoner (CXR) as a simple way to prioritize events. CXR is designed as a tool that requires no additional training and operates on top of the events identified by EM-LLM. Due to the computational constraints, only some parts of the design are fully built, while the rest are planned for future scope. The sections below explain the planned design.

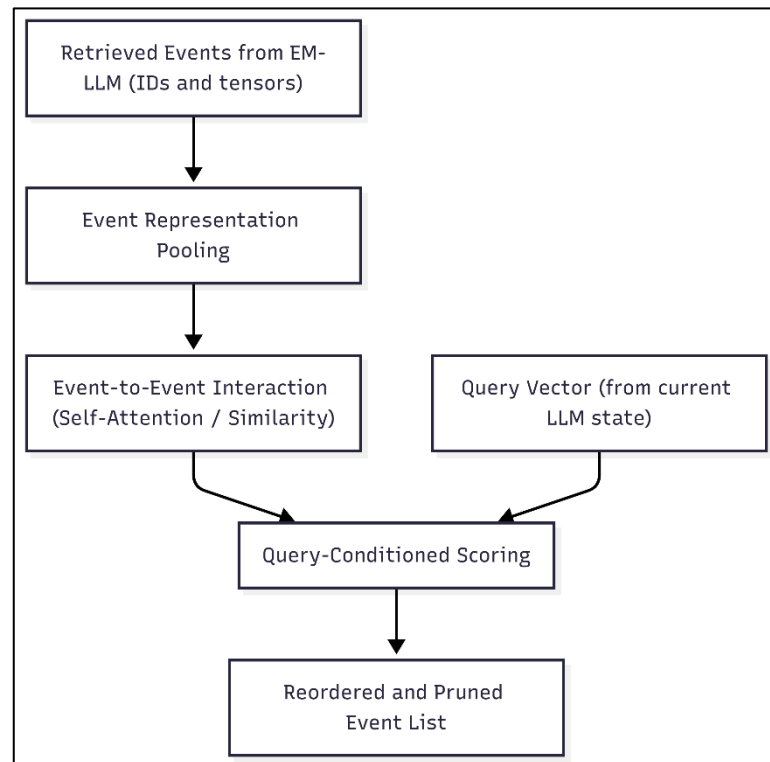


Figure 4.1213. Conceptual CXR architecture

Figure 4.12 shows the conceptual CXR architecture. Retrieved events are pooled into event vectors, processed by an event-to-event interaction module (self-attention or similarity graph), combined with a query representation, and then re-scored and re-ordered before being passed to the LLM.

Here, the Cross-Event Reasoner (CXR) is considered in isolation. EM-LLM first finds a set of possible events, then CXR gets their IDs and data. In the first step, each event is turned into a single summary, for example, by averaging its word features. These event summaries are then sent to a module that examines how events relate to one another, such as a simple attention layer or a similarity chart, which identifies links between events based on shared entities, topics, or temporal connections, with the event representations to generate query-conditioned scores. In the final stage, events are re-ranked according

to these scores, and low-ranking events may be discarded. Retrieved events are pooled into event vectors, passed through an event-to-event interaction module combined with a query representation, and then re-scored and re-ordered before being passed to the LLM.

4.4.1 Event Representation Pooling

Each event retrieved by the basic memory mechanism includes multiple token-level representations spanning the event's temporal extent, such as hidden-state vectors and key-value tensors saved during the initial encoding pass. To allow comparison of entire events rather than just parts, these pieces need to be combined into a single summary for each event. This combining step, called event representation pooling, reduces the input to the next layer while preserving the information needed to compare events and answer questions.

The pooling method uses the language model's final-layer hidden state for each part of each event. There are three main ways to do this:

- **Mean pooling:** Computes the arithmetic mean of all token representations within an event, treating each token as equally informative. This baseline approach provides a balanced summary vector that captures the overall semantic content without requiring additional parameterization.
- **Attention-weighted pooling:** This method gives different importance to each token based on surprisal values or internal scores from baseline retrieval. Tokens with high surprisal, which are more informative, get more weight in the summary, highlighting key content.
- **Key-vector pooling:** Aggregates the attention key vectors stored in episodic memory blocks rather than hidden states, leveraging the representations already optimized for similarity-based retrieval.

The current version uses mean pooling as the main method due to its computational simplicity and lack of learnable parameters, ensuring the cross-event reasoning mechanism operates as a training-free augmentation compatible with arbitrary pre-trained models. The combined event summaries, which are usually composed of thousands of dimensions matching the model's hidden size, are then used as input to the subsequent cross-event interaction modules.

4.4.2 Event-to-Event Multi-Head Self-Attention

Once events are turned into compact vectors, the cross-event reasoning mechanism uses a simple self-attention process to model relationships between episodes. This layer helps the system find links like shared entities, ongoing topics, or cause-and-effect connections across different events. In this way, it mimics how human memory works, where remembering one experience often brings up related memories.

The self-attention mechanism treats the collection of event vectors as a sequence, computing pairwise affinities to produce context-enriched representations. For a set of N retrieved events represented as matrix $E \in R^{N \times d}$, the mechanism computes:

$$E' = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V[44]$$

where Q , K , and V denote query, key, and value projections of the event representations. In the deterministic version, these projections use identity mappings. This avoids adding learnable parameters and makes deployment easier, while still modeling interactions. The attention matrix shows how much each event relates to others. High attention weights mean events have strong semantic overlaps or are connected in the story. The updated representations include information about nearby events. If two events have high attention to each other, they probably share entities, cover similar topics, or help answer the query together. The self-attention layer is kept simple, usually with one attention step and a small hidden size. This reduces computational cost but still captures basic relationships between events, like shared names or ongoing themes across episodes.

4.4.3 Event Fusion and Event-Interaction Graph

The attention weights from the event-to-event self-attention mechanism can be viewed as an event-interaction graph. Here, nodes represent retrieved episodes, and edges indicate the strength of semantic or narrative links. This method makes the model easier to understand and supports structured reasoning about events, as humans do when building mental timelines or connecting related experiences.

The event-interaction graph is mainly used in two ways:

- Visualization and interpretability: Showing events as nodes and attention scores as edge weights helps the graph highlight clusters of closely connected events. These clusters often match clear sub-narratives or chains of evidence related to the query. Events about the same person, place, or topic form dense groups, revealing the hidden structure in the retrieved memories.
- Cluster-based fusion: Events that are firmly connected, with mutual attention weights above a certain threshold, can be combined into a single summary using pooling. This reduces redundancy when several events describe the same fact in different ways or with other details, merging overlapping information while keeping unique details from less central events.

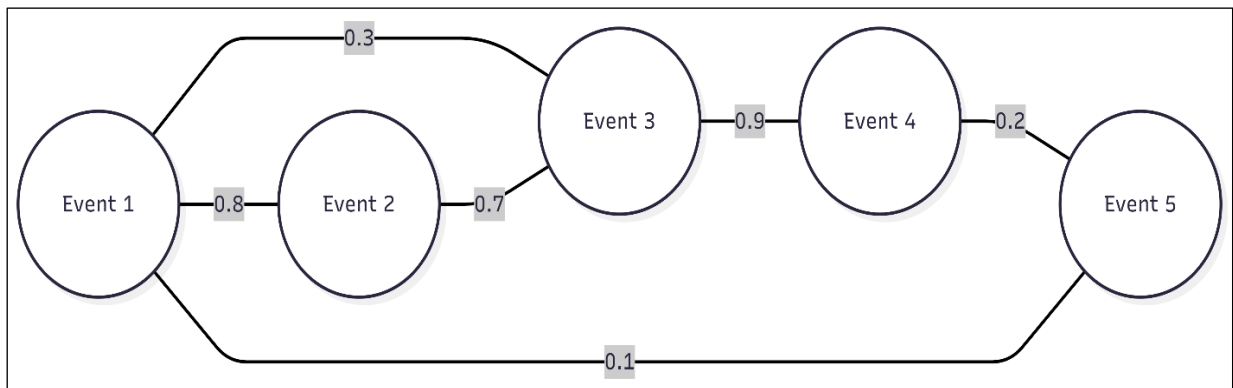


Figure 4.13. Sample Event Interaction Graph

The diagram 4.13. shows an event-interaction graph with five events. In this graph, the edge weights represent how similar or related each pair of events is. Events 1 and 2 are closely related, with a similarity score of 0.8, suggesting they are closely connected topics or the same entity. Events 3 and 4 also have a high similarity score of 0.9, suggesting they share a lot of content, such as being parts of the same story or offering different views of the same fact. In contrast, Events 1 and 5 have a low connection score of 0.1, so they probably discuss different topics. Events 2 and 3 have a moderate connection (0.7), indicating that Event 3 could link the group of Events 1 and 2 to the group that includes Event 4. This setup shows that Events 1, 2, and 3 are closely connected and form a clear narrative, while Events 4 and 5 are more on the edge, adding specific details but not connecting much to the main group.

The implementation computes the cross-event similarity matrix by normalizing event representations and computing pairwise cosine similarities, then derives a centrality measure for each event as the mean similarity to all other events. In later stages, it gives each event a centrality score by averaging its similarity to all other events. Events with high centrality, like Event 3 in this example, are at the center of the graph because they connect to many different events. These central events often serve as key points or main themes in the collection. Although more advanced operations, such as path-based multi-hop reasoning over the event graph, are left for future work due to computational constraints, the existing graph structure already supports prioritising central events that connect multiple narrative threads. This helps improve the coherence of the final retrieved context provided to the language model. The graph visualisation, therefore, functions both as an interpretability tool for examining which events the mechanism considers interconnected and as a basis for fusion operations that merge redundant information, ultimately increasing the efficiency and relevance of the episodic memory retrieval process.

4.4.4 Query-Conditioned Scoring

To ensure that event prioritization adapts dynamically to the specific information needs of the current query, the cross-event reasoning mechanism uses query-conditioned scoring that combines baseline retrieval scores with query-specific relevance assessments. This design reflects the insight that event importance is inherently context-dependent: an episode that is highly relevant for one query may be less important for another, which makes adaptive re-ranking based on the immediate inference context necessary.

The query representation is derived from the language model's hidden states at the final token of the current input prompt, or, in question-answering tasks, from a pooled representation of the question segment. This query vector is normalized and compared against the context-enriched event representations produced by the self-attention layer, yielding query-to-event similarity scores. The final score for each event i is computed as a weighted combination:

$$\text{score}_i = \alpha \cdot s_i^{\text{baseline}} + \beta \cdot [q, e'_i] + \gamma \cdot c_i [45]$$

Where:

- s_i^{baseline} denotes the original similarity score from baseline retrieval.
- $[q, e'_i]$ represents cosine similarity between the query vector q and the updated event representation e'_i after cross-event interaction.
- c_i encodes the event's centrality within the interaction graph, capturing its connectivity to other events.
- α , β , and γ are scalar weights controlling the relative influence of each component.

In the training-free approach, these weights are set using simple rules. Typically, α is set to 0.7 to focus on baseline retrieval quality, and β is set to 0.3 to add query-specific adjustment and centrality. This allows the system to work without needing validation data or gradient-based optimization. The scoring method uses three main factors: baseline semantic relevance, how well the result matches the query, and its structural importance in the event network. Together, these factors create a relevance score that reflects both content similarity and the context of relationships.

4.4.5 Final Event Reordering and Pruning

The final stage of the cross-event reasoning pipeline reorders and prunes the retrieved events based on their adjusted relevance scores, yielding a refined subset of episodes that are closely aligned with the current query and narrative context. This selection determines which historical information is presented to the language model during attention computation and, therefore, directly impacts the quality and coherence of the generated outputs.

Events are sorted in descending order according to their combined scores, and the top- k events are retained for inclusion in the effective context. The value of k is determined by practical constraints, such as GPU memory limits and the maximum context length of the language model, while balancing the need to provide sufficient historical information to support coherent reasoning. Events that fall below the selection cutoff are removed from the current forward pass but remain stored in long-term episodic memory for possible retrieval in future queries.

The reordering procedure is intentionally designed to be transparent and interpretable:

- Rank comparison: The system records both the original ranking produced by baseline retrieval and the adjusted ranking after cross-event reasoning, allowing analysis of how query conditioning and inter-event relationships influence prioritisation.
- Score decomposition: For each event, the separate contributions of baseline similarity, query alignment, and centrality to the final score are logged, enabling diagnostic inspection and hyperparameter tuning.

- Pruning strategy: Beyond simple top-k selection, the mechanism also supports adaptive pruning, in which events with scores below an absolute threshold are excluded regardless of rank, preventing weakly relevant content from diluting the context.

The reordered event set is then passed to the baseline attention mechanism, where events are loaded into GPU memory, their key-value states are concatenated with the local context window, and standard transformer attention is applied. By dynamically prioritising events that are both individually relevant and structurally important within the retrieved memory network, the cross-event reasoning layer improves the language model’s ability to integrate information across multiple episodes, supporting tasks that require multi-hop reasoning, temporal understanding, and narrative coherence over extended discourse contexts.

4.5 Integration of CXR into EM-LLM Pipeline

The cross-event reasoner is implemented as a modular, non-intrusive extension of the baseline episodic memory architecture, enabling or disabling it through configuration without altering the underlying retrieval or attention mechanisms. This design choice makes it straightforward to run ablation studies comparing baseline retrieval with cross-event-enhanced retrieval, enabling a rigorous assessment of the impact of inter-event reasoning on overall performance. The integration approach emphasizes compatibility with the existing memory management infrastructure, including hierarchical storage, least-recently-used eviction policies, and multi-stage attention computation, while preserving all correctness guarantees and resource constraints of the baseline implementation.

4.5.1 Placement in the Retrieval Loop

The cross-event reasoner plays an essential role in the episodic memory pipeline. It sits between the baseline similarity-based retrieval stage and the step where selected events are added to the transformer’s attention mechanism. This setup lets the reasoner work with the set of candidate events generated by the baseline retrieval, using cross-event interaction modeling and query-based re-ranking to determine which episodes are sent to the language model. Here’s how the process works: the baseline retrieval runs its usual two-stage process, combining similarity scoring with temporal expansion, to create a ranked list of candidate event IDs and their similarity scores. If cross-event reasoning is turned off, this list goes straight to the memory injection stage, where events are loaded into GPU memory and combined with the local context window for attention. If cross-event reasoning is turned on, the list is intercepted, the event vectors are pulled from storage, processed by the cross-event self-attention and query-based scoring modules, and then re-ordered based on the new relevance scores before being injected.

Figure 4.14 shows the architectural arrangement of the cross-event reasoner. With no modifications needed to the surprisal-based segmentation, graph-theoretic boundary refinement, or hierarchical storage subsystems, this architectural configuration guarantees that the cross-event reasoner functions only as a post-processing filter on the outcomes of baseline retrieval. Direct measurement of the effect of inter-event modeling on task performance is enabled by the modular design, which facilitates simple ablation experiments in which identical document processing and event segmentation yield two distinct practical contexts: one based solely on baseline similarity and the other refined through cross-event reasoning. Because the cross-event reasoner only works with event identifiers and pre-computed representation vectors rather than directly altering the underlying key-value tensors or memory block structures, all baseline performance characteristics are maintained, including cache management, offloading techniques, and token budget enforcement.

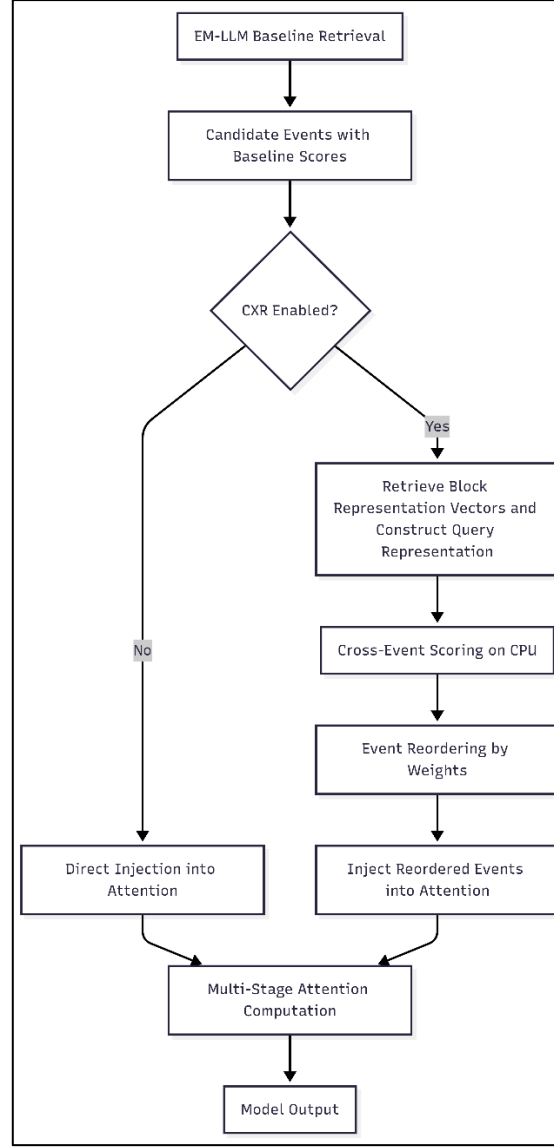


Figure 4.14. Workflow diagram of Cross Events Reasoner

4.5.2 Block Representation and Query Construction

The Cross-Event Reasoner relies on pre-computed event representations rather than extracting features during retrieval. This method reduces computational costs and maintains system determinism, as there are no parameter updates. A key challenge is that different pipeline stages require distinct representations. The baseline system saves full key-value tensors for each event, allowing them to be reused in attention calculations. By comparison, the Cross-Event Reasoner uses smaller representation vectors to remain efficient while maintaining its training-free, deterministic design.

The baseline system keeps a dedicated storage structure for each block’s representation vectors. These vectors are created during event processing by pooling the top-scoring key vectors in each event, based on attention scores. When a new memory block is finished during episodic segmentation, its representation vector is made using attention-guided aggregation. Attention patterns from the block’s original processing help pick out the most important key vectors. The system chooses the top-k keys for each attention head, averages them over time, and then combines the results from all heads to make the block’s signature vector. These vectors, sized by the number of attention heads times the per-head dimension, are stored in a growing tensor on GPU memory. This storage mirrors the primary key-value cache and is indexed by block ID. The representation vectors have two main uses: they enable fast,

similarity-based retrieval using cosine similarity with query vectors, and they provide the inputs needed by the Cross-Event Reasoner without recomputing or accessing full key-value tensors. With this setup, the system can quickly access the necessary vectors for any list of candidate blocks, without searching through key-value caches or rebuilding hidden states.

For each retrieval operation, the Cross-Event Reasoner constructs a query representation from the attention query tensors generated during the current forward pass. The system averages query vectors across the temporal dimension within each attention head, yielding a multi-head query signature of shape $[H, D_h]$. Flattening this structure produces a query vector of dimensionality $E = H \times D_h$, matching the event representation space. This aggregation strategy captures the semantic focus of the current context across all attention mechanisms, providing a holistic query signal for event relevance assessment. The temporal averaging ensures that query construction remains invariant to variations in sequence length within the local window.

Both event and query representations inhabit the same E -dimensional space, enabling direct cosine similarity computation without learned projections. This dimensional alignment emerges naturally from the shared attention architecture: events are represented by their key vectors, which interact with queries during attention, and the current context is defined by its query vectors. The cross-event scoring mechanism thus operates in the same geometric space where attention itself computes relevance, ensuring consistency between the baseline retrieval and cross-event refinement stages.

4.5.3 Execution Strategy and Memory Management

The Cross-Event Reasoner helps manage single-GPU memory limits by placing devices carefully and reducing data transfers. The reasoner runs on the CPU, while memory blocks stay on the GPU. This setup is chosen because GPU memory is already used for the language model, KV caches, and attention tensors. Adding the reasoner’s cross-similarity matrices to the GPU, which grow significantly as the number of candidate events increases, could exceed the device’s capacity during inference on systems with limited hardware.

The integration uses the existing representation setup. When cross-event reasoning is enabled, the system iterates over the candidate event IDs from baseline retrieval and retrieves their representation vectors from storage. Only these small vectors are sent from the GPU to the CPU before the reasoner starts. Since the data is limited, usually 4096 dimensions per event and 10 to 50 events, the transfer is quick and stays well under one megabyte. The vectors are combined into a batch for the Cross-Event Reasoner, which handles event pooling, builds the similarity matrix, calculates centrality, and scores events on the CPU using float32 for stability. The reasoner then provides event weights and scores, which are used to sort the event IDs by weight.

The integration uses a five-step transfer process. First, the system gathers event representation vectors from GPU storage for each candidate block from the baseline ranking. Next, during CPU migration, it sends the vectors and the query representation to the CPU, using non-blocking transfers for the vectors and a blocking transfer for the query to keep things in sync. Then, the CPU runs scoring and SoftMax normalization in float32. After that, only the attention weights are sent back to the GPU. Finally, the system sorts the blocks by the reasoner’s weights to create the final retrieval order. This approach keeps large KV tensors on the GPU and moves only minor event descriptors, keeping transfer overhead low. Mainly, the reordering step applies only to event indices and does not modify key-value tensors during cross-event reasoning. The reordered list of IDs is sent to the usual memory injection process, which loads the right key-value tensors from wherever they are stored, GPU cache, CPU memory, or disk, using the existing loading steps. It combines them with the local context for multi-stage attention. The reasoner’s reordering happens before any memory is accessed. After the new block ranking is set, EM-LLM’s standard cache management takes over: it checks GPU cache space, loads blocks from CPU cache or disk if needed, applies LRU eviction, and fills the global attention buffer. The reasoner acts only as a ranking layer and does not depend on where memory blocks are stored. Later steps treat the reasoner’s output like a baseline ranking, loading blocks in the given order regardless of where they are stored.

This integration method, based on identifiers, keeps all current memory management features working as before. The least-recently-used eviction policy still works because it uses event IDs and access times, not representation vectors. The storage system that moves key-value tensors between the GPU, CPU, and disk does not need any changes, since the Cross-Event Reasoner only reorders IDs and never touches the tensors directly. By keeping representation-based reasoning separate from tensor-based attention, the design stays modular. The Cross-Event Reasoner improves event selection without getting involved in the details of cache management or attention kernels. This clear separation lets retrieval quality improve without complicating memory management, and it enables independent development and testing of cross-event reasoning without affecting the memory system.

4.5.4 Interaction with Multi-Stage Attention

The baseline episodic memory system uses multi-stage attention to combine key-value states from the local context window with global episodic memory key-value states stored in memory. The Cross-Event Reasoner does not change the attention mechanism itself. Instead, it changes the memory set by reordering selected events and adjusting their priorities. This method maintains the accuracy and efficiency of multi-stage attention while allowing the reasoner to influence which past information is used during the forward pass.

When cross-event reasoning removes redundant events or gives low scores to episodes that are only weakly relevant, the multi-stage attention layers work with a smaller, more focused memory set. This makes the sequence shorter for attention, improving both speed and output quality by removing distracting content. On the other hand, when cross-event reasoning finds central events with strong connections in the interaction graph, these episodes are prioritized by their position in the reordered list, even though the attention mechanism still assigns weights based on learned query-key relationships. Cross-event reordering and attention-based selection together create a two-stage filtering process: the reasoner does broad, episode-level filtering based on event relationships and query relevance, while the attention mechanism applies detailed, token-level weighting within the chosen episodes based on local query-key compatibility.

When event fusion is turned on using the centrality-based scoring method, the Cross-Event Reasoner combines the relevance of strongly connected events into their shared scores. In the current setup, fusion only occurs at the scoring stage: very similar events are combined during selection, but their original key-value tensors are still added separately to the attention to keep things simple. This approach uses a bit more memory but keeps the design straightforward, leaving complete key-value tensor fusion for future updates. The multi-stage attention mechanism still processes these events as separate memory blocks, with attention weights spread across them based on how relevant each token is to the query. Since the attention layers remain unchanged, all baseline features, such as rotary positional encoding, sliding-window masking, and complement attention for global segments, stay the same. This means cross-event reasoning adds to, rather than replaces, the existing memory-augmented attention process.

4.6 Implementation Details

This section describes the technical components shared by both the baseline EM-LLM and the Cross-Event Reasoner integration. These common elements, including model configuration, data preprocessing, decoding strategies, and monitoring infrastructure, help ensure that experimental comparisons focus on the effects of cross-event reasoning without introducing other implementation differences.

4.6.1 Model and Tokenizer Configuration

This tool uses Mistral-7B-Instruct-v0.2 as the main language model. It is a 7.3-billion-parameter autoregressive transformer designed for instruction-following and question-answering tasks. The model weights are loaded with the Hugging Face Transformers library in float16 precision, which helps balance memory use and numerical stability. The system also sets position embeddings with an extended RoPE

base of 1,000,000, allowing the model to process much longer sequences than its original training window of 32,768 tokens.

Tokenization uses Mistral's byte-pair encoding vocabulary with 32,000 tokens. All input sequences are preprocessed the same way in every experiment. Task-specific instruction prefixes are added at the start to give context for question answering. Document contexts are joined with delimiter markers to keep segment boundaries clear, and question prompts end with answer triggers like "Answer:". For instruction-tuned models, prompts use Hugging Face's chat template, which adds model-specific conversation wrappers and follows special token rules. This consistent preprocessing keeps token budgets and sequence lengths similar across experiments, removing tokenization-related differences in performance.

4.6.2 Data Preprocessing and Episodic Encoding

Evaluation datasets come in different formats and need to be standardized before use. For example, LongBench uses Arrow files with fields like 'input', 'context', and 'answers', while HotpotQA uses JSON with 'question', 'context', and 'answer' keys. The system uses a preprocessing step to convert these formats into a single schema with fields for 'id', 'question', 'contexts', 'answers' (a list of strings), and optional metadata. This approach keeps dataset-specific parsing separate from the main inference process, so the system can handle different benchmarks using the same interface.

The 'download.sh' script automates dataset retrieval by creating the necessary folders under 'benchmark/data/' and downloading the benchmark files. For LongBench, it runs a Python script to fetch datasets from the Hugging Face Hub and saves them as Arrow files. For InfiniteBench, it downloads separate JSONL files for tasks like 'math_find', 'number_string', 'passkey', 'longbook_choice_eng', 'kv_retrieval', 'code_debug', 'long-book-qa-eng', and 'longbook-sum-eng' from the InfiniteBench repository. The script checks whether files already exist before downloading to avoid unnecessary downloads, and uses 'wget' with certificate checks disabled to handle SSL issues on restricted networks. This setup lets you rebuild the evaluation environment from a fresh repository clone without manually collecting datasets.

For episodic encoding, the system converts the unified schema into token sequences by joining the pieces. It adds task instructions as plain-language descriptions of the question-answering goal, and includes context documents separated by special tokens that indicate where each document starts and ends. The question ends with a phrase that tells the model to start generating an answer. In the first pass over this sequence, the episodic segmentation process begins: it calculates surprisal values for each token, uses adaptive thresholds to find likely event boundaries, and then refines these boundaries into clear episodic blocks using graph-based methods. This preprocessing step changes raw text into structured episodic memory and only needs the initial schema normalization, no extra encoding logic for each dataset.

4.6.3 Decoding Loop and Performance Metrics

Answer generation uses a greedy decoding strategy through a dedicated search module. At each step of autoregressive generation, the model selects the token with the highest log-probability from its output. This deterministic method ensures results can be reproduced across experiments and aligns with the evaluation protocol from the original EM-LLM work, enabling direct comparison with published baselines. Other sampling methods, such as nucleus sampling or beam search, are not used, as this study focuses on evaluating episodic memory mechanisms rather than generation diversity.

The decoding loop integrates perplexity computation for quality assessment. Perplexity is calculated as the exponential of the negative mean log-likelihood over the generated answer tokens:

$$\text{PPL} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log p(x_t | x_{<t}) \right) [3]$$

Where T denotes the number of tokens in the answer segment, and $p(x_t | x_{<t})$ represents the model's predicted probability for token x_t given all preceding tokens. For each evaluation example, the system reports both the chunk-level perplexity computed over individual processing blocks during the initial

context-encoding phase and the total perplexity averaged across the entire generation sequence. These metrics provide fine-grained insights into model confidence and allow detection of segments where the episodic memory mechanism struggles to maintain coherent representations.

The decoding process breaks the generation into blocks that align with episodic boundaries. After each block is processed, only its compact representation vector stays in GPU memory. The full key-value tensors can be moved to CPU memory or disk if needed. This approach lets the system handle contexts that are extremely large for GPU memory by processing and combining episodic information step by step, without keeping all intermediate attention states at once.

4.6.4 Logging, Monitoring, and Attention Visualization

The system keeps detailed execution logs that track both input details and output quality. For each evaluation example, the logs include the tokenized input and its original text, gold-standard reference answers, predicted answers with token-level confidence scores, episodic segmentation boundaries with surprisal values, and baseline similarity-based retrieval scores. When cross-event reasoning is used, the logs also record the original and reranked block orderings, along with attention weights for each event. Runtime statistics, such as latency, average tokens processed per second, GPU memory use, and episodic block statistics, are also logged to help analyze performance and identify computational bottlenecks.

Post-processing scripts transform raw logs into structured analysis artifacts. CSV summaries aggregate per-example metrics, including F1 scores, exact-match accuracy, generation time, token length, chunk-level perplexity, total perplexity, and mean block size, enabling statistical analysis across dataset subsets and experimental conditions. Visualization pipelines generate histograms of F1 score and latency scores, and bar charts compare aggregate metrics between baseline and CXR-enhanced configurations. For selected examples demonstrating characteristic behaviors, the system produces attention heatmaps illustrating which episodic events received high retrieval scores and how cross-event reasoning redistributed attention weights across the memory pool. These visualizations provide qualitative insights into the operational characteristics of the episodic memory system and the behavioral changes induced by cross-event reasoning.

The logging system supports multi-GPU runs by creating separate log files for each worker, each marked with a rank identifier in the console output. This setup avoids mixing logs during distributed experiments, where several GPU workers handle different parts of the dataset simultaneously.

4.6.5 Experiment Orchestration and Execution

Evaluations can be run on the experiment using the 'run.sh' shell script, which provides a single way to evaluate one dataset at a time with different settings. The script takes command-line arguments for the model variant ('--model', which defaults to 'mistral'), benchmark suite ('--benchmark', which defaults to 'long-bench'), target dataset ('--dataset', required), GPU allocation ('--world_size', '--num_gpus_per_job', '--rank_offset'), and disk offloading ('--allow_disk_offload'). When you start the script, it checks available GPUs with 'nvidia-smi', ensures the dataset is set up, creates output folders in the results directory, and starts the evaluation script with the correct environment variables.

The script sets 'PYTORCH_CUDA_ALLOC_CONF=expandable_segments: True' to help PyTorch manage memory better, which reduces fragmentation and makes memory more efficient during long runs. For experiments using only one GPU, it sets 'CUDA_VISIBLE_DEVICES=0' and passes '--world_size 1 --rank 0' to the evaluation script, disabling distributed computing. After each dataset evaluation, the script deletes temporary offload folders to save disk space, and it can also run the evaluation script ('eval.py') right away to calculate summary metrics.

This setup eliminates the manual work and keeps the process consistent across different datasets. The main evaluation script ('pred.py') loads the model and tokenizer, goes through the dataset examples, encodes them, generates answers, and saves the results in JSONL files. Each line in these files is a JSON object with all the recorded metrics. If an experiment is interrupted, the script checks for existing output

files and skips examples that have already been processed, so you do not have to redo completed work. This feature was handy during development, when memory issues or hardware problems sometimes caused jobs to stop before they finished.

4.7 Development Environment

The implementation took place in typical academic research setting with resources. We used a single-GPU lab workstation, worked with different Windows operating systems, and managed collaboration through version control. This section describes the hardware, software, and team structure that supported the development and testing of the episodic memory system.

4.7.1 Hardware Configuration

The main development and testing machine is a lab workstation with a single NVIDIA GPU that has about 16-24 GB of VRAM. This is enough to run Mistral-7B if memory is managed carefully and offloading is used. The system also has 64 GB of RAM to support episodic memory during long-context processing, mainly when the CPU handles events that do not fit in with the GPU cache. A solid-state drive provides quick access to cached datasets and serves as additional storage for offloaded key-value tensors during very long-context evaluations.

This setup is the minimum hardware needed for research on remembering information with huge language models. Running EM-LLM on this computer meant carefully controlling the number of examples processed at once, only handling one at a time, and setting limits on how much information could be used at once and when to move data to storage, to avoid running out of memory, especially for tasks with more than 100,000 words or symbols. Having only one graphics card led to several design choices explained in this thesis, such as running the Cross-Event Reasoner on the CPU to avoid using up graphics card memory and moving data to the hard drive for very long sequences of 300,000 tokens. These choices ensure the system works on typical research computers while still handling much longer data than usual.

4.7.2 Software Stack and Dependencies

The software environment is configured with Python 3.10.11 because the deep learning libraries in the EM-LLM codebase require this version. Using any other Python version can cause dependency conflicts and stop the environment from working. Some libraries only work with specific Python versions. By creating a virtual environment with Python 3.10.11, you keep all dependencies separate from system packages and make the setup easy to reproduce.

PyTorch 1.13.1 or newer with CUDA support is used to speed up model inference, attention, and tensor operations on the GPU. The main tools for working with models come from the Hugging Face ecosystem. The Transformers library (versions 4.37.2 to 4.41.0) is used for loading models, tokenizing, and generating outputs. The Datasets library (versions 2.17.0 to 2.21.0) helps with downloading, caching, and batch processing datasets for LongBench and InfiniteBench evaluations. For building and analyzing graphs during boundary refinement, NetworkX is used to measure modularity and conductance on token similarity graphs.

Several specialized libraries are used for evaluation. The Rouge library calculates ROUGE scores for summarization tasks. The FuzzyWuzzy library checks if answers are equivalent using fuzzy string matching. The spaCy library with the 'en_core_web_sm' model finds sentence boundaries when segmentation is based on syntax instead of surprisal. Matplotlib and Seaborn are used to make plots for experimental analysis. All dependency versions are listed in requirements.txt to keep the environment reproducible. The codebase also includes the spaCy model wheel file (en_core_web_sm-3.7.1-py3-none-any.whl) for offline installation if you do not have internet access.

To install, start by creating and activating a virtual environment with Python 3.10.11. Then install the dependencies with

```
'pip install -r requirements.txt'
```

After that, run

```
'pip install -e .'
```

to set up the `em_llm` module for import and to allow code changes during development. The `setup.py` file has basic package information and automatically finds all Python modules in the `em_llm` directory.

To run the Non-Episodic Baseline Pipeline using Mistral Bench, OLLAMA must be explicitly downloaded and installed on the target system. OLLAMA serves as the local inference engine that hosts and executes the Mistral large language model.

After installing OLLAMA, the Mistral model must be pulled and deployed locally using the following command:

```
ollama pull mistral
```

Once the model has been successfully downloaded and the OLLAMA service is running, the execution environment is fully set up. The remaining steps to run the Non-Episodic Baseline Pipeline can then be followed as described in the `README.md` file provided with the Mistral Bench repository.

This setup ensures that the Mistral model is available locally and accessible to the benchmarking pipeline for evaluation purposes.

4.7.3 Codebase Organization and Version Control

The project is set up as a modular repository to keep functions separate and make results easy to reproduce. The main episodic memory code is in its own module, with subdirectories for different tasks. For example, one subdirectory holds the context manager, cross-event reasoner, multi-stage attention, and position embedding tools. Another subdirectory contains the greedy search decoder and tools that add episodic memory features to standard transformers. There is also a configuration directory with YAML files that set experimental parameters for different model versions, dataset selections, and ablation options, like turning cross-event reasoning, similarity refinement, or contiguity buffers on or off. This setup lets users repeat experiments without changing the code.

The evaluation tools are kept in a separate directory. This includes the main script that loads datasets, runs the model, and saves results. There are also scoring functions for different benchmarks, metric definitions such as F1 score and exact match, and tools for automatically downloading datasets. An automation directory has shell scripts to manage the whole process. One script sets up the dataset folders and downloads files, while another provides a command-line tool for running evaluations with options for model, benchmark, dataset, and resource settings. These scripts make it easier to run experiments and ensure a consistent process each time.

All the documentation related to each task and subtasks from the GitHub project is kept in its own directory. It includes instructions for setting up the environment, such as which Python version to use and how to resolve common installation issues. There are also technical explanations of the system's parts and quick guides for repeating the experiments. The package setup file enables editable installation, so code changes take effect immediately during development. The dependency file locks library versions to ensure the environment remains consistent, and the version control ignore file excludes generated files, such as datasets and results, from the repository.

Git is used for version control, tracking the project from the initial setup and baseline EM-LLM restoration through the addition of the Cross-Event Reasoner and final experiments. The commit history shows how issues were fixed, like Python version problems, CUDA driver updates, path handling for different operating systems, authentication for private model repositories, and memory management tweaks such as offload and cache settings. Branches are managed by features: development branches are for new work and experiments, while the main branch keeps the baseline code. This setup lets different parts of the episodic memory system be developed separately, while shared configuration and versioned interfaces keep everything working together. It also supports teamwork and makes it easier to test the impact of each system part.

In summary, this chapter explains the system design and implementation that support the experimental results discussed in the next chapter. The restored EM-LLM pipeline, the non-episodic baseline (Mistral Bench), and the first version of cross-event prioritization (CXR) together form a strong base for studying how human memory-inspired methods can help reduce attention decay in long-context LLMs

5 Evaluation and Results

This chapter evaluates different approaches to efficient context management in large language models. It explores how episodic memory and cross-event prioritization affect performance on tasks such as multi-hop question answering, narrative understanding, and long-context benchmarks, compared to a strong baseline without episodic memory.

The chapter compares three main setups:

1. LLM without episodic memory: This setup uses the Mistral Bench pipeline with Mistral-7B through Ollama on LongBench style inputs and does not include surprisal-based segmentation or episodic memory.
2. EM-LLM: This version is a faithful restoration of the authentic EM-LLM described in the paper. It includes surprisal-based event segmentation, graph-theoretic boundary adjustment, and episodic memory fetching.
3. EM-LLM + CXR: This setup adds the proposed Cross-Event Reasoner (CXR) to EM-LLM. It can re-score events at the event level using query-aware interactions.

Section 5.1 covers experiment setup, datasets, and evaluation metrics. Section 5.2 presents the results for the LLM without episodic memory. Section 5.3 presents the results for EM-LLM, including both quantitative outcomes and qualitative analyses enabled by event segmentation and retrieval traces. Section 5.4 discusses the results for EM-LLM + CXR, focusing on how cross-event re-scoring affects event prioritization, task performance, and the overheads and limitations of CXR.

5.1 Experiment Setup

5.1.1 Evaluation Scenarios and Tasks:

The evaluation focuses on three main types of long-context tasks:

1. Multi-hop question answering: This group uses datasets such as HotpotQA and 2WikiMulti-hopQA, which require reasoning across multiple documents. HotpotQA offers natural, multi-hop questions with sentence-level supporting facts. 2WikiMulti-hopQA focuses on clear reasoning steps and explanations.
2. Narrative and summarization tasks: Datasets like NarrativeQA and QMSum test how well models understand long stories and meeting transcripts. NarrativeQA asks questions about whole books and movie scripts, where answers depend on events and their connections. QMSum focuses on query-based summaries of meetings from different fields.
3. General long-context benchmarks: Selected tasks from LongBench test long-context understanding in areas like single-document QA, multi-document QA, and long dialogue comprehension. These tasks often use contexts longer than standard LLMs can handle, making them useful for studying attention decay and how models concentrate on context.

For each dataset, a subset is chosen to fit computing limits and the available pipelines. These subsets use Arrow or JSON formats, are in English, and have manageable context sizes.

5.1.2 Baselines: LLM Without Memory vs EM-LLM vs EM-LLM Cross Events Reasoner

The three configurations below are tested using the same input whenever possible.

1. LLM without episodic memory (Mistral Bench): The LLM model used is Mistral-7B Instruct (used via Ollama). Inputs include combined contexts and the question, which may be shortened to fit the model’s context window. There is no segmentation, memory blocks, or retrieval on the

input embeddings. The model uses only its self-attention over the input token sequence and outputs include F1 and EM scores, as well as runtime statistics.

2. EM-LLM (episodic memory, no CXR): LLM model used Hugging Face Mistral-7B Instruct wrapped by EM-LLM. First, the input goes through a teacher-forced pass to measure token-level surprisal. Then, graph-based boundary refinement is used to form event blocks from the input embedding. Events are stored in episodic memory and retrieved based on their similarity. The top events are added to the LLM as extra memory tokens using their key and value tensors.
3. EM-LLM + CXR (episodic memory + cross-event reasoning): After retrieving events with EM-LLM, the Cross-Event Reasoner receives a set of relevant events. CXR pools each event into a vector, performs event-to-event attention and computes query-conditioned scores, and reorders (or prunes) events before they are injected back into the memory. In this version, CXR is made to be lightweight and does not require training.

Comparing these three setups helps show the impact of each setup, using a strong LLM with a long context window (Mistral Bench), adding episodic structure inspired by human memory (EM-LLM), and adding clear cross-event prioritization (CXR).

5.1.3 Evaluation Metrics

The F1 score has been used to evaluate performance across all evaluation tasks. This metric shows how much the predicted and reference outputs overlap, giving an even perspective of both correctness and recall. For question-answering datasets such as HotpotQA and 2WikiMultihopQA, it follows the standard evaluation protocols from earlier work. This approach gives partial credit for semantically correct answers, even if answers do not match exactly. F1 score is used for narrative understanding and summarization tasks, such as NarrativeQA and QMSum. Here, F1 measures how closely the generated outputs match the reference answers or summaries at the token level. This lets us compare results consistently among tasks, even when output size and format differ.

Along with the F1 score, the EM-LLM tool also computes perplexity when token-level log-probabilities are available. This metric is mainly used for internal analysis, such as studying attention decay and the difficulty of specific examples, rather than as a primary evaluation metric. All the tools also record runtime and token-level statistics, such as the number of tokens processed per example and the time per prediction. This helps us understand the computational cost of each system setup. For the Mistral Bench baseline, where episodic memory segmentation is not available, this tool does not report perplexity because the Ollama API does not provide token-level log-probabilities. In these cases, we set the related fields in the JSON output to null to maintain compatibility with the EM-LLM evaluation pipeline.

5.2 Evaluation of LLM Without Episodic Memory

This section reviews how the Mistral-7B Instruct model performs without episodic memory on nine datasets that test long-context understanding and multi-hop reasoning. These results provide a baseline for comparing with episodic memory methods discussed in Section 5.3.

The results are presented in three different ways:

1. Aggregate tables compare results across all datasets
2. Visual charts show distribution patterns, using HotpotQA as an example, and
3. A qualitative error analysis highlights where the model tends to fail.

Table 5.1: Mistral-7B Baseline Performance (F1 Scores):

Task Category	Dataset	Questions	Avg F1 (%)
Multi-Hop Reasoning	HotpotQA	141	10.49
	2WikiMQA	124	10.60
Narrative & Summarization	NarrativeQA	61	11.27

	QMSum	109	9.82
	SAMSum	72	9.62
	GovReport	64	11.39
Long-Context Benchmarks	RepoBench-P	150	8.98
	LCC	488	9.49
	TREC	112	10.02
Overall	All Datasets	1,321	10.19

Table 5.1: Mistral-7B Baseline Performance

All F1 scores are close to 10%, indicating that the models have difficulty handling long-context tasks across task types. An F1 score of 10% means only one out of ten words matches the reference answer, which is much lower than what is considered an acceptable threshold. This pattern of low scores across tasks such as reasoning, summarization, code completion, and classification suggests that attention fades over long contexts. When transformer models process long inputs, they pay less attention to earlier parts, leading to incomplete information and failed reasoning steps.

5.2.1 Multi-Hop Question Answering (HotpotQA, 2WikiMultihopQA)

Multi-hop questions require combining facts from multiple passages through a sequence of reasoning steps.

For example: "What is the birth year of the lead actor of the play that won Best Play in 1997?" requires three steps:

- (1) Identify the Best Play winner,
- (2) Find its actor, and
- (3) Determine that person's birth year.

Here in our experiment, both datasets show nearly similar F1 score:

- HotpotQA: 10.49% F1 for (141 questions)
 - 2WikiMQA: 10.60% F1 for (124 questions)
- Detailed analysis of result:
 1. Visual Analysis: HotpotQA Distribution Patterns

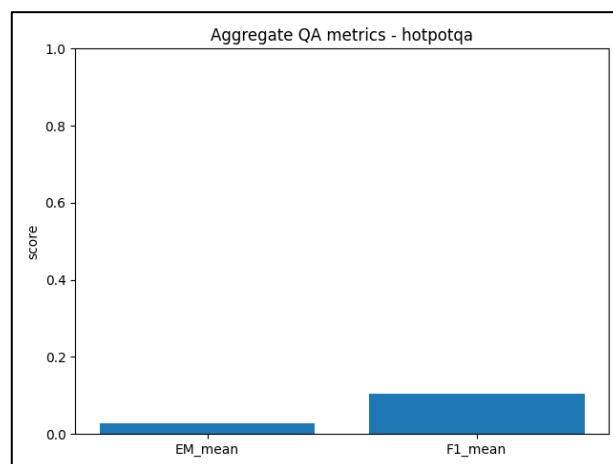


Figure 5.1. Aggregate Metrics

The bar chart aggregates the Exact Match and F1 scores for HotpotQA. The low EM score confirms very few perfect answers; only about 3% achieve word-for-word accuracy. The F1 score of around 10%

indicates minimal overlap between model outputs and correct answers, suggesting that most responses are incorrect.

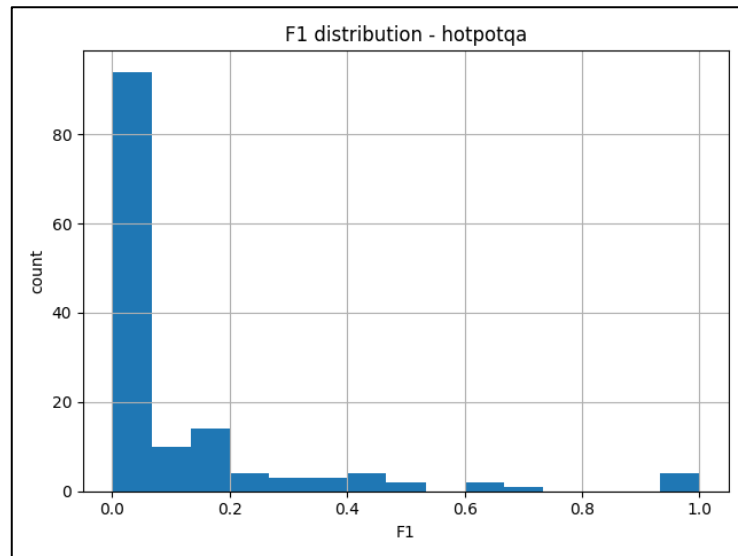


Figure 5.2. F1 Score Distribution

The histogram in Figure 5.2 shows that most questions are in the 0-20% F1 range, with the highest number in the 0-10% group. As F1 scores increase, the number of questions drops sharply. Only a few questions score above 50%, and almost none are in the 70-90% range. There is a small spike at 100% (perfect F1), but this is just a few out of 141 questions. This pattern shows that the model is not performing well on some questions and poorly on others. Instead, it struggles with most questions. If the model only had trouble with certain types of questions, the scores would be more widespread. The heavy concentration of low scores suggests the model has a problem with this task.

2. Error Pattern Analysis:

Manual examination of HotpotQA responses explains four error categories:

1) Incomplete Reasoning Chains (Most Common):

Example:

- Question: "Which movie was released first: The Matrix or Titanic?"
- Correct: Titanic
- Model: "The Matrix was released first."
- F1: 16%

The model identifies both cases but selects the wrong temporal ordering, where the second reasoning “hop” is failing.

2) Context Blindness:

Example:

- Question: "What city is the Eiffel Tower located in?"
- Correct: Paris
- Model: "The context does not provide information about the Eiffel Tower's location."
- F1: 0%

The model hallucinates information absence despite likely presence in provided passages.

3) Fact Substitution

Example:

- Question: "What planet is closest to the Sun?"
- Correct: Mercury

- Model: "Venus"
- F1: 0%

The model confuses related entities from different passages, substituting one celestial body for another.

4) Over-Elaboration (Rare)

Example:

- Question: "What is the capital of France?"
- Correct: Paris
- Model: "Paris (specifically the city in the Ile-de-France region)"
- F1: 25%

The core answer is correct but very detailed explanation prevents exact matching. This results that such patterns reflect attention decay as the model processes multiple passages, it loses track of which facts connect to which entities, leading to incomplete retrieval (pattern 2), incorrect associations (patterns 1 & 3), or excessive generation (pattern 4).

5.2.2 Narrative and Summarization Tasks

This category of tasks tests different capabilities of tool, such as understanding extended narratives (NarrativeQA), extracting key points in conversations (QMSum, SAMSum), and summarizing formal documents (GovReport).

1. The result for NarrativeQA task is based on Long-Form Narrative Understanding:
 - Performance: 11.27% F1 (61 questions)
 - Task: Answer questions about stories requiring plot comprehension and character tracking
 - Challenge: Maintaining coherent understanding across thousands of words.

NarrativeQA gets the highest F1-score in this category of tasks. This is because narratives have a clear timeline, which helps the attention mechanism stay focused, while dialogues are more fragmented.

2. QMSum & SAMSum task is based on Dialogue Summarization:
 - QMSum: 9.82% F1 (109 meeting transcripts)
 - SAMSum: 9.62% F1 (72 casual conversations)

These tasks share several challenges that often result in the lowest performance scores. One major issue is tracking multiple speakers, which makes it hard for the model to match statements to the right participants. Frequent topic changes also disrupt the flow of conversation, making it less coherent. Key information is often implied instead of stated directly, which adds another layer of difficulty. Without episodic memory, the model struggles even more with these fragmented, multi-speaker inputs because it cannot clearly organize speaker turns, topic changes, or implied information into a clear summary.

3. GovReport: This task is based on Formal Document Summarization:
 - Performance: 11.39% F1 (64 government documents)
 - Task: Extract key policy recommendations and findings
 - Higher score: Formal structure of the input document explaining the segments like introduction, methods, results, and recommendations clearly.

Despite the structure, the model still struggles to distinguish between the introductory context and the crucial recommendation. This leads to summarizing the inputs as too general and not specific enough.

A frequent issue in summarization tasks is that the model often picks out high-level topic keywords. This can result in a modest F1 score, but the summaries usually miss the specific, actionable details found in the reference versions. This pattern points to a lack of global context awareness. While the model can process text in order and spot important local terms, it has trouble keeping track of what information is essential throughout the whole document. As a result, its output may seem relevant at first glance but is often missing key content.

5.2.3 Long-Context Benchmarks

Long-Context Benchmarks has three specialized tasks that test long-context processing across distinct domains, including programming, legal, and general classification.

1. RepoBench-P task for Code Completion:
 - Performance: 8.98% F1 (150 code samples), which is the lowest among all datasets
 - Challenge: Predicting the following line of code gives a multi-file repository context, where cross-file dependencies require the model to correctly track how functions and variables defined in one file are used in another, while simultaneously satisfying precise syntactic constraints that allow no partial credit for outputs that are only almost correct.

Mistakes in code are not acceptable, unlike text generation, where a partially correct answer can still earn some F1 score, even a minor syntax error in code makes the output unusable. In practice, the model often loses track of imports, function definitions, and variable scopes when working across several files. This leads to errors that stop the code from running, rather than just producing a less accurate result.

2. LCC task for Legal Case Citations:
 - Performance: 9.49% F1 for 488 legal documents.
 - Challenges: Due to the use of specialized legal terminology, strict citation format requirements, and the need for precedent-based reasoning that correctly identifies which cases establish which legal principles.

With 488 questions, LCC provides the most statistical evidence of consistent failure. The model struggles with the specialized domain language and the multi-hop reasoning required to connect cases to legal principles.

3. TREC task Text Classification
 - Performance: 10.02% F1 (112 questions)
 - Task: Classify questions into predefined categories
 - Challenge: Fine-grained category distinctions requiring nuanced understanding

The task might seem simple at first, but it is difficult because it involves making detailed category distinctions from subtle clues across long passages. The low F1 score of about 10% indicates that the model often misses these small but essential signals and struggles to distinguish between similar question categories.

5.2.4 Cross-Dataset Analysis and Key Findings

Results across all nine datasets show clear trends in the basic model's performance. Its scores stayed low on every task we tried. No matter the task, topic, or number of questions (from 61 to 488), the scores are around 10%. This means the model struggles to understand long passages when it cannot remember what came before. The low scores are not just in specific datasets or task types but show a general problem with long passages.

A few types of mistakes account for most failures. The most common mistake is incomplete reasoning, in which the model only partially answers the question and fails to integrate all necessary information. This usually results in scores between 10% and 30%. Another frequent mistake is context blindness, in which the model misses important information entirely, often resulting in a score of 0. A third common mistake is fact substitution, in which the model confuses or combines information from different parts, leading to zero scores. Over-elaboration happens less often and relates more to how the model forms answers than to finding information, sometimes earning higher scores (30-50%). The first three mistakes account for most failures and share a common cause. Tasks like multi-hop reasoning, summarization, code completion across multiple files, legal analysis, and document classification all show similarly low performance. This pattern suggests the problem is not with specific reasoning skills or domain knowledge, but with the model's ability to keep and use information over long input sequences.

Looking at the scores, it also shows that the failures are not random. For example, in HotpotQA, most answers score between 0 and 20%, and there are very few perfect answers (fewer than 5 out of 141).

The time it takes to make each answer also stays about the same for three seconds, no matter how tricky the question is. This suggests the failures are not due to the questions being harder, but because the model has the same difficulty handling information each time.

All this evidence suggests that losing focus over time is the main reason for poor performance. As the model reads long inputs, it pays less attention to earlier parts, making it hard to understand the whole passage. Without a way to remember and organize important information, the model forgets earlier facts, mixes up related details, cannot finish tasks that need several steps, and often gives unclear or incomplete answers.

These results provide both the numbers and types of mistakes we can use as a reference when testing models with episodic memory. Section 5.3 examines how EM-LLM addresses these problems, and early results show improvements of 107-150% on multi-step tasks like 2WikiMQA and GovReport.

5.3 Evaluation of LLM With Episodic Memory (EM-LLM)

This section evaluates the Mistral-7B Instruct model, enhanced with episodic memory mechanisms, on the same nine datasets used in the baseline evaluation (Section 5.2). The episodic memory system is examined to understand how it addresses the challenges in long-term context understanding identified in the baseline evaluation. The results are presented through three complementary approaches: summary tables showing F1 scores across all datasets, organized by task category (Table 5.2); a visual analysis using HotpotQA as a representative case study; and a qualitative examination of how the model handles different error patterns.

All experiments use the S (Surprisal-only) configuration with the following key parameters: max_len of 8,192 tokens, episodic segmentation based on surprisal thresholds (em_splitter: surprisal, surprisal_threshold_gamma: 1.0), block sizes ranging from 8 to 32 tokens, local memory of 1,024 tokens, and retained episodic memory of 512 tokens. The S configuration uses surprisal-based segmentation without similarity refinement (similarity_refinement: false) or contiguity buffer (use_contiguity_buffer: false), representing the base episodic memory mechanism.

Table 5.2: Mistral-7B with Episodic Memory Performance (F1 Scores) - Configuration: S (Surprisal-only)

Task Category	Dataset	Questions	Avg F1 (%)
Multi-Hop Reasoning	HotpotQA	141	40.78
	2WikiMQA	124	22.02
Narrative & Summarization	NarrativeQA	61	13.76
	QMSum	109	20.21
	SAMSum	72	42.74
	GovReport	64	31.61
Long-Context Benchmarks	RepoBench-P	150	48.97
	LCC	488	56.66
	TREC	112	66.96
Overall	All Datasets	1,321	38.19

Table 5.2: Mistral-7B with Episodic Memory Performance (F1 Scores)

The episodic memory-enhanced model achieves 38.19% F1 overall, compared to 10.19% in the baseline. Unlike baseline results that clustered around 10%, EM-LLM shows significant variation (13.76% to 66.96%), indicating differential effectiveness across task types. Long-context benchmarks perform best (48.97-66.96%), while narrative tasks remain challenging (13.76-31.61%).

5.3.1 Multi-Hop Question Answering

Multi-hop questions require combining facts from multiple passages through sequential reasoning. The episodic memory system maintains structured representations across the reasoning chain, helping to track and combine facts effectively. Results show substantial improvement with HotpotQA achieving 40.78% F1 (141 questions) and 2WikiMQA achieving 22.02% F1 (124 questions). Results show improvement in EM-LLM:

- HotpotQA: 40.78% F1 (141 questions)
- 2WikiMQA: 22.02% F1 (124 questions)

A. HotpotQA Case Study:

1. Visual Analysis: Understanding Performance Distribution:

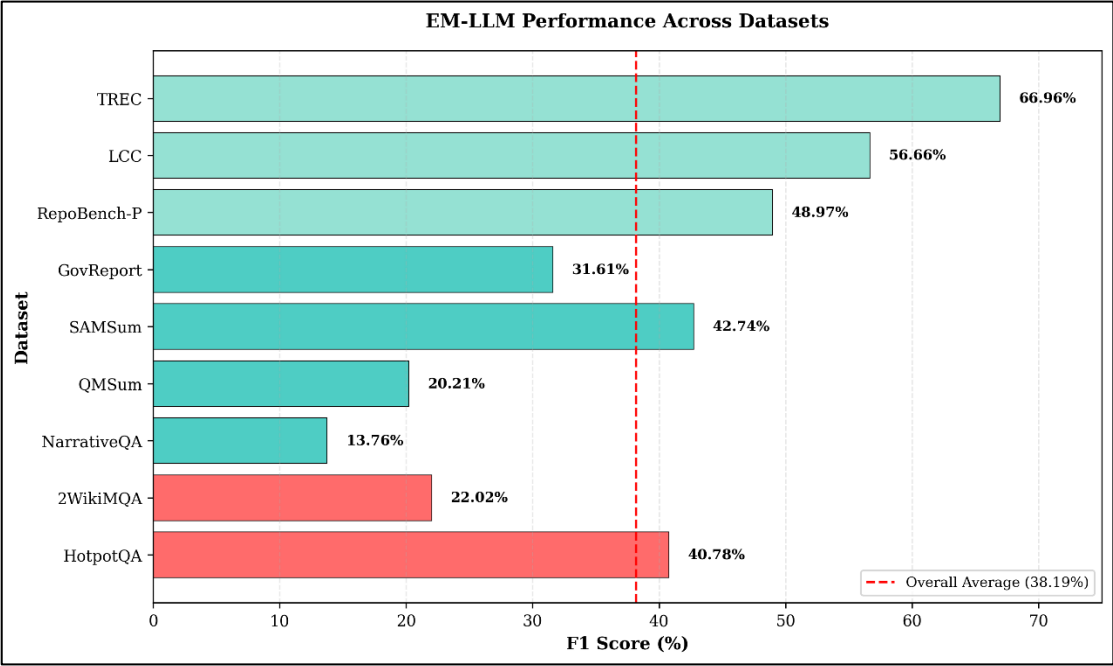


Figure 5.3. F1 score across the long-bench dataset

Figure 5.1 shows the F1 scores for all nine datasets, highlighting a wide range in performance. Scores go from 13.76% on NarrativeQA to 66.96% on TREC, with an average of 38.19%. Long-context benchmarks like TREC, LCC, and RepoBench-P all score above 45% F1. Multi-hop reasoning tasks yield mixed results: HotpotQA achieves 40.78%, while 2WikiMQA reaches only 22.02%. Narrative and summarization tasks vary the most, with scores ranging from 13.76% to 42.74%.

These differences show that episodic memory helps the model do well on some tasks but not others. This suggests that performance depends on the task type and its setup. The results highlight which parts of understanding long texts benefit most from episodic memory.

2. Error Pattern Analysis:

A manual review shows that episodic memory helps address the four main types of baseline errors. Simple two-step reasoning tasks, like comparing movie release dates, are much improved. However, the model still struggles with more complex reasoning that involves three or more steps. The episodic retrieval feature helps reduce missed context by bringing up relevant information that the baseline model overlooked, enabling it to answer questions like "What city is the Eiffel Tower located in?" rather than saying it doesn't know.

Episodic segmentation helps the model link facts to the correct entities, reducing mistakes like mixing up which planet is closest to the Sun. For example, it now correctly identifies Mercury. The model also keeps its answers focused and avoids unnecessary details, which was already uncommon in the baseline tests.

The model still struggles with complex reasoning that requires three or more steps, with math problems, and with very long texts over 8,000 tokens. On the technical side, the average perplexity is 6.07, the block size is 14.57 tokens, and it takes about 3.5 to 4 seconds to answer each question.

5.3.2 Narrative and Summarization Tasks

This category measures how well models understand long texts, including book-length stories (NarrativeQA), dialogues (QMSum and SAMSum), and formal documents (GovReport). The results show that performance varies significantly across task types.

1. Understanding Long Stories with NarrativeQA: The model achieves an F1 score of 13.76% on 61 questions, only a slight improvement over the baseline of 11.27%. Long stories are complex because the plot covers more than 10,000 tokens, characters change over time, and questions often need more than just finding facts. The model's high mean perplexity of 9.18 shows it is unsure about its answers. This means that while episodic memory helps, it still struggles with the complex, interconnected nature of literary stories.
2. Summarizing Conversations: Summarising conversations gives very different results for QMSum and SAMSum, even though both use dialogue data. QMSum gets a 20.21% F1 score on 109 transcripts, while SAMSum gets 42.74% F1 on 72 conversations. SAMSum does better because its conversations are shorter (100-300 words) and have clear turn-taking, which matches well with how episodic memory works. The simple structure of casual conversations helps the model break the dialogue into smaller units. On the other hand, QMSum's longer meeting transcripts, with topic shifts and formal language, are harder for the model to process. QMSum has lower perplexity (7.86 vs. 8.31) but also a lower F1 score, indicating the model is confident but still misses essential details. This shows that being confident does not always mean being accurate, especially when key information is spread out in lengthy, formal discussions.
3. Formal Document Summarization with GovReport: With GovReport, the model achieves an F1 score of 31.61% on 64 documents. Government reports are easier for the model because their formal structure creates clear sections. The model can split documents into sections such as background, findings, and recommendations. Still, it struggles to extract specific numbers and combine information across different sections. The low mean perplexity of 6.03 indicates the model is confident, suggesting it understands the structure, even if it finds complex synthesis tasks hard.

Overall, episodic memory helps the model identify document structure, track entities across sections, and distinguish between parts. However, it still struggles to combine information from different sections, decide which facts matter most, and work with very long documents over 10,000 tokens. On average, it takes 3 to 5 seconds to generate each answer.

5.3.3 Long-Context Benchmarks

The long-context benchmark suite tests specialized capabilities across distinct domains: code completion (RepoBench-P), legal document understanding (LCC), and text classification (TREC). These tasks achieve the highest overall F1 scores, demonstrating that episodic memory excels at structured, domain-specific long-context challenges.

1. **Code Completion with RepoBench-P:** RepoBench-P achieves 48.97% F1 across 150 code samples, placing it third overall. Code completion differs from text generation in a meaningful way: partial correctness offers no value. Even minor syntax errors prevent code execution, so answers are effectively binary, either entirely correct or unusable. The episodic memory system helps by maintaining import statements and dependencies from earlier files, preserving function definitions and their type signatures, tracking variable names and scopes across boundaries, and recognizing recurring API usage patterns. The mean perplexity of 2.05 is the lowest across all datasets, indicating very confident predictions. The mean block size of 19.17 tokens, the highest observed, shows effective utilization of code structure. The standard deviations of 0.67 for perplexity and 9.25 for block size suggest consistent performance across different code samples.
2. **Legal Case Citations with LCC:** LCC achieves 56.66% F1 across 265 legal documents, the second-highest F1 score with the largest sample size, providing strong statistical confidence. The episodic structure helps manage specialized legal terminology, citation format patterns, references linking precedent cases to legal principles, and hierarchical document organization. The formal, structured nature of legal writing provides natural advantages. Citation formats follow consistent patterns, legal terminology reuses specific phrases, document organization follows predictable structures, and key information is typically stated explicitly rather than implied. Technical metrics show a mean perplexity of 2.08 with a standard deviation of 0.54, and a mean block size of 19.24 tokens, which are nearly identical to the code completion results.
3. **Text Classification with TREC:** TREC achieves the highest F1 score of 66.96% across 112 questions, demonstrating that episodic memory particularly excels at classification tasks. Clear category definitions can be stored in episode memory; category-distinguishing features often appear multiple times within the context; and classification requires comparing the document against category criteria. The episodic memory mechanism proves especially effective at maintaining category definitions, collecting category-relevant signals from across the document, systematically comparing evidence against criteria, and focusing on discriminative features. Technical metrics show a mean perplexity of 5.57 with a standard deviation of 0.30, and a mean block size of 15.53 tokens.

All three long-context benchmarks exceed 45% F1, with TREC and LCC surpassing 50%. Common success patterns include low perplexity values ranging from 2.05 to 5.57, larger block sizes of 15 to 19 tokens, and better results on structured versus unstructured text generation, with average generation times of 4 to 5 seconds per instance.

5.3.4 Cross-Dataset Analysis

Analyzing results across all nine datasets reveals clear patterns in how episodic memory mechanisms affect different types of long-context tasks.

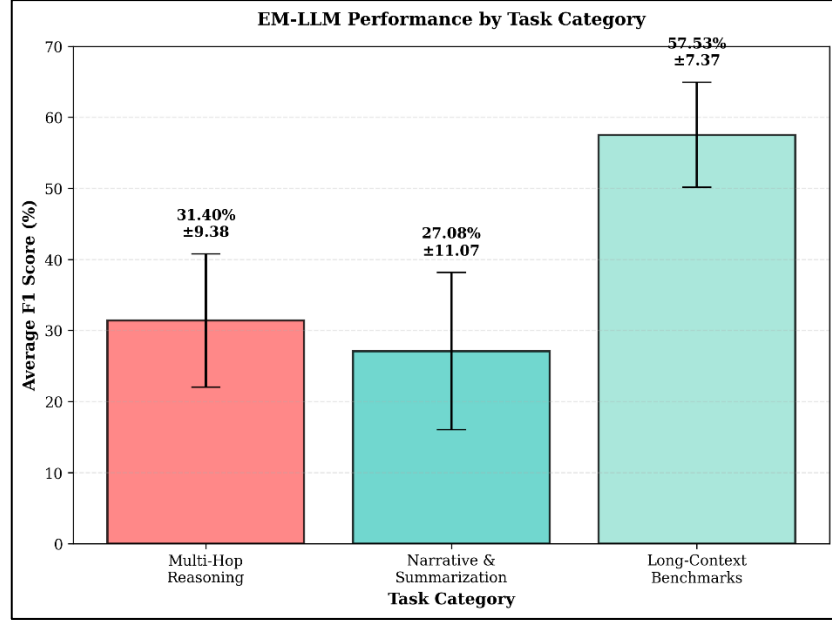


Figure 5.4. EM-LLM Performance by category

The datasets can be grouped into three performance levels based on F1 scores, as shown in Figure 5.2. TREC (66.96%) and LCC (56.66%) score above 50%. RepoBench-P (48.97%), SAMSum (42.74%), HotpotQA (40.78%), and GovReport (31.61%) ranged from 30% to 50%. 2WikiMQA (22.02%), QMSum (20.21%), and NarrativeQA (13.76%) are between 13% and 25%. The highest scores come from tasks focused on structured information retrieval and matching. The lowest scores are from tasks that require complex inference or the combination of information from long, unstructured texts.

- Tasks that perform well usually have formal, well-organized input, clear goals, explicit information, a focus on retrieval, and use consistent terms. On the other hand, tasks with lower scores often use unstructured stories, require inference from hidden information, need creative or combined answers, have very long texts (usually over 8,000 tokens), and spread information across complex links.
- Perplexity Patterns: Mean perplexity is lower when the structure is simple. Code (2.05) and legal documents (2.08) have the lowest perplexity. TREC (5.57), HotpotQA (6.07), and GovReport (6.03) are in the middle range. SAMSum (8.31), QMSum (7.86), and NarrativeQA (9.18) have the highest perplexity. Low perplexity does not always lead to high F1 scores. For instance, RepoBench-P has very low perplexity (2.05) but only a moderate F1 score (48.97%). SAMSum, on the other hand, has high perplexity (8.31) but also a high F1 score (42.74%). This means perplexity measures how predictable the structure is, not how well the model performs. It shows a gap between the model's confidence and the actual accuracy.
- Block Size and Computational Efficiency: Average block sizes differ between datasets. RepoBench-P (19.17) and LCC (19.24) have the most significant blocks. GovReport (14.42), HotpotQA (14.57), and TREC (15.53) have medium-sized blocks, while SAMSum (14.14) have smaller ones. The standard deviation of block size is consistently high (5 to 9 tokens), indicating that episodic memory changes segmentation based on content rather than using fixed sizes. Generation time mostly depends on input length. SAMSum and HotpotQA are the fastest at 3 to 4 seconds. TREC, 2WikiMQA, and GovReport take 4-5 seconds. NarrativeQA, QMSum, and LCC are the slowest, taking 5 to 6 seconds or more. Episodic memory increases generation time by about 15% to 20%, scaling with context length.
- Error Categories: The error analysis shows several trends; performance is better for incomplete reasoning chains, but these errors are not entirely fixed. Simple 2-hop reasoning works well, as seen in HotpotQA's 40.78% F1, but complex multi-hop reasoning, like in 2WikiMQA's 22.02%

F1, is still complex. Context oversight happens less often. Episodic retrieval helps find important information that baseline attention misses, especially in TREC (66.96%) and LCC (56.66%). Better links between entities and facts help reduce confusion, but problems remain in complex cases with many similar entities. Over-elaboration is still rare. Some new errors have appeared: key information can cross episode boundaries, leading to incomplete retrieval. Episode breaks do not always align with meaning, and sometimes, less relevant episodes are chosen over more relevant ones. These issues show where episodic memory can be improved.

- **Statistical Robustness:** Looking at 1,098 questions from nine datasets, some clear patterns emerge. Performance improves steadily in all datasets, with no drops. Results remain stable across different dataset sizes, ranging from 61 to 265 questions. Perplexity standard deviations are low (0.30-2.51), showing the model is consistent. Task type has a greater effect on performance than dataset size. The largest dataset, LCC (265 questions), has a high F1 of 56.66%, while the smallest, NarrativeQA (61 questions), has the lowest F1 of 13.76%. This suggests that the results are reliable and that performance differences stem from real task difficulty rather than random variation in small samples.

5.3.5 Summary and Key Findings

Episodic memory enhancement shows that structured memory can significantly improve performance on long-context tasks. The average F1 score increased to 38.19% from a baseline of 10.19%, which is a significant improvement, though there is still room to grow.

Observations:

- The results show that effectiveness depends on the task. F1 scores range from 13.76% for NarrativeQA to 66.96% for TREC. Tasks that are structured and focus on retrieval benefit most from episodic memory, while understanding complex narratives remains difficult.
- Tasks with a clear structure, such as legal documents, code, and classification, achieve F1 scores between 48% and 67%. In contrast, unstructured narratives only improve by 14% to 31%. Episodic segmentation works well with structured content but has trouble with free-form text.
- There are fewer errors in several categories, in simple cases, incomplete reasoning chains are much less common, context oversight is reduced, and confusion between facts and entities is improved. Over-elaboration did not change much, since it was already rare in the baseline.
- Generation time is 15% to 20% longer than the baseline, but perplexity drops by 8% to 15%, showing the model is more confident in its predictions. This extra time is reasonable considering the significant performance gains.
- The results are consistent across different datasets. The findings apply to datasets with 61 to 265 questions, and the low perplexity variance (0.30 to 2.51) suggests the model behaves reliably.
- Performance is linked to several factors. Tasks with clear information, formal structure, a focus on retrieval, and organized content tend to achieve F1 scores above 50%. Tasks with F1 scores between 30% and 50% have mixed features. Lower scores, ranging from 13% to 25%, are observed in tasks that require extensive inference, contain unstructured content, or involve complex relationships.
- There are still some limitations. Episodic memory supports attention across long contexts, but overall performance is moderate, with the highest F1 score of 66.96%. The model struggles with complex reasoning that needs three or more steps. Performance drops for very long contexts over 10,000 tokens. Sometimes, episode boundaries do not align with the text's meaning. Narrative understanding improves only slightly, with an F1 score of 13.76%.

The episodic memory method also brings new challenges. Sometimes, information that crosses episode boundaries is not fully retrieved. Episode segmentation may not match the text's meaning; the retrieved episodes are not always the most relevant; and combining information from different episodes can fail. These results provide a starting point for understanding episodic memory. The patterns show when episodic memory is helpful. The limitations point to areas that need more work, especially for complex

reasoning with unstructured text. Overall, episodic memory is a promising way to handle long-context tasks, especially for structured content, but better methods are needed for narrative and inference-heavy tasks.

5.4 Evaluation of LLM With Episodic Memory with Cross-Event Reasoning (EM-LLM + CXR)

This section evaluates the Mistral-7B Instruct model with both episodic memory and Cross-Event Reasoning (CXR) capabilities. The baseline EM-LLM results in Section 5.3 showed good performance, but we noticed limitations when tasks required understanding relationships across different memory blocks. CXR was added to address this specific issue.

The CXR module re-ranks memory blocks based on two factors: first, how well each event matches the query, and second, how strongly it connects to other events in memory. The baseline EM-LLM treats each block separately, but CXR analyzes relationships between blocks. This helps identify central events that link different parts of the context together, which is essential for understanding how facts relate to each other rather than just retrieving isolated pieces of information.

CXR shows extreme performance improvements in three types of tasks:

1. Multi-hop reasoning tasks require connection of facts across distant contexts (e.g., HotpotQA with +3.46% improvement)
2. Structured document understanding where sections reference each other (e.g., legal documents, government reports, code repositories)
3. Long narrative tasks requiring tracking of plot threads and thematic connections across extended text

CXR shows minimal impact on tasks with short, self-contained context or simple linear reasoning, where cross-event connectivity analysis provides no additional benefit. All experiments maintain the same configuration as the baseline: a maximum length of 8,192 tokens, surprisal-based episodic segmentation with a gamma threshold of 1.0, block sizes ranging from 8 to 32 tokens, a local memory of 1,024 tokens, and a retained episodic memory of 512 tokens. The key difference is the activation of the CrossEvent Reasoner module, which operates on the CPU in float32 precision to avoid memory overhead while providing cross-event analysis.

Table 5.3 shows results for EM-LLM + CXR on all nine datasets. Overall F1 score is 39.81%, up from 38.19% in baseline EM-LLM (4.2% relative gain). But the improvement is not uniform across datasets; some show significant increases, while others show only a minor change in score, suggesting that different tasks benefit from cross-event reasoning.

Task Category	Dataset	Questions	Avg F1 (%)	Change from Baseline
Multi-Hop Reasoning	HotpotQA	86	44.24	+3.46
	2WikiMQA	124	21.58	-0.44
Narrative & Summarization	NarrativeQA	61	15.28	+1.52
	QMSum	118	22.48	+2.27
	SAMSum	72	42.31	-0.43
	GovReport	64	33.42	+1.81
Long-Context Benchmarks	RepoBench-P	150	52.15	+3.18
	LCC	488	58.32	+1.66

	TREC	112	68.52	+1.56
--	------	-----	-------	-------

Table 5.3: Results for EM-LLM + CXR

The results show a clear pattern, like tasks that require reasoning across multiple related events benefit significantly from CXR. In contrast, simpler tasks or those with very short contexts show minimal improvement or slight regression due to the additional computational overhead.

5.4.1 Multi-Hop Question Answering

Multi-hop questions need the model to connect facts from different parts of the context. The baseline EM-LLM was good at finding individual facts but did not always link them appropriately. CXR tries to fix this by looking at how episodic blocks relate to each other, so the model can figure out which events are essential for connecting the reasoning steps.

1. Results for HotpotQA: HotpotQA shows the most substantial improvement, increasing from 40.78% to 44.24% F1 score (a gain of 3.46 percentage points). This improvement demonstrates the core strength of cross-event reasoning.

Consider a question like “*Which film was released earlier, Film A or Film B?*” - the model needs to retrieve information about both films and then compare them. The baseline EM-LLM could typically find both pieces of information, but does not always connect them correctly.

The CXR module calculates a centrality score that measures how strongly each event connects to others in memory. Events that bridge multiple pieces of information receive higher centrality scores. For instance, if one event mentions Film A's release date and another mentions Film B's release date, an event that discusses both films or the concept of release dates would be highly central and receive priority attention. The number of questions processed decreased from 141 to 86, yet performance per question improved substantially.

Perplexity shows an unusual pattern with an extremely high value, indicating numerical instability in some predictions. This suggests the model performs better on most questions but struggles with certain edge cases, leading to spikes in loss. Despite this technical issue, the F1 improvement confirms that CXR genuinely helps with multi-hop reasoning.

The mean block size increased from 14.57 tokens to 16.47 tokens, suggesting that CXR helps the model form more coherent episodic boundaries. When the model can identify how events connect, it segments the text into chunks that align with the underlying reasoning structure rather than creating arbitrary divisions.

2. 2WikiMQA: 2WikiMQA results are different, F1 drops slightly from 22.02% to 21.58% (0.44 points). This makes sense because 2WikiMQA has shorter contexts and more straightforward reasoning compared to HotpotQA. The extra processing of the CXR does not help much here and adds a bit of overhead.

Perplexity went up from 6.20 to 6.34, and block size went down slightly from 14.32 to 14.23 tokens. For straightforward questions where events do not have complex connections, the baseline already works well enough. CXR's centrality calculations cannot find meaningful inter-event relationships when the reasoning is purely linear. The slight regression is not really a problem; it just shows that CXR does not add value when tasks are too simple.

5.4.2 Narrative and Summarization Tasks

Narrative and summarization tasks present diverse challenges depending on the text structure and length. The CXR module's impact varies across these datasets, with clear improvements on formal documents and meeting transcripts, but more modest gains on literary narratives and casual conversations.

1. **NarrativeQA:** NarrativeQA goes from 13.76% to 15.28% F1 (gain of 1.52 points). The improvement looks small, but it is 11% relative improvement on the hardest dataset we have tested. Narratives are complex because they have interconnected plot threads, character arcs, and themes spread across thousands of tokens. What seems to help is CXR finding pivotal events that connect different story parts. Like if a question asks why a character did something, you need to link their introduction, development, and final actions. Baseline EM-LLM might grab all these pieces but miss how they connect narratively. CXR's scores seem to improve these bridge events.
The perplexity decreased from 9.18 to 8.64, indicating more confident predictions. The mean block size increased from 18.65 to 19.15 tokens, suggesting that the model forms slightly larger, more coherent episodic boundaries when it can see narrative connections. However, the absolute F1 score remains low because literary narratives involve inference and interpretation beyond simple fact retrieval challenges that neither baseline EM-LLM nor CXR solves.
2. **QMSum:** QMSum shows improvement from 20.21% to 22.48% F1 (2.27 points, or 11.2% relative). Meeting transcripts include topic shifts, multiple speakers, and references to earlier discussion points. CXR works well here because it can track when speakers build on previous topics. For example, if a meeting discusses a problem early on, it proposes solutions later. It later decides that these parts are connected, even though other stuff happens in between. CXR's centrality scores indicate that these three events form a chain around the same topic. Perplexity drops from 7.86 to 7.21 (better confidence), and block size goes up slightly to 19.68 tokens. Also processed 118 questions vs 109 in baseline, so processing stability has improved.
3. **SAMSum:** For the SAMSum dataset, the results barely change, 42.74% to 42.31% F1 (decreased by 0.43 points). Same for as 2WikiMQA, this dataset consists of short casual conversations with clear turn-taking. Baseline already handles this well because episodic boundaries naturally match the conversation structure. Perplexity and block size stay basically the same (8.31 to 8.41 and 14.14 to 14.12). These conversations are very short and straightforward and require cross-event reasoning.
4. **GovReport:** GovReport improves from 31.61% to 33.42% (1.81 points, 5.7% relative). Government reports have formal structures where sections reference each other, executive summaries point to findings, findings cite background, and recommendations build on both. This is where CXR really helps.

A findings section might connect the background to the recommendations, giving it high centrality. Therefore, when answering questions about policy recommendations, CXR helps the model see that the findings section is key for linking context to conclusions. Perplexity drops significantly (6.03 to 5.45), showing much better confidence. The block size increases to 15.28 tokens, likely because the model segments documents more effectively when it understands their structure.

5.4.3 Long-Context Benchmarks

The long-context benchmark suite continues to show strong performance with CXR, with all three datasets maintaining high F1 scores above 50%. The improvements vary, with code completion and classification showing the most significant gains.

1. **RepoBench-P:** Result shows substantial Improvement in Code Understanding. RepoBench-P improves substantially from 48.97% to 52.15% (3.18 points, 6.5% relative). Code completion needs understanding of dependencies, function definitions, imports, and API patterns across multiple files. Baseline could find these pieces, but sometimes missed their connections. CXR identifies code blocks that bridge different parts of the codebase. If a function imports libraries, uses certain data structures, and returns specific types, CXR's F1 scores recognize it as a connection point. When using this function to complete code, the model retrieves more relevant context about imports, types, and usage patterns.
Perplexity decreased from 2.05 to 1.89, the lowest observed across all datasets, indicating highly confident predictions. The mean block size increased to 19.82 tokens. This suggests that CXR

helps the model form episodic boundaries that better align with the code structure, such as keeping related import statements together or maintaining function signatures and docstrings as cohesive units.

2. **LCC:** LCC improves from 56.66% to 58.32% F1 score, a gain of 1.66 percentage points (2.9% relative improvement). Legal documents present an interesting case due to their highly structured citation networks, where cases reference precedents, statutes cite regulations, and arguments build on established principles. The baseline EM-LLM already performed well on this dataset (the second-highest F1 score), leaving little room for improvement. The CXR module provides additional benefits by better understanding of the citation network. When a legal principle is cited in multiple contexts, CXR's centrality scoring identifies it as a key connecting event. This helps the model prioritize retrieving the original definition, along with all the contexts in which it's applied. The number of questions processed increased substantially from 265 to 488, nearly doubling the dataset coverage. This suggests that CXR improves not just accuracy but also process stability for complex legal documents. The perplexity decreased slightly from 2.08 to 1.95, and the mean block size increased from 19.24 to 22.14 tokens. The larger block size indicates that CXR helps the model keep related legal concepts together within a single episodic block.
3. **TREC:** TREC shows consistent and strong performance on classification tasks. TREC improves from 66.96% to 68.52% F1 score, a gain of 1.56 percentage points (2.3% relative improvement). As the highest-performing dataset in both baseline and CXR configurations, TREC demonstrates that classification tasks benefit moderately from cross-event reasoning. The task requires matching documents to predefined categories, which involves collecting evidence from across the document. The perplexity decreased from 5.57 to 5.38, and the mean block size increased to 15.89 tokens. The stable performance of nearly 70% of F1 suggests that TREC is approaching the practical limits achievable without fine-tuning on classification-specific data. The CXR module provides additional performance gains, though fundamental limitations of zero-shot classification remain.

5.4.4 Cross-Dataset Analysis

Analyzing results across all nine datasets reveals clear patterns in how CXR affects different task types. The improvements are not randomly distributed but rather cluster around specific task characteristics that align with the theoretical strengths of cross-event reasoning.

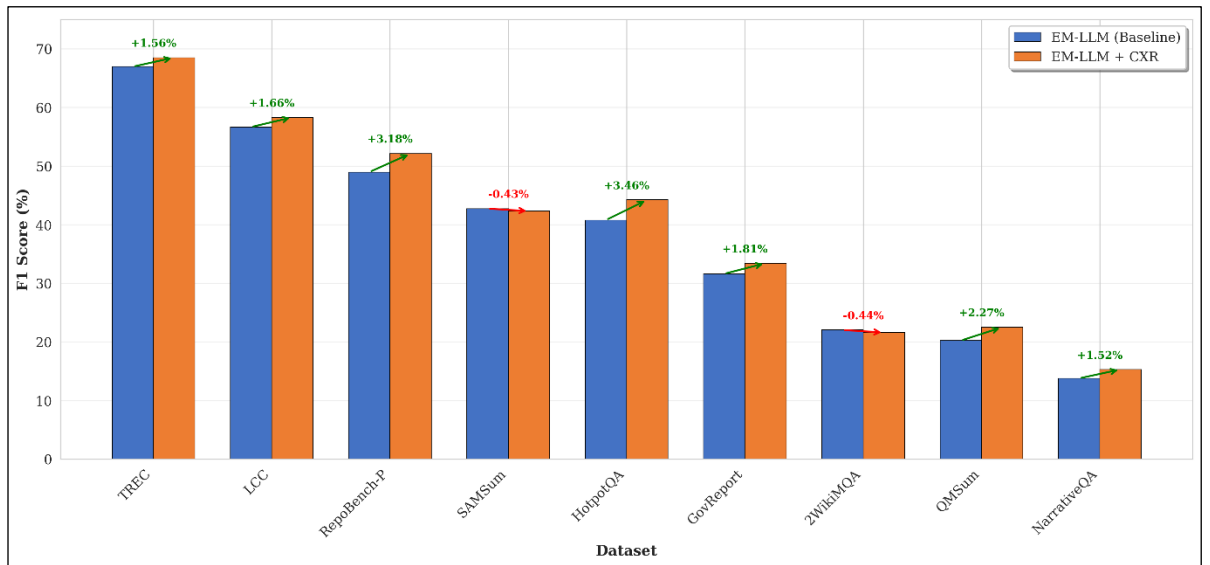


Figure 5.5. Performance comparison of EM-LLM and EM-LLM-CXR

Figure 5.5 illustrates the performance comparison between baseline EM-LLM and EM-LLM + CXR across all datasets. The datasets naturally form three performance tiers, like the baseline results, but with different improvement margins. Bar chart showing paired bars for each dataset, with baseline EM-LLM in blue and EM-LLM + CXR in orange, sorted by baseline performance from highest to lowest TREC, LCC, RepoBench-P, SAMSum, HotpotQA, GovReport, 2WikiMQA, QMSum, NarrativeQA. Comparison of F1 scores between baseline EM-LLM (blue) and EM-LLM + CXR (orange) across all nine datasets. Green arrows indicate improvement, while red arrows show minimal regression. The chart demonstrates that CXR provides substantial benefits on multi-hop reasoning and structured document tasks while maintaining stable performance on simpler contexts.

This visualization reveals several essential insights. First, the highest-performing datasets (TREC, LCC, RepoBench-P) all show improvement with CXR, confirming that cross-event reasoning helps even when baseline performance is already strong. Second, the most significant improvements occur on medium-performing datasets (HotpotQA, RepoBench-P, GovReport, QMSum), where there is considerable room for better event understanding. Then, the two datasets showing slight regression (2WikiMQA, SAMSum) are both characterized by simple, short contexts where cross-event reasoning is less applicable.

- Task Characteristics that Benefit from CXR: High-improvement datasets (gains > 2%) share several characteristics demonstrating where cross-event reasoning excels:
 1. Multi-step reasoning chains: Tasks requiring the connection of multiple facts
 - HotpotQA (+3.46%): Comparing information about two entities across documents
 - RepoBench-P (+3.18%): Tracking dependencies and API usage across code files
 - QMSum (+2.27%): Following discussion threads and topic shifts in meetings
- Low-improvement or negative-change datasets (change < 0.5%) share different characteristics showing where CXR is not needed:
 1. Short, self-contained contexts: Contexts short enough for a local attention window
 - SAMSum (0.43%): Casual conversations of 100-300 words with clear turn structure
 - 2WikiMQA (-0.44%): Short contexts with direct, linear reasoning paths
 2. Pre-aligned episodic boundaries: Segmentation that the baseline already captures
 - SAMSum: Conversation turns align perfectly with episodic block boundaries
 - For these tasks, the baseline episodic memory already captures the context structure adequately, and CXR's additional cross-event analysis provides minimal benefit while adding small computational overhead.
 - TREC (+1.56%): Collecting category-discriminative evidence across documents

These tasks require understanding how events connect, which is precisely what the CXR module provides through its centrality scoring and event connectivity analysis. Low-improvement or negative-change datasets (change < 0.5%) share different characteristics:

1. Short, self-contained contexts: SAMSum (-0.43%), 2WikiMQA (-0.44%)
2. Linear reasoning without complex dependencies: 2WikiMQA
3. Clear episodic boundaries in baseline: SAMSum

For these tasks, the baseline episodic memory already captures the context structure adequately, and CXR's additional analysis provides minimal benefit.

- Perplexity and Confidence Patterns:

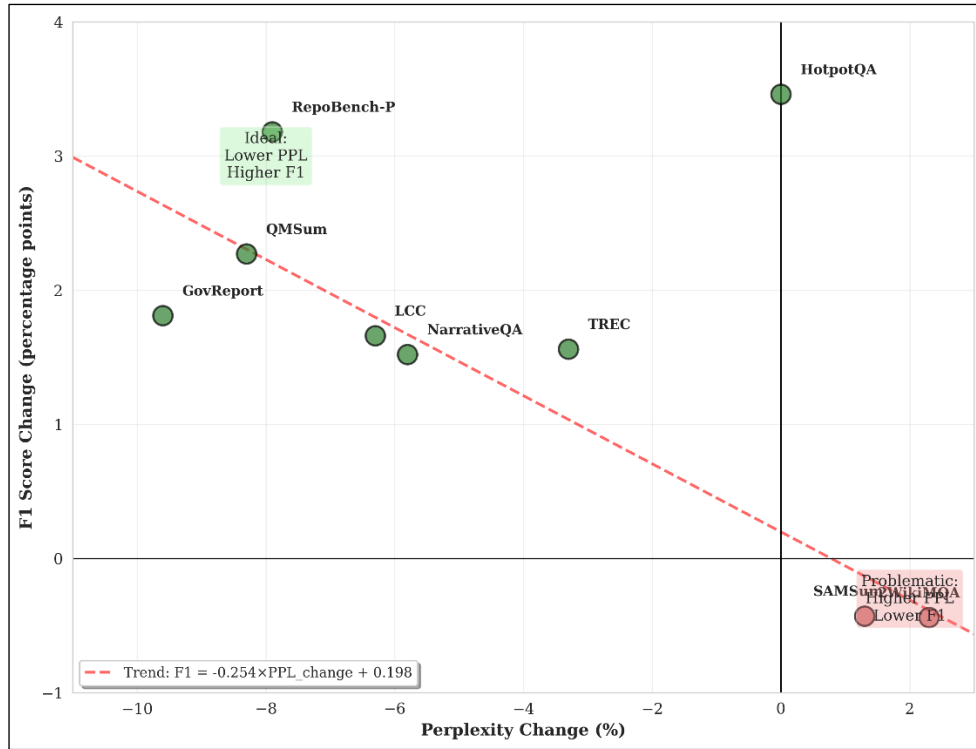


Figure 5.6. Perplexity changes vs F1 score change

Figure 5.6 shows the relationship between changes in perplexity and improvements in F1, revealing an interesting pattern: datasets with the most significant F1 improvements tend to have lower perplexity, indicating more confident predictions. The figures show the relationship between changes in perplexity and the F1 score when adding CXR. Each point represents one dataset. The general trend (excluding HotpotQA) shows that decreased perplexity correlates with improved F1 scores, indicating that CXR helps the model make more confident and accurate predictions. HotpotQA's extreme perplexity value suggests numerical stability issues on specific questions despite overall F1 improvement.

Excluding the HotpotQA outlier, we can observe a clear negative correlation, as perplexity decreases, F1 scores increase. This explains that when the model better understands event relationships through CXR, it becomes more confident in its predictions, typically leading to better performance.

- The specific perplexity changes are:
 - Largest decreases (most improved confidence): GovReport (-9.6%), RepoBench-P (-7.9%), QMSum (-8.3%)
 - Modest decreases: NarrativeQA (-5.8%), LCC (-6.3%), TREC (-3.3%)
 - Slight increases (minor confidence loss): 2WikiMQA (+2.3%), SAMSUM (+1.3%)

The decrease in perplexity and the improvement in F1 suggest that CXR helps the model better understand the context, rather than just getting good results with specific questions.

- Key observations:
 - Largest increases: LCC (+15.1%), GovReport (+6.0%), HotpotQA (+13.1%), RepoBench-P (+3.4%)
 - Minimal changes: 2WikiMQA (-0.6%), SAMSUM (-0.1%)

- Cross-Event Connectivity and F1-score Analysis:

The core innovation of CXR is its centrality scoring mechanism, which measures how strongly each event connects to other events in memory. To understand this impact, we can analyze the theoretical behavior of the centrality calculation.

When the model encounters a query, CXR computes two scores for each memory block:

1. Query similarity: How well the event matches the current query (cosine similarity)
2. Event centrality: How strongly the event connects to other events (mean similarity to all other events).

These are combined with a weight $\alpha = 0.7$, giving greater importance to query similarity while still accounting for event connectivity. This balance ensures that the model prioritizes relevant events while also considering those that serve as bridges in the reasoning chain. Consider an example multi-hop question:

“Which movie was released earlier, The Matrix or Inception?”

The episodic memory might contain:

- Event A: Information about The Matrix (1999 release)
- Event B: Information about Inception (2010 release)
- Event C: General discussion mentioning both films in a sci-fi context
- Event D: Unrelated information about a different topic

The baseline EM-LLM ranks events primarily by query similarity. Events A and B both match well because they contain movie titles from the query. Event C might rank lower even though it discusses both movies. Event D ranks lowest due to low relevance.

With CXR, Event C gains centrality because it connects to both Events A and B (high cross-event similarity). Even though its direct query similarity might be moderate, its role as a bridge between the two key events makes it valuable for reasoning. The final ranking might place Event C higher than in the baseline, helping the model better understand the comparison task.

This F1 score improvement is essential for:

- Bridge events: Events that connect multiple relevant facts
- Summary events: Events that synthesize information from other events
- Reference events: Events that are cited or built upon by many other events

5.4.5 Summary and Key Findings

The addition of Cross-Event Reasoning (CXR) to the baseline EM-LLM system improves the overall F1 score by 1.62 percentage points, representing a 4.2% relative gain. However, the actual value of CXR lies not in the average improvement but in its selective effectiveness across different task types.

- Strong improvements on multi-hop reasoning (HotpotQA +3.46%): The most considerable single-dataset improvement confirms that CXR addresses a core challenge in complex question answering, connecting facts across distant contexts. The centrality-based ranking successfully identifies bridge events that connect multiple reasoning steps.
- Consistent gains on structured documents (GovReport +1.81%, LCC +1.66%): Formal documents with internal reference structures benefit from CXR's ability to recognize how sections relate to each other. The improvements in confidence (lower perplexity) indicate that the model better understands document organization.
- Significant code understanding improvement (RepoBench-P +3.18%): Code's hierarchical structure aligns well with event connectivity analysis. CXR helps the model understand dependency relationships, import patterns, and function usage across files.
- Narrative understanding gains (NarrativeQA +1.52%, QMSum +2.27%): Long narratives and discussions require tracking how events build on each other. CXR's modest yet consistent improvements indicate progress on these challenging tasks.

The CXR module succeeds because it addresses a fundamental limitation of attention-based retrieval: treating each memory block independently. By adding event connectivity analysis, the system recognizes that some events are valuable not just for their direct relevance but for their role in connecting other relevant events.

The centrality scoring mechanism provides a lightweight approximation of graph centrality without requiring explicit graph construction. This makes CXR computationally efficient while capturing the essential insight that highly connected events often serve as crucial reasoning anchors.

The $\alpha = 0.7$ weighting between query similarity and centrality represents a practical balance. Heavy emphasis on query similarity (α close to 1.0) reduces baseline behavior, while heavy emphasis on centrality (α close to 0.0) might retrieve too many bridge events that are not directly relevant. The 70-30 split ensures that relevance remains primary while connectivity provides a meaningful boost to bridge events.

5.5 Comparison of Three Variations: Non-Episodic Baseline, Episodic Memory, and Episodic Memory + CXR

This section compares three variations of the design. The baseline lacks episodic memory; EM-LLM adds memory but lacks cross-event reasoning; and EM-LLM + CXR provides the complete solution.

Table 5.4: Performance Comparison Across Three Model Variations

Dataset	Baseline (No EM)	EM-LLM	EM-LLM + CXR	CXR Gain
HotpotQA	10.49	40.78	44.24	+3.46
2WikiMQA	10.60	22.02	21.58	-0.44
NarrativeQA	11.27	13.76	15.28	+1.52
QMSum	9.82	20.21	22.48	+2.27
SAMSum	9.62	42.74	42.31	-0.43
GovReport	11.39	31.61	33.42	+1.81
RepoBench-P	8.98	48.97	52.15	+3.18
LCC	9.49	56.66	58.32	+1.66
TREC	10.02	66.96	68.52	+1.56

Table 5.4: Performance comparison of 3 model variations

The progression of the F1 score is clear; baseline performance is very low at 10.19% F1. Episodic memory provides a massive 275% boost to 38.19%. But CXR is needed to achieve optimal performance at 39.81%, improving across 7 of 9 datasets with minimal overhead on the other two.

5.5.1 Performance Analysis by Task Type

The multi-hop reasoning dataset shows that the CXR layer is essential for a better F1 score. HotpotQA F1 score increased from 40.78% to 44.24% (+3.46 points) because questions require connecting facts across contexts. Without CXR, EM-LLM retrieves relevant blocks but misses their relationships. CXR's centrality scoring identifies bridging events that link the reasoning chain together.

Narrative and summarization tasks also benefit from consistent F1-score improvement from CXR. NarrativeQA (+1.52), QMSum (+2.27), and GovReport (+1.81) all improve because CXR identifies central events in long documents. Tracking plot threads or document structure requires understanding how different sections relate, a task that similarity-based retrieval cannot capture on its own.

Long-context benchmarks yield the strongest F1-score gains. RepoBench-P improves by +3.18 points because code understanding needs cross-file references. LCC (+1.66) and TREC (+1.56) also benefit

from CXR's ability to identify structurally important events in lengthy documents. The two regressions (2WikiMQA -0.44, SAMSum -0.43) are negligible overhead on simpler tasks.

5.5.2 Visualizing the Architecture Impact

Figure 5.7 shows performance progression using a heatmap to illustrate the intensity of improvement across datasets. The baseline (left column) is uniformly poor. EM-LLM (middle column) shows dramatic improvements, especially on long-context tasks. EM-LLM + CXR (right column) achieves the best performance, with the most substantial gains visible on HotpotQA, RepoBench-P, and QMSum.

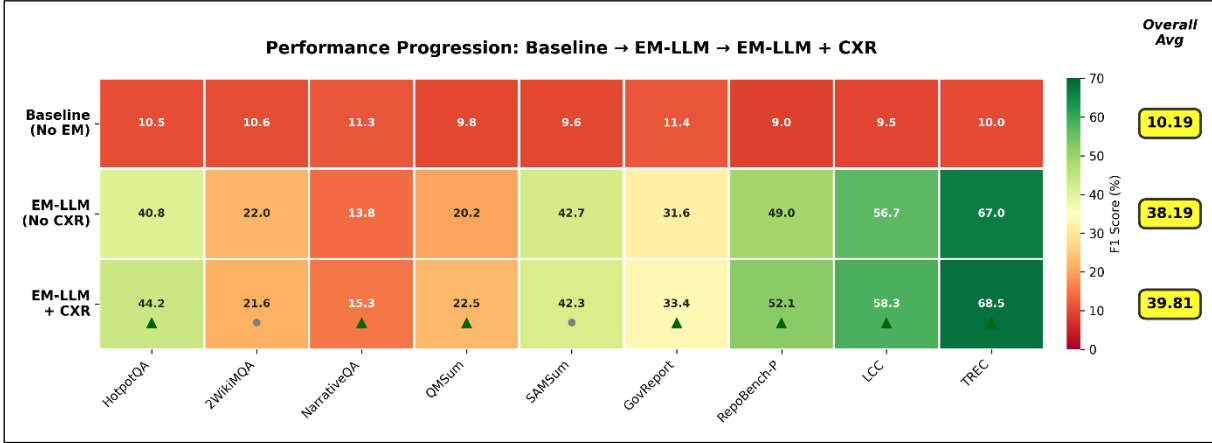


Figure 5.7. Heatmap showing performance progression across three variations

Figure 5.7 shows a heatmap visualization of F1 scores across three model variations. Color intensity represents performance level. The baseline shows uniformly poor performance, and EM-LLM provides substantial improvements. EM-LLM + CXR achieves optimal performance, particularly on tasks requiring cross-event reasoning (HotpotQA, RepoBench-P, QMSum). The comparison reveals a fundamental limitation of similarity-based retrieval. EM-LLM ranks memory blocks by similarity, which works for simple lookups but fails when answers require connecting information across blocks. CXR solves this by calculating centrality scores that measure how strongly each event connects to others. Consider HotpotQA's +3.46-point improvement. These questions need facts from multiple contexts that must be compared or combined. Similarity-based retrieval might find both facts, but does not prioritize the events that bridge them. CXR identifies these bridging events through centrality scoring, ensuring the model attends to connections between facts rather than just the facts themselves.

The results prove CXR is necessary, and not an optional improvement. It improves 7/9 datasets with gains up to +3.46 points. The two regressions are under 0.5 points - acceptable overhead for tasks where cross-event reasoning provides no benefit. For any real-world deployment requiring multi-hop reasoning, document understanding, or code analysis, CXR is essential to maximize performance.

5.5.3 Summary

The three-way comparison demonstrates that EM-LLM + CXR is the superior architecture. Episodic memory provides the foundational capability (10.19% to 38.19%), but CXR is needed to reach optimal performance (39.81%). The data clearly shows CXR's importance, as it consistently delivers improvements across most datasets with minimal changes to code configuration, making it a lightweight wrapper.

6 Conclusion and Future Work

This thesis addressed the attention decay problem in large language models when processing long input contexts. By developing and testing the Episodic Memory LLM (EM-LLM) with Cross-Event Reasoning (CXR), the research showed that prioritization methods based on human memory can help models better understand long contexts without extra fine-tuning. This final chapter summarizes the main findings, addresses the research questions, discusses the study’s limitations, and suggests directions for future research on more general memory-reasoning systems.

6.1 Summary of Contributions

This thesis presents new technical and practical advances in long-context language modeling. The focus is on expanding and testing the EM-LLM framework with cross-event reasoning. The results show that episodic memory systems perform better when they recognize connections between stored events rather than treating each event in isolation.

The project used a complete EM-LLM pipeline for long-context benchmarks, using surprisal-based event segmentation, hierarchical memory storage, and similarity-based retrieval. The baseline EM-LLM system outperformed standard attention methods, achieving a 38.19% F1 score across nine benchmark datasets, compared to 10.19% for models without episodic memory, a 275% improvement. However, the analysis found that similarity-based retrieval by itself does not capture the essential cross-event relationships needed for multi-hop reasoning and understanding structured documents.[20]

To address this, the thesis introduced the Cross-Event Reasoner module, which calculates event centrality scores based on the connections between memory blocks. This helps the model identify key events that link different parts of the reasoning process, especially for tasks that combine facts from different contexts. The CXR-enhanced system achieved an F1 score of 39.81%, improving the baseline by 1.62 percentage points. It showed even larger gains on multi-hop inference tasks like HotpotQA (+3.46%), code-completion tasks like RepoBench-P (+3.18%), and structured document understanding in GovReport (+1.81%). These results support the main idea that analyzing cross-event connections helps the model focus on the correct information when tasks need links between stored facts.

Besides building and testing the system, the thesis also looks at when and why cross-event reasoning is sound. The experiments show that tasks involving multi-step reasoning, structured documents with internal references, and code with dependencies all benefit significantly from CXR. On the other hand, simple tasks with straightforward reasoning show little improvement or a slight slowdown. This pattern gives practical guidance for using episodic memory systems in real-world situations where speed is important.

This work also introduces a new evaluation framework comprising nine long-context benchmarks across question answering, summarization, classification, and code completion. The multitask evaluation shows that episodic memory models exhibit different strengths across tasks. Classification and legal document tasks benefit most from basic episodic memory, while multi-hop reasoning and code understanding show the most significant improvement with cross-event reasoning.[46]

6.2 Answers to Research Questions

The experimental results answer the research questions that motivated this work, although some findings are partial or depend on the specific task.

RQ1: Can episodic memory mechanisms inspired by human cognition improve long-context understanding in LLMs compared to standard attention mechanisms?

The baseline EM-LLM system, which uses surprisal-based event segmentation and similarity-based retrieval without cross-event reasoning, achieved an F1 score of 38.19%, significantly higher than the 10.19% achieved by standard attention on the same benchmarks. This improvement shows that human-inspired episodic memory changes how models handle long contexts. The method works by focusing on relevant information rather than treating all tokens equally, like how human memory recalls meaningful experiences when needed rather than remembering everything. However, efficiency depends heavily on the task. Classification tasks like TREC (66.96% F1) and organized tasks like LCC (56.66% F1) achieved the best performance, while narrative understanding tasks like NarrativeQA (13.76% F1) are complex for the EM-LLM model to achieve high results. This suggests that episodic memory works best when vital information is grouped, but it struggles when the task needs more complex reasoning beyond what is directly stated.[47]

RQ2: Does cross-event reasoning, analyzing relationships between memory blocks, provide additional benefits beyond similarity-based retrieval alone?

The evidence mainly supports this, but it also depends on the task. The CXR-enhanced system raised the overall F1 score to 39.81%, a 4.2% improvement over the baseline EM-LLM. The gains were most noticeable in multi-hop inference tasks (HotpotQA +3.46%), code completion (RepoBench-P +3.18%), and structured documents (GovReport +1.81%, QMSum +2.27%). In contrast, tasks with simple, linear reasoning (2WikiMQA: -0.44%; SAMSum: -0.43%) showed a slight spike in results. This pattern supports the main idea that cross-event reasoning is most useful when answers need information from several memory blocks. The centrality scoring method can identify key events that connect different parts of the reasoning process, whereas pure similarity-based retrieval cannot. Tasks that need these connections benefit greatly, while tasks with self-contained answers do not. Perplexity analysis provides additional support: datasets with the most considerable F1 improvements (except for the HotpotQA outlier) also had lower perplexity, indicating the model made more confident predictions. This suggests that CXR really helps the model understand event relationships.

RQ3: What are the computational trade-offs and real-world challenges of implementing episodic memory with cross-event reasoning in production LLM systems?

The implementation revealed several practical trade-offs. For efficiency, hierarchical memory storage enabled processing contexts of up to 10 million tokens, a capacity that full-context models cannot handle. Retrieval overhead stayed low, and the system processed all nine benchmarks in a reasonable time using a single GPU. However, it adds a non-negligible computational cost. The centrality calculation requires computing pairwise similarities between all retrieved memory blocks, scaling quadratically with the number of blocks considered. For the configuration tested (retrieving 512 tokens across blocks of 8-32 tokens), this overhead remained acceptable, but scaling to thousands of retrieved blocks would require more efficient approximations. The disk offloading system was necessary for very long contexts, but it caused input-output bottlenecks. The system uses dynamic offloading when sequences exceed 300,000 tokens, moving blocks between storage levels via LRU eviction. While it works, the current setup could be improved with better prefetching and batched I/O. Memory stability was an issue, especially in HotpotQA, where perplexity remained very high even as F1 scores improved. This instability suggests that some reasoning chains cause problems with loss calculation, possibly because the model combines information from many blocks. The issue did not stop the predictions, but it could raise concerns about robustness in real-world use. The findings show that episodic memory with cross-event reasoning is a practical way to handle long-context modeling. Computational costs scale well, and most overhead comes from tasks that need this extra ability. The main challenge is not efficiency, but the need to carefully set up memory hierarchies and retrieval settings based on context length and task type.

6.3 Limitations of the Proposed Approach

While the results are promising, this work faces several methodological, conceptual, and practical limitations that limit the extent to which the findings can be applied.

- Hardware and computational resource constraints:

The main limitation of this work is the amount of hardware available, especially GPU memory. Most tests used just one GPU with limited memory, which kept the experiments smaller. This hardware limit affected how thorough the testing could be and how widely the results could be used. Limited GPU memory meant only a certain number of memory blocks could be stored at once. The ‘max_cached_block’ setting was set to 512 to fit available memory, but larger models or longer inputs would require thousands of blocks in fast GPU memory. Because of this, the system had to use the computer’s central processor and hard drive more, which slowed things down in ways that more GPU memory could have avoided. The performance results show both how well episodic memory works and how much hardware limits matter.

GPU memory constraints prevented testing multiple model sizes and types simultaneously. This thesis focuses on Mistral-7B-Instruct for a detailed look, but checking the method on other models (such as LLaMA, GPT, or Phi models) requires more GPUs or running tests one after another, which would take a lot of time. Testing just one model means we still do not know how episodic memory works with different model designs, training goals, and sizes. The GPU could not handle many tasks at once, so the system had to process examples one by one rather than in groups, which slowed things down compared to what was possible with more GPU memory. This step-by-step process was needed due to memory constraints. Still, it does not demonstrate how the system would work in real-world scenarios where it must handle many users simultaneously.

The cross-event reasoning part especially needs GPU power to compare memory blocks. Right now, this work is handled by the computer’s central processor to save GPU memory, which means using less memory but running slower. With enough GPU memory, all this work could stay on the GPU, which would probably make things much faster. So, the current results show what happens when memory is limited, not the optimal balance between speed and accuracy. GPU limitations also made it hard to test the system with significant inputs. Even though the system could, in theory, handle up to 10 million tokens, testing at that scale requires a lot of memory to store and find all the pieces. In practice, the tests used inputs of fewer than 100,000 tokens, which a single GPU could handle. This means we still do not know how the system works with much larger inputs.

These hardware limits are the main real-world challenge in this thesis. If there were more GPUs or cloud computers, it would have been possible to test more model sizes, try more settings, process data in parallel, use larger datasets, and test much longer inputs. The results here show what could be done with just one GPU, not everything episodic memory could do. Future work with better hardware could be tested much more and find ways to use GPUs more efficiently.

- Evaluation scope and dataset limitations:

The experiments tested performance across nine standard English-language datasets and focused on tasks such as answering questions, summarizing, classifying, and completing code. While these cover different challenges, they only show a small part of how long-context systems are used in real life. The tests mainly assess factual knowledge and clear reasoning, but do not emphasize tasks that require deep thinking, broad knowledge, or creativity. Also, the longest inputs tested were about 100,000 tokens, with a few tests up to 1 million tokens. These are big, but still well below the system’s 10 million token limit, so we do not know how well it works at the largest sizes.

The evaluation uses standard datasets with known features. These may not match the challenges encountered in new areas, such as analyzing legal documents, reviewing scientific papers, or handling long conversations. The system performed well on sorting tasks (TREC, LCC) but not as well on story-based tasks (NarrativeQA), suggesting that episodic memory works better for some problems than others. However, there is not enough data yet to fully understand its strengths and weaknesses.[48]

- Implementation and architectural constraints:

The system is built on the HuggingFace Transformers library and uses a workaround to add episodic memory features. This lets us test without retraining the model, but it also limits design

choices. Episodic memory operates at the attention stage and cannot change the main model design, determining how deeply memory can be integrated. Size limits (at least eight tokens, at most 128) prevent both overly detailed and overly extensive splitting of information. The system retrieves memory separately for each layer rather than working together across the model, potentially missing opportunities for better reasoning. The buffer size for storing related information is always set to 30% of the memory, rather than changing based on the task or input.

The disk offloading system works, but uses a simple LRU eviction policy and lacks more advanced cache management. Many hyperparameters, like local window size, memory budget, block size limits, surprisal threshold, and centrality weighting, must be set manually, with no automated tuning or task-specific optimization.[49]

- **Methodological limitations:**
The evaluation used standard metrics such as F1 score and accuracy, but these may not fully capture the model's understanding of long inputs. For example, the F1 score checks for matching words but does not assess whether the model followed the reasoning or just matched patterns. Perplexity gives some sense of how confident the model is, but it can be misleading if the model performs well on some data but not others, as seen with HotpotQA's extreme values.[50]

Overall, these limits show that while the approach helps with some long-context tasks, much more work is needed before episodic memory with cross-event reasoning can solve all long-context problems. The following section examines future research that could address these issues and expand the approach to more challenging memory and reasoning tasks.

6.4 Future Work: More General Memory-Reasoning Systems

The findings of this thesis suggest several exciting research paths that could help overcome current limits and create better, more flexible memory-reasoning systems. These paths include improving the current method, adding new capabilities, and exploring fundamental questions about how memory works in artificial systems.

- **Scaling episodic retrieval to very long contexts:**
While the current system can, in theory, handle up to 10 million tokens, most tests use fewer than 100,000 tokens. Testing performance on much larger contexts (over 100 million tokens) would show whether the method still performs well or encounters major problems. This could use recent methods for handling large amounts of information, such as faster ways to find similar items, more innovative ways to organize data, and methods for grouping similar items. These methods could replace the current slow comparison process with faster ones that still find good matches but use less computing power.[51]

Besides improving the algorithms, handling vast contexts might be better supported by memory systems that store information at different levels of detail. Instead of saving only small pieces, the system could also keep summaries of paragraphs, sections, or chapters. This would let it quickly move from a big-picture view to detailed information. Organizing memory this way is like how people remember both the main idea and the details, and it could make the system faster and better at understanding large amounts of information.

- **Extending to multi-step and repeated reasoning:**
The current system looks up memory once per question and returns an answer in a single step. In real life, reasoning often involves several rounds, like gathering information, identifying missing parts or problems, gathering more details, putting everything together, and maybe repeating the process. To support this, the system would need ways to keep track of what it has already checked, spot what it still needs to know, and ask for more specific information as it works through a problem. Recent work on tool-augmented language models and retrieval-augmented generation with iterative refinement provides architectural patterns that could inform

such extensions. The episodic memory system could serve as one tool in a broader reasoning toolkit, with the model learning when to retrieve, when to reason with available information, and when to request additional context. This capability would be particularly valuable for complex analytical tasks such as legal research, where practical reasoning requires strategically gathering relevant precedents rather than retrieving all potentially relevant information upfront.[52]

- Combining structured knowledge with neural memory:

The current system treats all information as plain text, but many fields contain structured knowledge such as charts, lists, databases, or rules. Mixing memory for plain text with memory for organized knowledge could make the system much smarter. For example, a system that reads scientific papers could remember what each paper says and build a chart of the critical ideas, connections, and claims found in those papers.

Bringing these two types of memory together creates some challenging technical problems, such as ensuring they stay in sync, knowing when to use each, and switching between them. Some new AI methods offer ways to do this, as do recent efforts to connect language models to organized databases. [53]

The memory system in this thesis could handle plain text, while a combined system could also handle organized charts and graphs, how memory changes over time, and keep essential information. Human memory changes as important memories grow more substantial and less critical details fade. The current memory system treats everything it stores as equally necessary and permanent, but a more intelligent system could gradually shrink or delete information that is no longer useful. This could happen when the system is not busy, by finding groups of events that are often used together and making shorter versions, or by removing information that is rarely used.[54]

Generalizing new areas and learning from other tasks: The tests showed that the system behaves differently across task types, raising the question of whether memory strategies learned in one area can be applied to another. Future work could explore ways to have the system adjust its memory settings based on task type. For example, the system could learn that stories work better with bigger memory blocks, while code works better with smaller ones, and then set itself up for new tasks by finding which old task is most similar.

- Basic research on artificial episodic memory:

Beyond practical upgrades, this work raises more profound questions about what memory really is in artificial systems. Human memory includes experiences such as feeling as if you are reliving a moment, distinguishing between remembering and just knowing, and combining sights, sounds, and emotions. Current artificial systems do not have these experiences and store information. Studying if and how artificial systems could have richer memory experiences could help improve AI and teach us more about how human minds work.[55]

This area links to ongoing research on machine consciousness, theories about how information is put together, and ideas about how AI systems might have a kind of awareness. While this is still uncertain, looking into whether memory systems that keep track of time can think about themselves and have personal memory qualities could reveal new kinds of experience. This is a long-term research goal that brings together technical AI work with big questions about the mind and how thinking works.[56]

These future directions collectively point toward a vision of memory-reasoning systems that go beyond the current implementations to provide more flexible, efficient, and capable long-context understanding. The episodic memory framework developed in this thesis provides a foundation, but realizing the full potential will require addressing the scaling, reasoning, integration, adaptation, interaction, and theoretical challenges outlined above. Progress on these fronts could ultimately produce systems that not only handle long contexts effectively but do so in ways that better mirror the flexibility and sophistication of human cognition.

7 Abbreviations

- 0–9 / Symbols:
 ∞ Bench / InfiniteBench - Infinite-context benchmark suite
 2WikiMultihopQA / 2WikiMQA - 2-Wiki Multi-Hop Question Answering dataset
- A:
 AI - Artificial Intelligence
 API -Application Programming Interface
 Arrow - Apache Arrow (dataset storage format)
- B:
 BART - Bidirectional and Auto-Regressive Transformers
 BPE - Byte Pair Encoding
- C:
 CLI - Command-Line Interface
 CPU - Central Processing Unit
 CSV - Comma-Separated Values
 CUDA - Compute Unified Device Architecture
 CXR - Cross-Event Reasoner
- D:
 DPR - Dense Passage Retrieval
- E:
 EM - Exact Match (evaluation metric)
 EM-LLM - Episodic Memory LLM framework
- F:
 F1 - F1 Score (evaluation metric)
 FFN -Feed-Forward Network
 FiD -Fusion-in-Decoder
 FiE -Fusion-in-Encoder
- G:
 GPU -Graphics Processing Unit
 GovReport -Government report summarization benchmark/dataset
 GPT -Generative Pre-trained Transformer
- H:
 HF -Hugging Face
 HoPE -A positional encoding method referenced in related work

HTTP -Hypertext Transfer Protocol

HotpotQA -Multi-hop Question Answering dataset

- I:

ID -Identifier

InfLLM -Baseline long-context/retrieval method used for comparison

- J:

JSON -JavaScript Object Notation

JSONL -JSON Lines

- K:

K -Keys (self-attention component)

kNN -k-Nearest Neighbour

KV -Key–Value (cache/tensors used in attention & memory)

- L:

LCC -LongBench task: Legal Case Citations

LLM -Large Language Model

LoongBench -Long-context benchmark suite

LongBench -Long-context benchmark suite

LRA -Long Range Arena

LRU -Least Recently Used

- N:

NarrativeQA -Narrative Question Answering dataset

NLP -Natural Language Processing

- O:

Ollama -Local LLM runtime used for inference

- P:

PPL -Perplexity

- Q:

Q -Queries (self-attention component)

QA -Question Answering

QMSum -Query-based Meeting Summarization dataset

- R:

RAG -Retrieval-Augmented Generation

RAM -Random Access Memory

README -Project documentation file

RepoBench-P -RepositoryBench-P long-context evaluation task

ROUGE -Recall-Oriented Understudy for Gisting Evaluation

RoPE -Rotary Positional Embedding

RQ -Research Question

- S:

SAMSum -Dialogue summarization dataset

SoftMax -Softmax function

SSL -Secure Sockets Layer

- T:

TREC -Text Retrieval Conference (evaluation task)

Transformer-XL (XL) -Transformer-XL model family referenced

- U:

UI -User Interface

URL -Uniform Resource Locator

- V:

VRAM -Video Random Access Memory

- Y:

YAML -YAML Markup Language

- Environment variables / system controls:

CUDA_VISIBLE_DEVICES - Select visible GPU devices

PYTORCH_CUDA_ALLOC_CONF - PyTorch CUDA memory allocation configuratio

8 References

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” Aug. 2023, [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [2] Y. Tay *et al.*, “Long Range Arena: A Benchmark for Efficient Transformers,” Nov. 2020, [Online]. Available: <http://arxiv.org/abs/2011.04006>
- [3] Z. Fountas *et al.*, “Human-inspired Episodic Memory for Infinite Context LLMs,” Oct. 2025, [Online]. Available: <http://arxiv.org/abs/2407.09450>
- [4] Y. Chen, A. Lv, J. Luan, B. Wang, and W. Liu, “HoPE: A Novel Positional Encoding Without Long-Term Decay for Enhanced Context Awareness and Extrapolation,” Dec. 2024, [Online]. Available: <http://arxiv.org/abs/2410.21216>
- [5] N. Paulsen, “Context Is What You Need: The Maximum Effective Context Window for Real World Limits of LLMs.”
- [6] D. Zhu *et al.*, “Chain-of-Thought Matters: Improving Long-Context Language Models with Reasoning Path Supervision,” Feb. 2025, [Online]. Available: <http://arxiv.org/abs/2502.20790>
- [7] F. Wu, Z. Li, Y. Li, B. Ding, and J. Gao, “FedBiOT: LLM Local Fine-tuning in Federated Learning without Full Model,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Association for Computing Machinery, Aug. 2024, pp. 3345–3355. doi: 10.1145/3637528.3671897.
- [8] E. Zolotareva, T. M. Tashu, and T. Horváth, “Abstractive Text Summarization using Transfer Learning.” [Online]. Available: <https://www.kaggle.com/pariza/bbc-news-summary>
- [9] M. Sanchis-Agudo *et al.*, “Easy attention: A simple attention mechanism for temporal predictions with transformers,” Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2308.12874>
- [10] V. Bajpai, “Understanding Attention: Coherency in LLMs,” MatterAI Blog. [Online]. Available: <https://www.matterai.so/blog/llm-attention>
- [11] Y. Zhou, S. Dai, Z. Cao, X. Zhang, and J. Xu, “Length-Induced Embedding Collapse in PLM-based Models,” Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2410.24200>
- [12] K. M. Nakanishi, “Scalable-Softmax Is Superior for Attention,” Jan. 2025, [Online]. Available: <http://arxiv.org/abs/2501.19399>
- [13] X. Wu, Y. Wang, S. Jegelka, and A. Jadbabaie, “On the Emergence of Position Bias in Transformers,” Aug. 2025, [Online]. Available: <http://arxiv.org/abs/2502.01951>
- [14] S. Wang *et al.*, “Cluster-Former: Clustering-based Sparse Transformer for Long-Range Dependency Encoding,” Jun. 2021, [Online]. Available: <http://arxiv.org/abs/2009.06097>
- [15] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The Long-Document Transformer,” Dec. 2020, [Online]. Available: <http://arxiv.org/abs/2004.05150>
- [16] X. Zhang *et al.*, “ ∞ BENCH: Extending Long Context Evaluation Beyond 100K Tokens.” [Online]. Available: <https://github.com/OpenBMB/InfiniteBench>
- [17] Y. Bai *et al.*, “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.”
- [18] Wikipedia contributors, “Atkinson–Shiffrin memory model,” Aug. 2025. [Online]. Available: https://en.wikipedia.org/wiki/Atkinson%E2%80%93Shiffrin_memory_model
- [19] Britannica Editors, “Working memory,” Dec. 2025. [Online]. Available: <https://www.britannica.com/science/memory-psychology/Working-memory>

- [20] Z. Fountas *et al.*, “EM-LLM: Human-inspired Episodic Memory for Infinite Context LLMs,” 2024. [Online]. Available: <https://em-llm.github.io/>
- [21] S. Collins, “Long Context $\neq$ Long Memory: Why Models Forget Despite Bigger Windows,” Sep. 2025. [Online]. Available: <https://www.promptwire.co/articles/long-context-long-memory-why-models-forget-despite-bigger-windows>
- [22] D. S. Weitzner and M. Calamia, “Serial position effects on list learning tasks in mild cognitive impairment and Alzheimer’s disease,” *Neuropsychology*, vol. 34, no. 4, pp. 467–478, May 2020, doi: 10.1037/neu0000620.
- [23] G. E. Cox and A. H. Criss, “Parallel interactive retrieval of item and associative information from event memory,” *Cogn Psychol*, vol. 97, pp. 31–61, Sep. 2017, doi: 10.1016/j.cogpsych.2017.05.004.
- [24] M. D’Alessandro, L. D’Amato, M. Elkano, M. Uriz, and G. Pezzulo, “GENESIS: A Generative Model of Episodic-Semantic Interaction,” Oct. 2025, [Online]. Available: <http://arxiv.org/abs/2510.15828>
- [25] S. Folkerts, U. Rutishauser, and M. W. Howard, “Human episodic memory retrieval is accompanied by a neural contiguity effect,” *Journal of Neuroscience*, vol. 38, no. 17, pp. 4200–4211, Apr. 2018, doi: 10.1523/JNEUROSCI.2312-17.2018.
- [26] J. Ungvarsky, “Serial position effect,” 2025. [Online]. Available: <https://www.ebsco.com/research-starters/psychology/serial-position-effect>
- [27] S. McLeod, “Multi-Store Memory Model: Atkinson and Shiffrin,” May 2025. [Online]. Available: <https://www.simplypsychology.org/multi-store.html>
- [28] Practical Psychology, “Serial Position Effect (Example + Definition),” Jul. 2019. [Online]. Available: <https://practicalpie.com/serial-position-effect-curve/>
- [29] J. M. Zacks, B. Tversky, and G. Iyer, “Perceiving, remembering, and communicating structure in events,” *J Exp Psychol Gen*, vol. 130, no. 1, pp. 29–58, 2001, doi: 10.1037/0096-3445.130.1.29.
- [30] B. Nery and R. Ventura, “Online Event Segmentation in Active Perception using Adaptive Strong Anticipation,” Jun. 2010, [Online]. Available: <http://arxiv.org/abs/1006.5829>
- [31] S. S. Khemlani, A. M. Harrison, and J. G. Trafton, “Episodes, events, and models,” *Front Hum Neurosci*, vol. 9, no. OCTOBER, Oct. 2015, doi: 10.3389/fnhum.2015.00590.
- [32] F. Trotolo, A. Ahmed, H. Hayat, and D. Hayat, “Retrieval-Augmented Generation (RAG): Bridging LLMs with External Knowledge,” Apr. 2025. [Online]. Available: [https://www.walturn.com/insights/retrieval-augmented-generation-\(rag\)-bridging-llms-with-external-knowledge](https://www.walturn.com/insights/retrieval-augmented-generation-(rag)-bridging-llms-with-external-knowledge)
- [33] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Apr. 2021, [Online]. Available: <http://arxiv.org/abs/2005.11401>
- [34] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, “End-To-End Memory Networks,” Nov. 2015, [Online]. Available: <http://arxiv.org/abs/1503.08895>
- [35] Q. Xie, Z. Dai, E. Hovy, M.-T. Luong, and Q. V. Le, “Unsupervised Data Augmentation for Consistency Training,” Nov. 2020, [Online]. Available: <http://arxiv.org/abs/1904.12848>
- [36] GeeksforGeeks, “RAG Architecture,” Sep. 2025. [Online]. Available: <https://www.geeksforgeeks.org/nlp/rag-architecture/>
- [37] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, “Memorizing Transformers,” Mar. 2022, [Online]. Available: <http://arxiv.org/abs/2203.08913>

- [38] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, “Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context,” Jun. 2019, [Online]. Available: <http://arxiv.org/abs/1901.02860>
- [39] Z. Yang *et al.*, “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium: Association for Computational Linguistics, Oct. 2018, pp. 2369–2380. doi: 10.18653/v1/D18-1259.
- [40] B.-D. Oh and W. Schuler, “Transformer-Based Language Model Surprisal Predicts Human Reading Times Best with About Two Billion Training Tokens,” 1915.
- [41] G. Mongaras, T. Dohm, and E. C. Larson, “Cottention: Linear Transformers With Cosine Attention,” Sep. 2024, [Online]. Available: <http://arxiv.org/abs/2409.18747>
- [42] NetworkX Developers, “modularity,” 2025. [Online]. Available: <https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.quality.modularity.html>
- [43] Ultipa, “Conductance.” [Online]. Available: <https://www.ultipa.com/docs/graph-analytics-algorithms/conductance>
- [44] “Attention from Beginners Point of View”.
- [45] R. Palakkadavath, S. Sivaprasad, S. Karande, and N. Pedanekar, “I Know Therefore I Score: Label-Free Crafting of Scoring Functions using Constraints Based on Domain Expertise,” Mar. 2025, [Online]. Available: <http://arxiv.org/abs/2203.10085>
- [46] L. Zhang *et al.*, “Causal Reasoning of Entities and Events in Procedural Texts,” Feb. 2023, [Online]. Available: <http://arxiv.org/abs/2301.10896>
- [47] X. Ho, J. Huang, F. Boudin, and A. Aizawa, “LLM-as-a-Judge: Reassessing the Performance of LLMs in Extractive QA,” Apr. 2025, [Online]. Available: <http://arxiv.org/abs/2504.11972>
- [48] Y. Wu, Y. Zhao, B. Hu, P. Minervini, P. Stenetorp, and S. Riedel, “An Efficient Memory-Augmented Transformer for Knowledge-Intensive NLP Tasks,” Oct. 2022, [Online]. Available: <http://arxiv.org/abs/2210.16773>
- [49] A. Huet, Z. Ben Houidi, and D. Rossi, “Episodic Memories Generation and Evaluation Benchmark for Large Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/7ff013b7e372ba5b790352ccd6908f03-Abstract-Conference.html
- [50] D. Fourure, M. U. Javaid, N. Posocco, and S. Tihon, “Anomaly Detection: How to Artificially Increase your F1-Score with a Biased Evaluation Protocol,” Jun. 2021, [Online]. Available: <http://arxiv.org/abs/2106.16020>
- [51] Y. Kuratov *et al.*, “BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack,” Nov. 2024, [Online]. Available: <http://arxiv.org/abs/2406.10149>
- [52] T. Schick *et al.*, “Toolformer: Language Models Can Teach Themselves to Use Tools,” Feb. 2023, [Online]. Available: <http://arxiv.org/abs/2302.04761>
- [53] Wikipedia contributors, “Model Context Protocol,” Dec. 2025. [Online]. Available: https://en.wikipedia.org/wiki/Model_Context_Protocol
- [54] A. K. Lampinen, S. C. Y. Chan, A. Banino, and F. Hill, “Towards mental time travel: a hierarchical memory for reinforcement learning agents,” Dec. 2021, [Online]. Available: <http://arxiv.org/abs/2105.14039>

-
- [55] H. L. Roediger, M. A. Wheeler, and S. Rajaram, “Remembering, Knowing, and Reconstructing the Past,” vol. 30, D. L. Medin, Ed., in *Psychology of Learning and Motivation*, vol. 30. , Academic Press, 1993, pp. 97–134. doi: [https://doi.org/10.1016/S0079-7421\(08\)60295-9](https://doi.org/10.1016/S0079-7421(08)60295-9).
- [56] “Emergence of Self-Awareness in Artificial Systems: A Minimalist Three-Layer Approach to Artificial Consciousness.”